Contenido

Unidad 1: Ingeniería de software en contexto 7
Introducción a la ingeniería de software. ¿Qué es? 7
Software 7
Ingeniería de Software 7
Estado actual y antecedentes. La crisis del software. 7
Causas 7
Disciplinas que conforman la ingeniería de software. 8
Ejemplos de grandes proyectos de software fallidos y exitosos. 8
Proyectos fallidos 8
Proyectos exitosos 8
Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software. 8
Relación entre el ciclo de vida del proyecto y del producto 9
Clasificación de los ciclos de vida 9
Administración de proyectos dependiendo en ciclo de vida 11
Procesos de Desarrollo Empíricos vs. Definidos. 11
Procesos Definidos 12
Procesos Empíricos 13
Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software. 13
Ventajas y desventajas de c/u de los ciclos de vida. Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto. 13
Criterios para la selección de un modelo de proceso 13
Diferentes modelos de ciclo de vida 14
Componentes de un Proyecto de Sistemas de Información. 16
Proyecto 16
Administración de proyecto de software 16
La triple restricción – enfoque tradicional 17
Líder de Proyecto 18
Equipo de proyecto 19
Stakeholders 19
Plan de proyecto 19
Planes de Soporte 24
Causas de fracasos de proyectos 24
Vinculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software. 25
No Silver Bullet 25
Unidad 2: Gestión Lean-Agile de Productos de Software 29
Manifiesto Ágil/Filosofía Lean 29

Manifiesto ágil 29
Filosofía Ágil 32
Pilares del empirismo 32
¿Cuándo es aplicable ágil? 33
¿Por qué ir a ágil? 34
El triángulo ágil 34
Filosofía Lean 35
7 Principios Lean 35
Relación Lean-Ágil 37
Requerimientos en ambientes lean ágil 38
Introducción al Desarrollo ágil 38
Requerimientos en ambientes ágiles – User Stories 38
Fundamentación de los requerimientos ágiles 38
La triple restricción – enfoque ágil 39
BRUF – Big Requirements Up Front 40
Product Backlog 40
Requerimientos Just in Time 40
Fuente de requerimientos 41
Tipos de requerimientos 41
Principios ágiles relacionados con requerimientos ágiles 43
User Stories 43
Partes de la User Story 44
Ventajas 44
Desventajas 45
Product Backlog y las User Stories 45
Tamaño 45
Story Points 46
Niveles de abstracción 46
Modelado de Roles 47
Criterios de aceptación 47
Pruebas de aceptación 48
DoR (Definition of Ready) – Definición de Listo 48
INVEST Model 48
DoD (Definition of Done) – Definición de Hecho 49
Spikes 49
Investment Themes (Temas de Inversión) 50
Estimación en ambientes ágiles 51
Errores en las estimaciones 51

¿Por qué estimamos? 51
Proceso de estimación 52
Métodos utilizados para estimar 52
Problemas de estimaciones tradicionales 53
Estimaciones en ambientes ágiles 53
¿Qué se estima? 55
Tamaño vs. Esfuerzo 55
Velocidad 55
Reestimación 56
Planificación 56
Tamaño 57
Story Point 57
¿Para qué se estima? 57
¿Cómo se estima la duración del proyecto? 58
Poker Planning o Poker Estimation 58
Estimaciones en Lean 59
Estimaciones en proyectos pequeños 59
Diferencias con las estimaciones en ambientes tradicionales 59
Frameworks de SCRUM a nivel de esquipo y escala 60
SCRUM 60
Herramientas de SCRUM 69
Frameworks para escalar SCRUM 71
Framework Nexus 71
Framework LeSS 73
Métricas ágiles y en otros enfoques 76
¿Qué es una métrica? 76
Dominio de métricas 76
Métricas en el enfoque tradicional 78
Métricas básicas para un proyecto de software 78
Métricas en ambientes ágiles 78
Métricas de software en Lean (KANBAN) 80
Métricas de Producto de Software 81
Resumen métricas en cada enfoque 81
Gestión de productos de software – Planificación de productos – herramientas para definición de productos de software 81
Evolución de los productos de software 82
Planificación del producto ágil 82
Design Thinking 84

Lean UX 85
Explicación MVP, MVF, MMF con el dinosaurio Lean/Agile 85
Productos mínimos para la gestión de productos 88
Relación entre MVP, MMF, MMP, MMR 89
Unidad 3: Gestión del Software como producto 91
Conceptos introductorios de la gestión de configuración 91
Introducción 91
Definición de Gestión de Configuración 91
¿Cuál es el problema que se está tratando de solucionar? 92
Ítem de Configuración 92
Versión 93
Variante 93
Configuración 93
Repositorio 94
Release 95
Rama (Branch) 95
El rol de las líneas base y su administración 95
Planificación de la gestión de configuración de software 96
Actividades relacionadas a la gestión de configuración 97
Identificación de Ítems de Configuración 97
Control de cambios 98
Auditorías de configuración 99
Informe de estado 101
Gestión de configuración en ambientes agiles 101
Continuous Integration 102
Continuous Delivery 102
Continuous Deployment – Estrategias de Deployments – Canary Deployments – Blue/Green Deployment 103
Unidad 4: Aseguramiento de calidad de Proceso y de Producto 104
Conceptos generales sobre calidad 104
Importancia de trabajar para y con Calidad. Ventajas y Desventajas. 104
Calidad 104
Gestión de calidad 104
Aseguramiento de calidad 104
Problemas en la calidad 105
Aseguramiento de calidad de Proceso y de Producto 105
Calidad en el desarrollo de Software 106
Principios de Calidad 106
Calidad para Quién – Visiones de Calidad 107

Calidad en el Software 108
Calidad del producto 108
Modelos de calidad de Producto 109
Calidad del Proceso 110
Estándares de Gestión de Calidad del Software: 111
Actividades relacionadas con el Aseguramiento de la Calidad del Software. 113
Administración de la calidad de Software 113
Principales Modelos de Calidad existentes (CMMI – SPICE – ISO) y sus métodos de evaluación. 114
Lineamientos para la implementación de modelos de calidad en las organizaciones. 114
Modelos para la mejora de procesos 114
Modelo de Calidad para evaluar procesos 116
Roles – Grupos 119
Relación de CMMI con Ágil 120
Diferentes tipos de Auditorias: Auditorías de Proyecto y Auditorías al Grupo de Calidad. 122
Auditorías de calidad de Software 122
Beneficios de las auditorías 122
Tipos de auditorías 123
Roles en una auditoría 124
Proceso de Auditorías: Responsabilidades. Preparación y ejecución. Reporte y seguimiento. 124
Etapas de una auditoría 124
Herramientas y técnicas utilizadas en auditorías 127
Resultados/hallazgos de una auditoría 127
Reporte de auditoría 127
Métricas de auditoría 128
Calidad de Producto: Planificación de pruebas para el software- Niveles y tipos de pruebas para el software. Técnicas y herramientas para probar software. Técnicas y Herramientas para la realización de revisiones técnicas del software. 128
Verificación y validación 128
Revisiones técnicas – Peer Review 129
Definición de estándares 132
Testing en ambientes Ágiles. 132
Contexto 132
Definición de Testing 133
Principios del Testing 133
Mitos del Testing 136
¿Cuánto Testing es suficiente? 136
Conceptos importantes 137
Niveles de prueba 139

Modelo en V 140
Ambientes de Testing 140
Proceso de pruebas 141
Artefactos de Testing 142
El Testing y el Ciclo de Vida 143
Estrategias de prueba 143
Tipos de prueba 149
Test Driven Development – TDD 150
Mejora continua de procesos con Kanban 150
Kanban 150
Definición 150
Prácticas de Kanban 151
¿Cómo aplicar Kanban? 152
Métricas de Kanban 153
Valores de Kanban 153
Principios directores 154
Principios fundacionales 154
Anexo 156
Diferencias entre Scrum y Kanban 156
Ser ágil y hacer ágil 157
Cuadro comparativo Gestión Tradicional vs. Gestión ágil 158
Preguntas frecuentes de la ingeniería de software (Sommerville) 159
Cuadro comparativo Auditorías vs. Revisiones técnicas 160
Cuadro comparativo CMMI por Etapas y Continuo 161
Respuesta a pregunta de parcial 162

Filosofía Lean

Es una filosofía o enfoque más viejo que agile, que nace en Japón (‘40) con Toyota, y busca maximizar el valor generado al cliente con el mínimo consumo de recursos, argumentando que todo el esfuerzo que no cree valor, se considera desperdicio. Esto permite reducir los esfuerzos en crear artefactos que no agreguen valor a un producto, concentrando los esfuerzos en los aspectos esenciales.

La filosofía “Lean”, conduce a una visión integrada de la cultura y la estrategia para atender al cliente final con alta calidad, bajo costo y tiempo de entrega, produciendo exactamente lo que el cliente final quiere, cuando lo quiere, dónde lo quiere, a un costo mínimo y precio justo. Siendo el cliente quien determina si el servicio o producto que la empresa entrega tiene valor o no.

Para esto utiliza el método Just in Time, el cual permite el intercambio del producto entre fases del proceso de desarrollo o a la entrega del producto final al cliente justo a tiempo, esto es poco antes de que lo usen y en la cantidad justa y necesaria, “analice cuando lo necesite, no antes”.

7 Principios Lean

Estos principios apuntan al desarrollo de productos en la industria de manufactura. Como el software es una actividad de desarrollo de un producto intangible, a los principios originales de Lean, hubo que adaptarlos al desarrollo de software (matrimonio Poppendieck en “Lean SW Development”).

1. **Eliminar desperdicios**

   Es quizás el más importante. Debemos tener un proceso que sea eficiente y que sume valor en todos los pasos del proceso, todo paso o parte que no contribuye a la generación de valor produce un desperdicio. Todo esto no quiere decir que el proceso es perfecto, porque según todas las filosofías de mejora continua (como esta o agile) marcan que siempre se puede mejorar el proceso.

   Lo que se busca evitar:

   - Producir funcionalidades en exceso, que luego no se usan (el 80% del valor está en el 20% de las funcionalidades [Pareto]).
   - Retrabajo.
   - Que los artefactos se vuelvan obsoletos antes de terminarlas.

   Este principio se relaciona con los principios agile 3 y 7 (entregar software funcionando frecuentemente y el software funcionando es la principal medida de progreso) y 10 (la simplicidad o maximizar el trabajo no realizado es esencial), porque en el software el trabajo parcialmente hecho y la funcionalidad extra son el desperdicio (en industrias tangibles sería el inventario).

   Los 7 desperdicios de Lean

   Se pondrán los desperdicios para software y entre corchetes para la industria manufacturera.

- **Características extras [producir productos extras]**

  No solo tiene que ver con las funcionalidades que se desarrollan demás, sino también con Up Front Specification (especificar anticipadamente), porque cuando se intenta satisfacer esos requerimientos en etapas posteriores de implementación, la información está obsoleta, incompleta o errónea, debido a que cuando se especificó el requerimiento había incertidumbre o falta de información.

  Esto se relaciona con el principio lean 4 (diferir compromisos hasta el último momento responsable).

- **Trabajo a medias [acumulación de stock]**

  Esto representa el trabajo que se realiza a medias (US, casos de uso, bugs, etc.) sin terminar, lo cual genera que deba ser retomado posteriormente para su finalización. Esto se relaciona con el

desperdicio de Cambio de Tareas. Por eso en agile se usa una gestión binaria (la unidad de trabajo está lista o no), ya que estimar un porcentaje de avance, es un desperdicio en sí.

- **Proceso extra**

  Esto se refiere a los pasos extras que se ejecutan en el proceso de desarrollo que no aportan valor al producto. El objetivo es identificarlos y eliminarlos, para no destinar esfuerzo a actividades que no aportan valor.

  Los procesos definidos sobrecargaron mucho a las personas de tareas que no aportan demasiado valor, y como resistencia surgieron los procesos empíricos, que buscan eliminar todos aquellos pasos en los procesos que no aportan valor al desarrollo del producto.

- **Búsqueda de información**

  Esto termina siendo un problema importante y difícil de manejar si no se implementa la disciplina SCM, porque no contamos con información suficiente y/o de fácil acceso para tener una trazabilidad sobre los ítems de configuración, lo que dificulta identificar el impacto de los cambios y ralentiza la resolución de errores.

- **Defectos**

  Son errores que se trasladan de una etapa a otra etapa posterior en la cual se introdujo. Esto provoca retrabajo, debido a que Testing debe "devolver" la funcionalidad a desarrollo para la resolución de defectos o bugs.

  Un antídoto a este problema es la disciplina de Aseguramiento de Calidad, que permite evitar esos errores a lo largo del desarrollo del producto.

- **Esperas [costos de logística]**

  Esto es el tiempo de espera que una persona tiene que pasar para poder realizar su trabajo. Ese tiempo de espera puede ser causado porque depende del trabajo de otra persona, porque requiere aprobación de un superior, etc. (por eso la importancia de equipos multidisciplinarios → para evitar tiempos de espera).

- **Cambios de tareas (Multitasking)**

  Como el desarrollo de software es un trabajo intelectual, uno de los desperdicios más grandes al hacer este tipo de trabajo, es el cambio de tareas o multitasking. Esto genera una pérdida de tiempo en prepararnos para realizar cada tarea (tiempo de seteo).

  Por eso en Kanban se hace uso de limitar el WIP (Work in Progress), porque busca reducir el multitasking. Estos dos últimos desperdicios tienen que ver con el JIT (Just In Time).

2. **Amplificar el aprendizaje**

   Tiene que ver con la transparencia y transformación de conocimiento implícito en explícito, lo cual fortalece al equipo de trabajo.

   Crear y mantener una cultura de mejoramiento continuo, donde los individuos intercambien sus experiencias y conocimientos para contribuir con el aprendizaje colectivo. Por otro lado, todo conocimiento que se genere debe compartirse para que sea fácilmente accedido por toda la organización.

   Se relaciona con los principios ágiles de (2) recibir requerimientos aún en etapas finales y el principio de (4) personas técnicas y no técnicas trabajando juntos todo el proyecto.

3. **Embeber la integridad conceptual**

   La calidad del producto no se negocia y la simplicidad es un factor clave (principios Agile): el producto tiene que satisfacer las expectativas y necesidades del cliente, esa es la principal prioridad y cada decisión se alinea con esto, no importa si el equipo por momentos se enfoca en otras cosas, no se debe perder de vista que esa es la prioridad principal. Para esto se deben encastrar todas las partes del producto que tengan coherencia y consistencia para hacer al producto más íntegro; donde los requerimientos que cubren la solución se observan como un todo cohesionados armónicamente.

4. **Diferir compromisos hasta el último momento responsable**

   Esto apunta a postergar la toma de decisiones lo suficientemente como para poder tener la mayor información necesaria para tomar esa decisión, pero a la vez, antes de que ese momento sea muy tarde (diferir la decisión irreversible).

   Esto se puede reflejar en Agile con el Product Backlog, donde nunca se tiene el 100% de los requerimientos, sino que se va completando a medida que se tiene más información sobre lo que se necesita implementar. Las mejores estrategias de diseño de software están basadas en dejar abiertas opciones de forma tal que las decisiones irreversibles se tomen lo más tarde posible. Se relaciona con los principios ágiles de maximizar el trabajo no hecho y de recibir cambios de requerimientos aun en etapas finales.

5. **Dar poder al equipo**

   Otorgar libertad de acción y poder de decisión al equipo. Para ello, el equipo debe ser multifuncional y autogestionado, y sus miembros deben estar capacitados y motivados. Relacionado con el principio 11 del Manifiesto Ágil ("Las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados").

   Esto implica no subordinar por parte de los superiores, debido a que esto anula las capacidades intelectuales, entrenar líderes, delegar decisiones y fomentar buena ética laboral. Pero a la vez, implica un compromiso por parte de los miembros del equipo, para desempeñarse y cumplir con lo pactado de forma completa y correcta.

   Este aspecto termina siendo un talón de Aquiles en las filosofías Agile y Lean, ya que en la práctica pocas veces se logran reunir todas estas características.

6. **Ver el todo**

   En Lean se busca tener una visión holística, que permita asociar y comprender el todo, el producto, el valor agregado que hay detrás, el servicio que brinda el complemento de los productos, más allá de los objetivos particulares por áreas o gerencias, porque la idea es que los objetivos particulares de las partes estén alineados con los objetivos globales.

7. **Entregar lo antes posible**

   La idea es entregarle al cliente software funcionando lo más rápido posible y que este producto sea útil para él. Se habla de entregarle software antes de que las necesidades de negocio cambien, porque el ambiente donde se desempeña el mismo cambia.

   Para esto Lean propone acotar ciclos de desarrollo (principio 1 de Agile: satisfacer al cliente con entregas tempranas) y entregar rápidamente incrementos pequeños de valor, lo que permite salir pronto al mercado con un producto mínimo que sea valioso (principio ágil de entregas tempranas y frecuentes de software de valor para el cliente).

Relación Lean-Ágil

Lean es previo al Manifiesto Ágil. Algunos principios de Agile heredan de los fundamentos de Lean. Ambos están orientados al cliente, a proveer el máximo valor posible. Para esto, entienden que los individuos de los equipos deben estar motivados, de forma tal que la sinergia de este facilite el desarrollo del proyecto. Esta sinergia, se interpreta como un valor agregado para el cliente. La flexibilidad para adaptarse a los cambios y ofrecer valor al cliente, es también un denominador común de ambos enfoques. Otro aspecto que tienen en común es generar productos de calidad y de mejora continua.

Combined Burn Chart

Visualiza cuánto trabajo queda para terminar y cuanto trabajo se ha realizado.

Frameworks para escalar SCRUM

Dado que Scrum impone un límite en la cantidad máxima de integrantes del Equipo de Desarrollo, surge la necesidad de escalar este Framework para poder gestionar productos de mayor envergadura, ya que las practicas agiles se utilizan en ambientes complejos para reducir tal complejidad.

Existen diferentes frameworks para escalar como Nexus, scale scrum, less, SAFe.

Framework Nexus

Nexus es un Framework que consiste en roles, eventos, artefactos y técnicas que vinculan el trabajo de aproximadamente tres a nueve equipos Scrum que trabaja sobre un único Product Backlog común para la construcción de un incremento integrado de producto "Terminado". Con un solo Product Owner para entregar un único producto.

Nexus surge para lidiar con la complejidad que supone varios Equipos Scrum trabajando sobre un mismo Product Backlog. Esta complejidad está dada por las siguientes dependencias entre equipos:

1. Requerimientos: el alcance de estos puede superponerse. La forma en que se implementan también puede afectar a los demás.
2. Conocimiento del dominio: el conocimiento del sistema de negocio debería mapearse a los Equipos Scrum para minimizar las interrupciones entre los mismos durante el Sprint.

Responsabilidades

- **Nexus Integration Team:** Responsable de asegurar que se produzca un incremento integrado al menos una vez en cada sprint. La integración incluye abordar las restricciones técnicas y no técnicas del equipo multifuncional que pueden impedir la capacidad de un Nexus para entregar constantemente un incremento integrado.

- **Product Owner:** Es responsable de maximizar el valor del producto y el trabajo realizado e integrado por los Scrum Teams en un Nexus, así mismo también es responsable de la gestión eficaz del Product Backlog.

- **Scrum Master:** responsable de asegurar que el marco de trabajo Nexus se entienda y se promulgue como se describe en la Guía de Nexus. Puede ser un Scrum Master en uno o más de los scrums teams en el Nexus.

  osea que puede desempeñar ambos roles a

- **Uno o más miembros del Nexus Integration Team:** a menudo está formado por miembros de los Scrum Teams que ayudan a los Scrum Teams a adoptar herramientas y prácticas que contribuyen a mejorar la capacidad de los Scrum Teams para entregar un Integrated Increment útil y de valor que cumple con la Definición de Terminado.

Eventos

Nexus agrega o extiende los eventos definidos por Scrum. La duración de los eventos Nexus se guía por la duración de los eventos correspondientes en la Guía de Scrum. Tienen definido un bloque de tiempo adicional a sus correspondientes eventos de Scrum.

- **Nexus Sprint:** Los scrums teams producen un solo Integrated Increment (Incremento de integración), es lo mismo que scrum.

- **Refinamiento entre equipos:** El Refinamiento Entre Equipos del trabajo del Product Backlog reduce o elimina las dependencias entre equipos dentro de un Nexus. El Product Backlog debe descomponerse para que las dependencias sean transparentes, se identifiquen entre equipos y se eliminen o se minimicen.
  El Refinamiento Entre Equipos del Product Backlog a escala sirve a un propósito dual:
  - Ayuda a los Scrum Teams a prever qué equipo entregará qué elementos del Product Backlog.
  - Identifica dependencias entre estos equipos.

  El Refinamiento Entre Equipos es continuo. La frecuencia, duración y participación del Refinamiento Entre Equipos varía para optimizar estos dos propósitos.

- **Nexus Sprint Planning:** El propósito de la Nexus Sprint Planning es coordinar las actividades de todos los Scrum Teams dentro de un Nexus para un solo Sprint. Los representantes apropiados de cada Scrum Team y el Product Owner se reúnen para planificar el Sprint.

  Resultados:
  - Objetivo de Sprint para cada Scrum team.
  - Objetivo de Nexus.
  - Un Nexus sprint backlog.
  - Un Sprint backlog.

- **Nexus Daily Scrum:**
  - Se discuten problemas de integración y se inspecciona el progreso hacia el objetivo de sprint del nexus.
  - Solo asisten representantes de cada Scrum team.
  - La Daily Scrum de cada Scrum Team complementa la Nexus Daily Scrum creando planes para el día, centrados principalmente en abordar los problemas de integración planteados durante la Nexus Daily Scrum.
  - Los Scrum teams no solo se reúnen o coordinan en el Nexus daily scrum, sino que durante el día, puede existir comunicación entre equipos con mayor detalle sobre adaptar o replanificar el resto del trabajo del sprint.
- **Nexus Sprint Review:** La Nexus Sprint Review se lleva a cabo al final del Sprint para brindar retroalimentación al Integrated Increment terminado que el Nexus ha construido durante el Sprint y determinar adaptaciones futuras.
  La nexus sprint review reemplaza a las sprint reviews de cada scrum team.

- **Nexus Sprint Retrospective:** El propósito de la Nexus Sprint Retrospective es planificar formas de mejorar la calidad y la eficacia en todo el Nexus. El Nexus inspecciona cómo fue el último Sprint con respecto a individuos, equipos, interacciones, procesos, herramientas y su Definición de Terminado. Además de las mejoras de cada equipo, las Retrospectivas de los Scrum Teams complementan la Nexus Sprint Retrospective mediante el uso de inteligencia de abajo hacia arriba para centrarse en los problemas que afectan al Nexus en su conjunto.

Artefactos

Representan trabajo o valor y están diseñados para maximizar la transparencia. El Nexus Integration Team trabaja con los Scrum Teams dentro de un Nexus para garantizar que se logre la transparencia en todos los artefactos y que el estado del Integrated Increment se entienda.

Cada artefacto contiene un compromiso, y estos existen para reforzar el empirismo y el valor de Scrum para el Nexus y sus interesados.

- **Product Backlog:** Hay uno solo que contiene la lista de todo lo que el Nexus y sus Scrum Teams necesitan para mejorar el producto. El Product Backlog debe entenderse con tal nivel que permita detectar y minimizar las dependencias. Es responsabilidad del Product Owner.
  El compromiso es el Objetivo del Producto, que describe el estado futuro del producto y sirve como un objetivo a largo plazo para el Nexus.

- **Nexus Sprint Backlog:** está compuesto del Objetivo del Sprint de Nexus y elementos del Product Backlog de cada Scrum Team del Nexus. Se usa para resaltar las dependencias y el flujo de trabajo durante el Sprint, y se actualiza durante el mismo a medida que se obtiene más conocimiento. Este debe tener un nivel de detalle tal que permita que Nexus pueda inspeccionar su progreso en la Nexus Daily Scrum.
  Su compromiso es el Objetivo de Sprint de Nexus, que es un único objetivo para Nexus. Es la suma de todo el trabajo y los Objetivos de Sprint de los Scrum Teams dentro del Nexus. Se crea en la Nexus Sprint Planning y se agrega al Sprint Backlog de Nexus. Los Scrum Teams lo tienen en cuenta a medida que trabajan. En la Nexus Sprint Review se debe de mostrar la funcionalidad de valor y útil que está terminada para alcanzar el Objetivo del Sprint.

- **Integrated Increment:** es la suma actual de todo el trabajo integrado completado por un Nexus en relación con el Objetivo del Producto. Se inspecciona en la Nexus Sprint Review pero puede entregarse antes de la misma. Debe cumplir con la definición de terminado.
  El compromiso de este artefacto es la Definición de Terminado, que define el estado del trabajo integrado cuando cumple con la calidad y las medidas requeridas para el producto. El incremento se termina sólo cuando está integrado, es de valor y utilizable. Es responsabilidad del Nexus Integration team una definición de terminado que se pueda aplicar en cada sprint, y todos los Scrum Team deben definir y adherirse a esta definición. Cada Scrum Team se autogestiona para llegar a este estado, pudiendo aplicar una definición de terminado más estricta pero nunca usar criterios menos rigurosos de lo acordado para el Integrated Increment.

Framework LeSS

LeSS es un marco de trabajo que permite escalar Scrum y se aplica a productos de entre dos a ochos equipos. LeSS propone un solo Product Owner, un Scrum Master que puede servir de uno a tres equipos, gestionando un único Product Backlog. El Sprint es común para todos los equipos. Si se trabaja con más de ocho equipos, se utiliza LeSS Huge.

Principios

- Scrum a gran escala es Scrum;
- Más con LeSS;
- Pensamiento sistémico;
- pensamiento Lean;
- Control empírico de procesos;
- Transparencia;
- Mejora continua hacia la perfección;
- Centrado en el consumidor;
- Enfoque de producto completo;
- Teoría de colas.

Miembros del Equipo

- **Scrum Master:** enseña Scrum y LeSS a la organización y los entrena en su adopción. Domina Scrum y LeSS y usa este conocimiento para guiar a todos a descubrir cómo pueden contribuir mejor a crear el producto más valioso, crea el ambiente para que las personas tengan éxito. El Scrum Master dedica todo su tiempo a cumplir ese rol y un Scrum Master sirve entre uno a tres equipos. Su enfoque es hacia los equipos, el Product Owner, la organización y las prácticas de desarrollo; se enfoca en todo el sistema organizacional no en un solo equipo.

- **Product Owner:** enseña Scrum y LeSS a la organización y los entrena en su adopción. Domina Scrum y LeSS y usa este conocimiento para guiar a todos a descubrir cómo pueden contribuir mejor a crear el producto más valioso, crea el ambiente para que las personas tengan éxito. El Scrum Master dedica todo su tiempo a cumplir ese rol y un Scrum Master sirve entre uno a tres equipos. Su enfoque es hacia los equipos, el Product Owner, la organización y las prácticas de desarrollo; se enfoca en todo el sistema organizacional no en un solo equipo.

- **Equipos:** un equipo en LeSS es el mismo que en Scrum de un solo equipo. El objetivo del equipo en LeSS es agregar ítems del Product Backlog al producto durante el Sprint. Los equipos trabajan en estrecha colaboración con los clientes/usuarios para aclarar la definición de los elementos y con el Product Owner en la priorización. Coordinan e integran su trabajo con el de los demás equipos, para final del Sprint haber producido un solo incremento del producto. Cada equipo tiene la responsabilidad de manejar las relaciones con los demás equipos.
  Cada equipo es autogestionado, multifuncional.

Eventos

- **Sprint Planning One:** es atendida por el Product Owner y los equipos o representantes de estos. Juntos seleccionan tentativamente a los ítems que cada equipo trabajará en ese Sprint. Los equipos identifican oportunidades para trabajar juntos y se aclaran las preguntas finales.

- **Sprint Planning Two:** es para que decidan cómo harán lis ítems seleccionados. Esto generalmente involucra el diseño y la creación de su Sprint Backlog.

- **Daily Scrum:** cada equipo tiene su propia Daily Scrum y no hay diferencia con las Daily Scrum para un solo equipo. Tiene una duración de 15 minutos. Y los miembros del equipo deben responder a tres preguntas.

    - ¿Qué hice ayer?
    - ¿En qué voy a trabajar hoy?
    - ¿Qué queda por hacer?

- **Sprint Review:** hay un review para el producto y es común para todos los equipos. Es el punto de inspección – adaptación y se realiza al final del Sprint; es la ocasión para todos los equipos de ver el incremento (el enfoque se pone en todo el producto). Durante esta ceremonia, clientes y Stakeholders examinan lo que los equipos construyeron durante el Sprint y se discute sobre cambios y se aportan nuevas ideas. Los equipos deben estar juntos y el PO, clientes7Stakeholders definen la dirección del producto. Es importante que los Stakeholders formen parte y aporten la información necesaria para una inspección y adaptación efectiva.

- **Retrospective**: Al final del Sprint, cada equipo individualmente realiza su propia Sprint Retrospective.

- **Overall Retrospective:** nueva reunión el LeSS. Su propósito es discutir problemas entre equipos organizacionales y sistémicos dentro de la organización. Algunas preguntas que se hacen son: ¿Qué tan bien

trabajan los equipos juntos?; ¿Hay algo que los equipos hagan que puedan compartir?; ¿Están los equipos aprendiendo juntos?

- **Product Backlog Refinement:** es necesario dentro de cada Sprint para refinar los elementos y estar listos para futuros Sprints. Las actividades claves son: dividir elementos grandes; aclarar elementos hasta que estén listos para la implementación y estimar tamaño, valor, riesgos. A esto lo realiza todo el equipo junto y no el PO por separado.

- **LeSS Sprint:** hay un sprint a nivel producto, no un Sprint diferente para cada equipo. Cada equipo comienza y termina el Sprint al mismo tiempo.

Artefactos

- **Incremento de producto potencialmente entregable:** la salida de cada Sprint es un incremento de producto potencialmente entregable. Es la forma de integrar el trabajo de todos los equipos al finalizar el Sprint. Potencialmente enviable es una declaración sobre la calidad del software y no sobre el valor o comerciabilidad del este. Si el producto se puede enviar realmente dependerá del Definition of Done.

  Hay un DoD común para todos los equipos.
  - Cada equipo puede tener su propia DoD más detallada, pero siempre expandiendo de la común.
  - El objetivo es mejorar la DoD para que resulte en su envío de producto en cada Sprint (o incluso con más frecuencia).

- **Product Backlog:** Todos los equipos construyen un solo producto en base a los ítems de un solo Product Backlog. Estos ítems no están preasignados a los equipos. La definición de producto debe ser tan amplia y centrada en el usuario final/cliente como sea práctico. Con el tiempo, la definición de producto podrá expandirse. El Product Backlog de LeSS es el mismo que en un entorno Scrum de un solo equipo.

- **Sprint Backlog:** cada equipo tiene su propio Sprint Backlog. Es la lista de trabajo que el equipo debe realizar para poder completar los ítems del Product Backlog. Por lo tanto, el Sprint Backlog es por equipo y no hay diferencia entre un LeSS Sprint Backlog y un Sprint Backlog.

LeSS Huge

Para más de 8 equipos.

- ¿Qué es igual a LeSS?
  - Un Product Backlog para todos.
  - Un DoD.
  - Un DoR.
  - Un Product Manager.
  - El Sprint es común para todos los equipos.
  - Un incremento potencialmente entregable.

- ¿Qué es diferente a LeSS?
  - Ahora hay un Area Product Manager.
  - Hay un Area Product Owners.
  - Hay un Area Product Backlogs.
  - Hay un Area Product Backlogs.

- Hay un Area Prodcut Vision.
- Hay más de 8 equipos.

Métricas ágiles y en otros enfoques

¿Qué es una métrica?

Una métrica es un número que representa el grado o presencia de un determinado conjunto de atributos respecto de lo que se quiere medir (proceso, producto o proyecto), expresadas de tal forma que permitan una medición objetiva de la realidad. Por ejemplo, la escala: NS, S, B, MB, E no sirve. Lo que sí sirve es cuántos E hay, cuántos S hay, etc. Debe ser posible medir en términos de la definición de esta y del esfuerzo que se debe aplicar para medirla. Es decir que la métrica no es la escala sino la cantidad de cierta escala.

Todas las métricas poseen un costo asociado, debido a que se deben destinar recursos para su planificación, ejecución y análisis, y no siempre es factible disponer de esos recursos en un proyecto. Esto implica un análisis de costo-beneficio para evitar definir métricas que no aporten un valor a la organización/equipo/proyecto/etc. En ese análisis costo-beneficio, también influye la relación precisión-medición, ya que muchas veces no es necesaria tanta precisión. Esto también quiere decir que las métricas no sólo hay que calcularlas y dejarlas, hay que utilizarlas para que justifiquen ese costo. (El paso de "no medir" a "comenzar a medir" no tiene que ser excesivo, porque ese es otro de los factores que termina generando resistencia, por costo excesivo, ser una pérdida de tiempo en algo que después no se mira, etc.)

Las métricas no son para evaluar a la gente. No deben ser utilizadas para castigar o beneficiar a la gente, ya que es posible que las personas comiencen a fingir en el valor de las métricas para lograr objetivos finales, lo cual resulta perjudicial para el análisis del fenómeno que se desea medir.

En el software es complicado tener métricas de algunas características del producto, como por ejemplo la usabilidad del producto, por lo que no se mide directamente al producto, sino que se utilizan conceptos asociados (epifenómenos) a la característica que se desea medir. Por ejemplo, si se quiere medir el uso de una funcionalidad, se puede medir a través de la cantidad de clicks en el botón que permite ejecutar esa funcionalidad.

Por ejemplo, una métrica que dice cantidad de requerimientos tomado en función de la cantidad de requerimientos que deberíamos tomar, no es una métrica válida.

¿Para qué medir?

- Para señalar, controlar y supervisar el desarrollo del proyecto.
- Para predecir al estimar proyectos de software.
- Para evaluar, es decir, para tener una noción de los costos, por ejemplo.
- Para mejorar el proceso, el proyecto o el producto.

Dominio de métricas

Las métricas se dividen en tres dominios, el cual, cada uno posee un foco distinto de medición:

- **Métricas de proceso**: son métricas estratégicas a nivel organizacional, orientadas a mejorar el proceso y tienen como objetivo generar indicadores que permitan mejorar los procesos de software a largo plazo. Son públicas y se despersonifican las mediciones de personas, áreas o proyectos particulares, para obtener un número aislado y tener una métrica sobre el proceso de la organización en general, como un todo.
  Son responsabilidad del Ingeniero de Procesos, quien realiza las mediciones y luego publica los datos para que todos los empleados de la organización puedan acceder a ellos.

Por ejemplo, se toman métricas de cantidad de defectos en todos los productos que se realizaron, se despersonalizan, se promedian a cada una de ellas y se publican a toda la organización sin hablar particularmente de un proyecto o producto, sino en forma general al proceso de la organización.
En este enfoque, se tiene en cuenta que la experiencia de cada proyecto es extrapolable, por eso hacen uso de esta forma de tomar las métricas, desvinculándose de un producto o proyecto particular para hablar del comportamiento de la organización.

Los indicadores de proceso permiten tener una visión profunda de la eficacia de un proceso, determinando qué funciona de acuerdo a lo esperado y qué no.
Proporcionan beneficios significativos a medida que la organización trabaja para mejorar su nivel global de madurez del proceso.

Ejemplos:

- Desviación organizacional de estimaciones.
- Defectos por severidad en productos de la organización.
- Errores previos a releases por proyecto.
- Defectos detectados por usuarios.
- Esfuerzo realizado.
- Tiempos de planificación promedio por proyectos.
- Propagación de errores de fase a fase.

- **Métricas de proyecto:** están enfocadas a los recursos que se dedican al proyecto, como costos, esfuerzos, estimaciones y tiempo. Son responsabilidad del Líder de Proyecto y permiten al equipo adaptar el desarrollo de los proyectos y de las actividades técnicas. Estas métricas son privadas de ese proyecto y solo son visibles para los involucrados en el mismo.
  Se utilizan para mejorar la planificación del desarrollo, generando ajustes que eviten retrasos, reduzcan riesgos potenciales y por lo tanto, problemas.
  Además, se utilizan para evaluar la calidad de los productos en todo momento y en caso de ser necesario, modificar el enfoque para mejorar la calidad, minimizando defectos, retrabajo y por ende el costo total del proyecto.
  Las métricas de proyecto se consolidan con el fin de crear métricas de procesos que sean públicas para la organización de software como un todo.

  Ejemplos:
  - Eficiencia de estimación del proyecto.
  - Costos estimados versus costos reales.
  - Esfuerzo/Tiempo por tarea del Ingeniero de Software.
  - Errores no cubiertos por hora de revisión.
  - Fechas de entregas reales versus programadas.
  - Cantidad de cambios y sus características.

- **Métricas de Producto:** están enfocadas en lo que se construye, son responsabilidad del equipo de desarrollo y de Testing y son particulares de ese producto.
  Se utilizan con propósitos técnicos y tienen como objetivo generar indicadores en tiempo real de la eficacia del análisis, el diseño, la estructura del código, la efectividad de los casos de prueba y calidad del software a construir.
  Se deben controlar los artefactos resultantes del proceso de desarrollo (componentes y modelos) para garantizar:
  - Que cumplan con los requerimientos del cliente;
  - Que cumplan con los requerimientos de calidad;

- Que estén libre de errores;
- Que se realizaron bajo los procedimientos de calidad.

Ejemplos:

- Cantidad de líneas de código del producto;
- Defectos por severidad de un producto;
- Promedio de métodos por clase;
- Cantidad de métodos de cada clase;
- Cantidad de casos de uso por complejidad.

Métricas en el enfoque tradicional

En el enfoque tradicional se hace énfasis en los 3 dominios arriba mencionado.

Están basadas en la gestión de proyectos definidos, poseen un abanico de métricas mayor a los demás enfoques.

Son una disciplina transversal. Cuando se planifican los proyectos, allí se definen qué métricas se utilizarán, quiénes serán responsables de estas, cómo se calcularán, etc.

No todas las métricas le interesan a todo el mundo, dependiendo del momento y rol es lo que interesa medir. Los perfiles más técnicos por ejemplo apuntan a cuestiones relacionadas con el producto y esfuerzo (porque son ahí donde los desarrolladores asumen compromiso) y en ámbitos organizacionales interesa plata (costo) y tiempo. Por ejemplo:

- Testers: les interesa el esfuerzo y cantidad de defectos encontrados principalmente, porque es lo que más afecta a este rol.
- Líder de proyecto: El enfoque de las métricas estará más relacionado al calendario, los costos, plazos de tiempo a cumplir, etc. por el contrato que se tiene con el cliente, y la relación entre el esfuerzo y el tiempo.

Métricas básicas para un proyecto de software

son las mínimas e imprescindibles que uno tiene que utilizar si tiene la intención de desarrollar una cultura de medición:

- Tamaño del producto: asociada a métrica de producto.
- Esfuerzo: asociada a métrica de proyecto.
- Calendario: asociada a métrica de proyecto.
- Defectos: asociada a métrica de producto.

Estas métricas están relacionadas con la triple restricción en un proyecto (esfuerzo, alcance y tiempo).

Métricas en ambientes ágiles

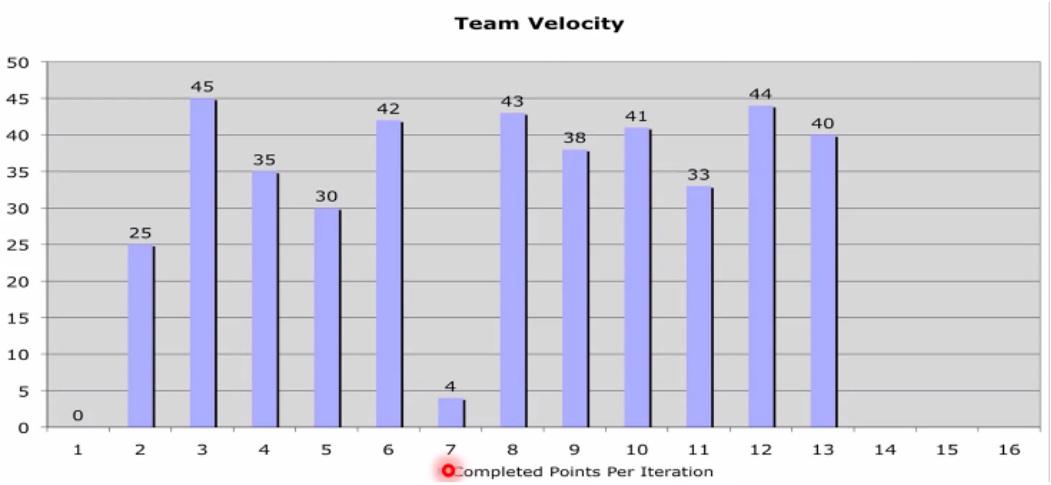
Las métricas sirven para un equipo y no son extrapolables por lo que dice el agilismo a otros proyectos o equipos. En ágil hay un principio que habla específicamente de métricas (la mejor métrica de progreso es el software funcionando). Este principio apunta a que vamos a medir producto (no proceso ni proyecto). Esto es una reacción a proyectos tradicionales que tenían todas las métricas posibles cubriendo los tres enfoques, pero no teniendo avances sobre el producto (se perdía mucho tiempo y no se progresaba). Solo se debe medir lo que sea necesario y nada más, es decir, lo que agregue valor para el cliente.

Métrica de velocidad

Es la métrica más importante del enfoque (métrica de producto), y mide la cantidad de puntos de historia que se realizaron en un Sprint y fueron aceptados por el Product Owner. Hay que recordar que no se cuentan las historias parcialmente terminadas, sólo las que están completadas y aceptadas por el PO.

Esta métrica se calcula (no se estima) luego de que el PO. acepte la implementación de una US.

Esta métrica suele representarse con un gráfico de barras para medir la estabilidad del equipo. Estos gráficos son permanentes durante todo el proyecto y son visibles para todo el mundo.



Esta métrica, y sus valores a lo largo de un proyecto ágil, permiten analizar si el equipo posee una estabilidad a lo largo del mismo, lo cual también se relaciona con un principio del manifiesto ágil (desarrollo sostenible).

Métrica de capacidad

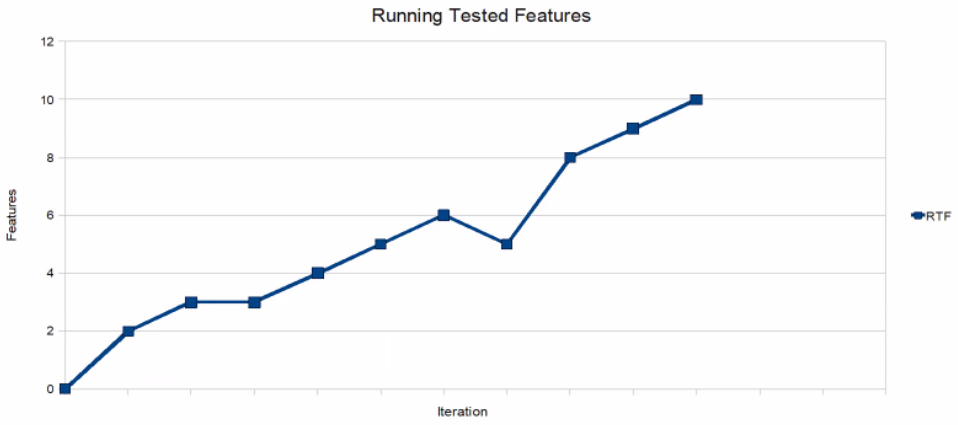
Es una métrica de proyecto que mide el compromiso de un equipo para un determinado sprint, en horas de trabajo ideales. Es por esto que, se utiliza para planificar, ya que permite definir cuántas historias de usuario se van a tomar del Product Backlog para implementar en el próximo sprint.

Se estima al principio de un sprint, por lo que también se la llama velocidad estimada. Se puede estimar en horas ideales o en Story Points.

Running Tested Features (RTF)

Mide la cantidad de features testeadas que están funcionando, es decir, cuantas piezas de producto (historias de usuarios, casos de usos, requerimientos) se terminaron y están en ejecución.

El problema de esta métrica es que no tiene en cuenta la complejidad de las piezas de producto que se implementan. Por ejemplo, si se toma como piezas de producto a historias de usuario, se mide cuántas historias de usuarios están funcionando, pero no se sabe de cuantos puntos de historias tiene cada una de ellas. Es una medición absoluta.

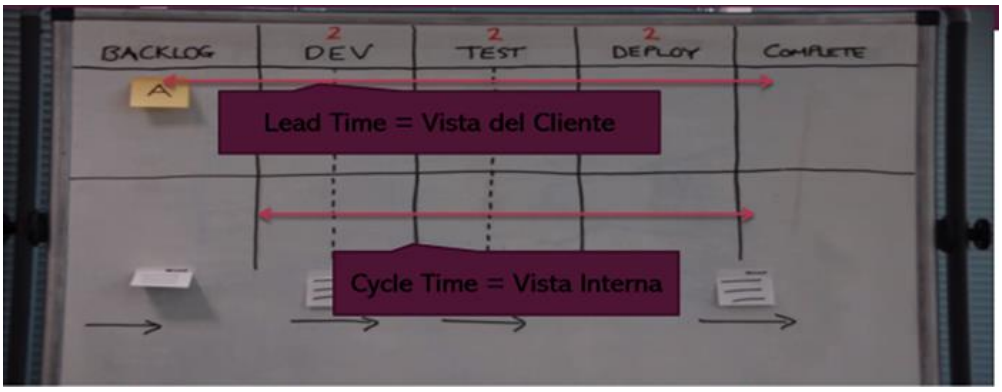


Si la curva es constante (llana) o tiene pendiente negativa, entonces indica la existencia de un problema, ya que las características no incrementan o disminuyen en el tiempo. Esta última situación se presenta cuando una funcionalidad del sistema deja de ser válida.

Las métricas de defectos también son necesarias en todos los ambientes.

Métricas de software en Lean (KANBAN)

El foco de medición de Kanban es el proceso, ya que el sistema de trabajo de Kanban es introducir mejoras en un flujo de trabajo continuo, es decir, no existe un proyecto con principio y fin. Es decir, se mide el comportamiento del proceso en función al tiempo que se demoran las características.



Lead Time o Elapsed Time

Métrica de vista o perspectiva del cliente. Es la más importante para el cliente. Mide desde el momento en que el cliente me pide algo (entra al Backlog) hasta que yo se lo entrego.

Cycle Time

Mide el tiempo desde que el equipo comenzó a trabajar sobre una funcionalidad pedida, hasta que se lo entrega, eliminando el tiempo de espera en el Backlog. Por esto se la considera como una vista interna, que sirve al equipo de trabajo y no es tan relevante para el cliente. Es igual al Lead Time menos el tiempo que estuvo en el Backlog.

Touch Time

Mide los tiempos en los cuales las personas están efectivamente trabajando sobre una unidad de trabajo (columnas de producción), sin tener en cuenta aquellos tiempos de espera (columnas de acumulación).

En cada columna, por ejemplo, en la columna Test de la imagen, se tiene una columna de trabajo y otra de acumulación para poder realizar el concepto de sistema "pull" o de arrastre.

$$\text{Touch Time} \leq \text{Cycle Time} \leq \text{Lead Time}$$

Eficiencia del ciclo de proceso

Esta métrica mide la eficiencia del tiempo para entregar un trabajo, respecto al tiempo destinado a su ejecución. Se calcula como Touch time / Lead time.

Mientras más cercano a 1 sea su valor, más eficiente es el proceso, ya que todo el tiempo se estuvo trabajando sobre alguna funcionalidad, y no hubo muchos momentos de espera en columnas de acumulación.

Métricas de Producto de Software



Hoy en día no es recomendable medir en líneas de código.

Defectos

- Cobertura: tratar de que sea la mayor posible para cubrir el 100% del análisis de los defectos del producto.
- Defectos por severidad: cantidad de defectos por escala de severidad de cada defecto. 5 defectos graves.
- Densidad de defectos: cuantos defectos se encuentran por historia de usuario.

Resumen métricas en cada enfoque

Tradicional	Ágil	Lean
✓ Esfuerzo	✓ Velocidad	✓ Lead time – Elapsed time
✓ Tiempo	✓ Capacidad	✓ Cycle time
✓ Costos	✓ Running Tested Features	✓ Touch time
✓ Riesgos		✓ Eficiencia de proceso

Gestión de productos de software – Planificación de productos – herramientas para definición de productos de software

Un producto de Software es un artefacto que se construye para satisfacer una necesidad específica en base a ciertos requerimientos.

Podemos decir que va de la mano con el manifiesto ágil porque recordemos que se buscaba estar siempre preparados para el cambio y justamente, la gestión de configuración busca responder a los cambios y mantener la integridad del producto.

La diferencia es que el manifiesto ágil se enfoca en los proyectos, mientras que la gestión de configuración de software se enfoca en el producto. Es decir, trasciende el proyecto.

Mientras el manifiesto ágil apunta a cómo trabajar en el contexto del proyecto de desarrollo y la gestión de configuración es una práctica específica del producto que garantiza la calidad del producto.

Características

1. Dada la naturaleza de los requerimientos ágiles, la SCM responde a los cambios en lugar de evitarlos.
2. La automatización debe ser aplicada donde sea posible. Esto colabora con la entrega frecuente y rápida de software (integración continua).
3. Provee retroalimentación continua sobre calidad, estabilidad e integridad.
4. Las actividades de SCM se encuentran embebidas en las demás tareas para alcanzar el objetivo del sprint.
5. Es responsabilidad de todo el equipo, por lo tanto, requiere capacitación.
6. Adopción de las prácticas continuas.

Continuous Integration

Es una práctica de desarrollo que promueve que los desarrolladores adopten la costumbre de integrar su código a un repositorio compartido varias veces al día. Cada integración de código es luego verificada por una serie de pruebas automatizadas, permitiéndole al equipo detectar problemas de manera temprana. Ya que al integrar el código al repositorio de manera frecuente resulta más fácil detectar y corregir los errores.

La integración continua es asegurar que el software pueda ser desplegado en cualquier momento, es decir, que el código compile y que sea de calidad.

Cada desarrollador en su entorno de trabajo realiza pruebas unitarias (en la medida de lo posible, automatizadas) desarrollando con algo que se llama TDD (desarrollo conducido por pruebas) y cuando terminó ese componente de código y lo probó y sabe que funciona, lo sube a un repositorio de integración.

Así, la versión del producto está en condiciones de ir a las pruebas de aceptación de usuario sin problemas.

Continuous Delivery

Es una disciplina de desarrollo de software en la que el software se construye de tal manera que puede ser liberado en producción en cualquier momento. Suma la automatización de las pruebas de aceptación y entonces el producto ya está listo para desplegarlo a producción.

No implica necesariamente que se libere cada vez que hay un cambio, sólo ocurre si el responsable de negocio, el Product Owner de turno, decide pasar a producción. Es decir que hay un componente «humano» a la hora de tomar la decisión. Pero, en cualquier caso, la versión está lista de inmediato. Esto implica, entre otros, que se prioriza que la versión esté en un estado en el que pueda ser puesta en producción.

La integración continua es prerrequisito de la entrega continua. Con esto se refiere a que los artefactos producidos en el servidor de integración continua son puestos en producción con solo un click. Hay que asegurarse de tener un despliegue automatizado, para que cada puesta en producción no lleve demasiado tiempo, lo que hará que las puestas puedan llegar a hacerse más seguidas, generando que lo que se despliegue no sean demasiados cambios y el riesgo

de que algo se rompa disminuya. El software debe estar siempre en un estado de “entregable”, es decir, el software buildea, el código se compila y los tests pasan.

Continuous Deployment – Estrategias de Deployments – Canary Deployments – Blue/Green Deployment

Consiste en poner en el ambiente de producción del usuario final el producto. A diferencia de la entrega continua, en el despliegue continuo no existe la intervención humana para desplegar el producto en producción. Para esto, se utilizan pipelines que contienen una serie de pasos que deben ejecutarse en un orden determinado para que la instalación sea satisfactoria.

El propósito de las estrategias es que sea transparente para el usuario que pusiste en producción una nueva versión del producto.

Se recomienda hacer el despliegue a poco tiempo de haber trabajado con los cambios en el código, pues un error en producción un día después de haberlo hecho significa que todavía nos acordamos de lo que hicimos y será más fácil de resolver.

Canary Deployment

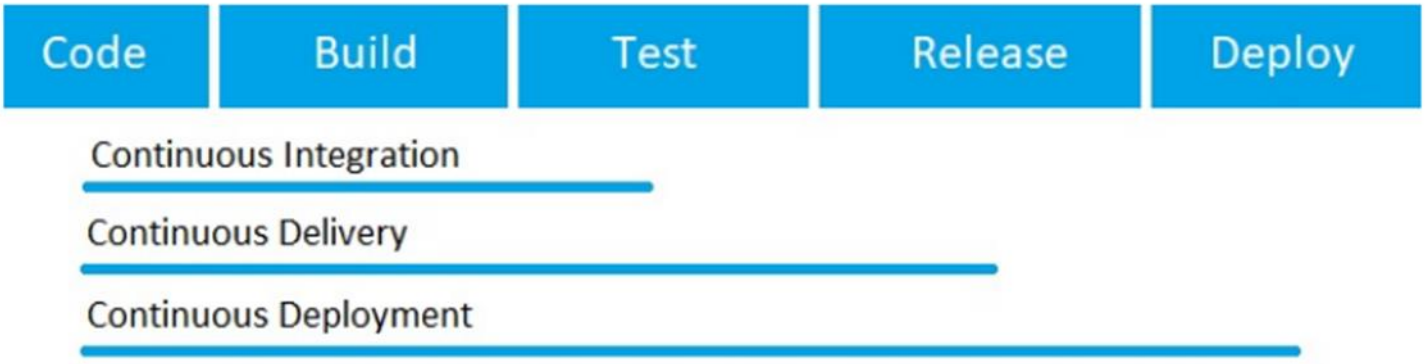
En ingeniería de software, Canary deployment es la práctica de realizar lanzamientos por etapas. Primero implementamos una actualización de software para una pequeña parte de los usuarios, para que puedan probarla y proporcionar comentarios. Una vez aceptado el cambio, la actualización se extiende al resto de usuarios.

Esta estrategia nos muestra cómo los usuarios interactúan con los cambios de la aplicación en el mundo real. Al igual que en Blue-Green Deployment, la estrategia Canary ofrece actualizaciones sin tiempo de inactividad y reversiones sencillas. A diferencia de Blue-Green, las implementaciones Canary son más fluidas y las fallas tienen un impacto limitado.

Blue-Green Deployment

Ees un modelo de lanzamiento de aplicaciones que transfiere gradualmente el tráfico de usuarios de una versión anterior de una aplicación o microservicio a una nueva versión casi idéntica, las cuales se ejecutan en producción.

La versión anterior puede denominarse entorno azul, mientras que la nueva versión puede denominarse entorno verde. Una vez que el tráfico de producción se transfiere por completo de azul a verde, el azul puede quedar en espera en caso de reversión o retirada de producción y se actualiza para convertirse en la plantilla sobre la que se realiza la próxima actualización.



La diferencia entre estas tres prácticas es el nivel de automatización de las pruebas.

Unidad 4: Aseguramiento de calidad de Proceso y de Producto

Conceptos generales sobre calidad

Importancia de trabajar para y con Calidad. Ventajas y Desventajas.

Calidad

Formalmente se puede definir como el cumplimiento de los requerimientos funcionales y de performance explícitamente definidos, de los estándares de desarrollo explícitamente documentados y de las características implícitas esperadas del desarrollo de software profesional.

Todos los aspectos y características de un producto o servicio que se relacionan con la habilidad de alcanzar las necesidades manifiestas o implícitas. Está relacionado con MIS necesidades y MIS expectativas (se relaciona con las necesidades implícitas ya que no son expresadas).

Está directamente relacionada con la persona (es subjetiva a quién la analice), las circunstancias, el contexto, la edad en un momento de tiempo dado (puede para mí algo no tener calidad, pero para otra persona sí).

Gestión de calidad

La gestión de calidad del software para los sistemas de software tiene tres intereses fundamentales:

- A nivel de organización, la gestión de calidad se ocupa de establecer un marco de proceso y estándares de organización que conducirán a software de mejor calidad.
- A nivel del proceso, la gestión de calidad implica la aplicación de procesos específicos de calidad y la verificación de que continúen dichos procesos planeados.
- A nivel del proyecto, la gestión de calidad se ocupa también de establecer un plan de calidad para un proyecto.

Aseguramiento de calidad

Es la definición de procesos y estándares que deben conducir a la obtención de productos de alta calidad y, en el proceso de fabricación, a la introducción de procesos de calidad.

El espíritu del aseguramiento de la calidad del SW es actuar como medida preventiva a diferencia del testing que llega tarde.

El equipo QA debe ser independiente del equipo de desarrollo para que pueda tener una perspectiva objetiva del software.

La planeación de calidad es el proceso de desarrollar un plan de calidad para un proyecto. El plan de calidad debe establecer las cualidades deseadas de software y describir cómo se valorarán. Por lo tanto, define lo que realmente significa software de “alta calidad” para un sistema particular.

Humphrey (1989), en su libro referente a la gestión del software, sugiere un bosquejo de estructura para un plan de calidad. Éste incluye:

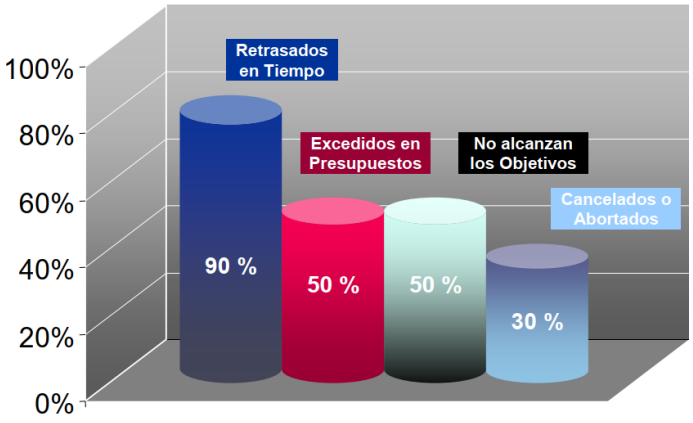
- Introducción del proyecto;
- Planes del producto: fechas de entrega críticas y las responsabilidades para el producto;
- Descripciones de procesos;

- Metas de calidad: Las metas y los planes de calidad para el producto, incluyendo una identificación y justificación de los atributos esenciales de calidad del producto;
- Riesgos y gestión de riesgos.

Aunque los estándares y procesos son importantes, **los administradores de calidad deben enfocarse también a desarrollar una "cultura de calidad"** en la que todo responsable del desarrollo del software se comprometa a lograr un alto nivel de calidad del producto. Deben exhortar a los equipos a asumir la responsabilidad de la calidad de su trabajo y desarrollar nuevos enfoques para el mejoramiento de la calidad.

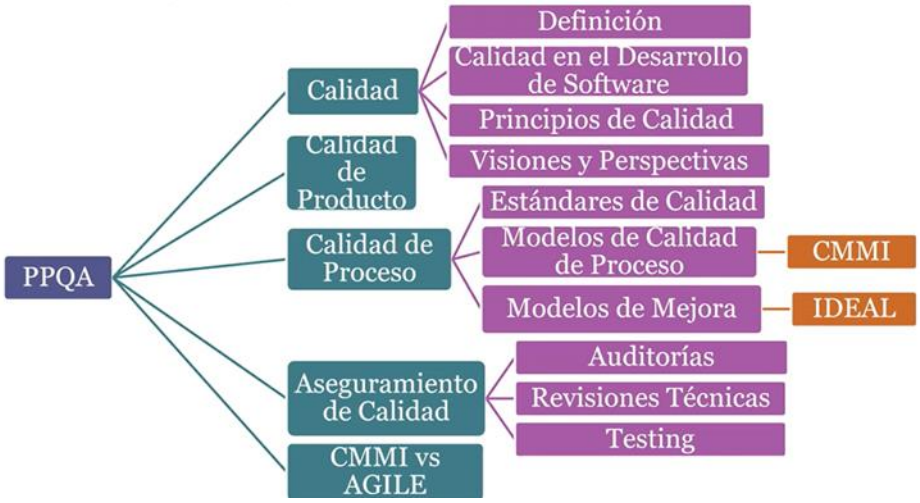
Problemas en la calidad

- Atrasos en las entregas -> Relacionado a calidad del proyecto.
- Requerimientos no claros -> Relacionado a calidad del producto.
- Software que no hace lo que debería -> Relacionado a calidad del producto.
- Costos excedidos -> Relacionado a calidad del proyecto.
- Trabajo fuera de hora -> Relacionado a calidad del proyecto.
- Fenómeno 90-90 (90% hecho 90% restante) -> Relacionado a calidad del proyecto.
- No se aplica SCM. -> Relacionado a calidad del producto.



Un software de calidad no solo debería cumplir con los requerimientos, sino que también debería poder entregarse a tiempo, no exceder costos, etc.

Aseguramiento de calidad de Proceso y de Producto



Calidad en el desarrollo de Software

Los costos excedidos, retrasos en la entrega, falta de cumplimiento de los compromisos, requerimientos no claros hacen que la percepción de nuestro cliente sea mala. El proyecto es el medio que permite alcanzar el producto. Si el medio no tiene calidad, difícilmente se pueda lograr un producto de calidad.

Para que un SW sea de calidad debe satisfacer:

- Las expectativas del cliente;
- Las expectativas del usuario;
- Las necesidades de la gerencia;
- Las necesidades del equipo de desarrollo y mantenimiento;
- Las expectativas de otros interesados.

Principios de Calidad

- La calidad NO se inyecta, debe estar embebida: la calidad es algo que se concibe desde el momento cero, desde requerimientos en adelante. Lo que se hace con el testing es controlarla.
- Es una responsabilidad de todos los involucrados en el proyecto.
- Las personas son la clave para lograrla: se hace mucho hincapié en la capacitación, para brindar herramientas y conocimientos a los involucrados. "Si usted cree que la capacitación es cara entonces pruebe con la ignorancia". Hacer software es una actividad humano-intensiva
- Se necesita sponsor a nivel gerencial, pero se puede empezar por uno.
- Se debe liderar con el ejemplo.
- No se puede controlar lo que no se mide, debemos usar métricas.
- Simplicidad, empezar por lo básico.
- Debe planificarse el aseguramiento de calidad.
- El aumento de las pruebas no aumenta la calidad. La calidad se obtiene y se inyecta a lo largo del proyecto, no al final.
- Debe ser razonable para mi negocio.
- Prevención mejor que corrección: prevenir es menos costoso, ya que corregir demanda un costo muy alto asociado al retrabajo.

En la actualidad, el trabajo con calidad es fundamental en la producción del software ya que:

- Es un aspecto competitivo, que permite sobrevivir en el mercado internacional. Las empresas certifican estándares de calidad (de proceso no de producto) para poder competir en el mismo.
- Retiene a los clientes e incrementa los beneficios. Siempre que hablamos de calidad, en el fondo nos referimos al cumplimiento de las expectativas del cliente en todos los sentidos.
- Determina un equilibrio del costo-efectividad. Trabajar con calidad reduce los costos y el retrabajo, ya que se asume que las fallas serán menores.

Las metodologías ágiles hablan constantemente de calidad de SW y se asume que los equipos ya están capacitados y orientados a procesos de calidad.

La disciplina de Gestión de la Administración de Configuración nos da el contexto de base para poder trabajar con calidad.

Calidad para Quién – Visiones de Calidad

A la calidad se la puede analizar desde distintas perspectivas o visiones:

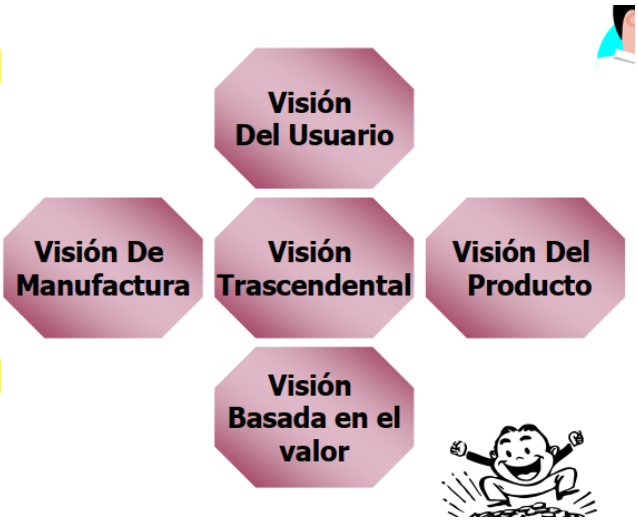
- **Visión del usuario:** esto tiene que ver con las expectativas que tiene el usuario para con el producto, y si este las satisface. Esta es quizás la más complicada de establecer, porque está en la cabeza de los usuarios, lo cual es una razón más por la que el principio de comunicación constante es crucial, para ir validando en todo momento si estamos en el camino correcto. El agilismo trata de incluir la visión de calidad del usuario a través del rol de Product Owner, el cual es ocupado por una persona con capacidad de decisión del cliente.

- **Visión del producto:** esto se asocia al nivel de satisfacción de los requerimientos particulares de cada producto.

- **Visión del proceso (manufactura):** si el proceso de desarrollo utilizado es el correcto para el producto que se desea desarrollar, es decir, el proceso aporta valor al producto y no produce desperdicios.

- **Visión del valor:** encontrar un equilibrio en la relación costo-beneficio, para obtener siempre el mayor valor para el cliente posible y, obviamente generar ganancias con el desarrollo del software.

- **Visión Trascendental:** esta visión es un tanto utópica, ya que se asocia a objetivos difíciles de alcanzar. Por ejemplo 0 defectos en el producto, pero son aquellos objetivos que motivan a seguir en busca de la mejora continua.



La calidad es un concepto subjetivo que tiene que ver con, si para uno cumple con las necesidades y expectativas. Las expectativas son implícitas ya que son las cosas que quieres que abarque tu producto o servicio, pero no están dichas en ningún lado. Por ejemplo, que contenga una interfaz agradable el sistema, pero si no ocurren estas expectativas empiezan las disconformidades.

La calidad subjetiva de un sistema de software se basa principalmente en sus características no funcionales. Esto refleja la experiencia práctica del usuario: Si la funcionalidad del software no es lo que se esperaba, entonces los usuarios con frecuencia solos le darán la vuelta para realizar lo que necesitan y encontrarán otras formas de hacer lo que quieren. Sin embargo, si el software no es fiable o es muy lento, entonces es prácticamente imposible que los usuarios logren sus objetivos con el sistema.

Con demasiada frecuencia, la causa real de los problemas en la calidad del software no es una gestión deficiente, procesos inadecuados o capacitación de escasa calidad. Más bien, es el hecho de que las organizaciones deben competir para sobrevivir. Una empresa, puede estimar el esfuerzo requerido o prometer la entrega rápida de un

sistema en plazos de tiempo ajustados, para lograr un acuerdo con el cliente. En consecuencia, al no haber suficiente tiempo para el desarrollo, es probable que el software entregado tenga funcionalidades reducidas o niveles más bajos de fiabilidad o rendimiento, por lo tanto, esto conlleva a que la calidad del software se vea afectada, en ese intento de lograr cerrar un acuerdo de negocio con el cliente que necesita el software.

Calidad en el Software

La gente necesita definir un proceso para llevar a cabo el software, y generalmente en las organizaciones se basa en modelos para tomar de referencia.

Si quiero funcionar con mejora continua tengo modelos para mejorar esos procesos. También tenemos modelos de evaluación que ven el grado de adherencia del proceso al modelo que se tomó de referencia. Por ejemplo, si tomo la ISO 9001, si llamo a una auditoría de ISO esa auditoría debería decirte todas las diferencias que se tienen en ese proceso definido respecto de lo que deberías tener.

Los procesos se instancian en los proyectos. Los procesos son solo una indicación de cómo hacer las cosas, pero es el proyecto que al ejecutarse se insertan actividades para ir regulando si lo que estoy haciendo es lo que se había pensado.

Tenemos las revisiones técnicas que se hacen entre pares, no se somete a la persona a evaluación sino el producto. Se puede hacer sobre cualquier artefacto que queramos: requerimientos, etc. Luego tenemos las auditorías que son realizadas por personas externas (los empíricos están muy en contra porque creen que los equipos son capaces de reconocer y corregir por sí mismos, esto puede verse en la retrospectiva).



El producto que tengo que someter a evaluación es el que se va generando en el contexto del proyecto, por lo que tengo que insertar tareas para asegurar la calidad del producto. Ahí es donde aparecen varias técnicas como revisiones técnicas (realizado entre pares, con el propósito de detectar tempranamente el defecto), auditorías de configuración funcional (la que valida una versión de la línea base), auditorías de configuración física (la que verifica una versión de la línea base), Testing (es para controlar la calidad cuando el producto ya está listo).

Los modelos de calidad dicen: hace tu proceso de manera que tenga calidad para vos (empresa), pero este proceso debe ser compatible con lo que yo te digo (modelo de referencia).

Calidad del producto

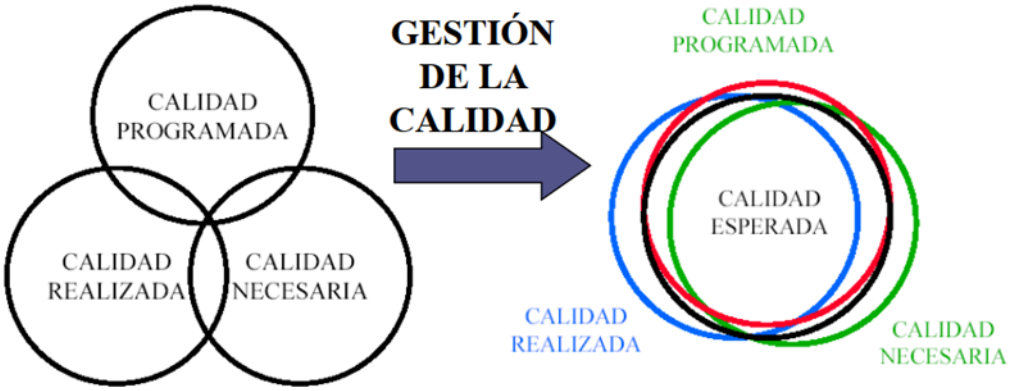
Definimos anteriormente que la calidad es el nivel de cumplimiento o adecuación a las necesidades (explícitas e implícitas) y para el caso del producto estas deberían estar explicitadas en los requerimientos. Es por esto que los

requerimientos son tan importantes, porque si no están o están mal definidos, no tenemos contra qué validar. Esto quiere decir que para que los requerimientos nos sirvan estos tienen que ser medibles (verificables) y objetivos (no ambiguos).

La gestión de calidad en productos son acciones sistemáticas de las organizaciones para lograr calidad, las cuales requieren equilibrio entre:

- Calidad programada: los alcances del producto planificados.
- Calidad realizada: lo que realmente se ha desarrollado del producto.
- Calidad necesaria: mínimas características que el producto debe tener, para que este satisfaga los requerimientos.

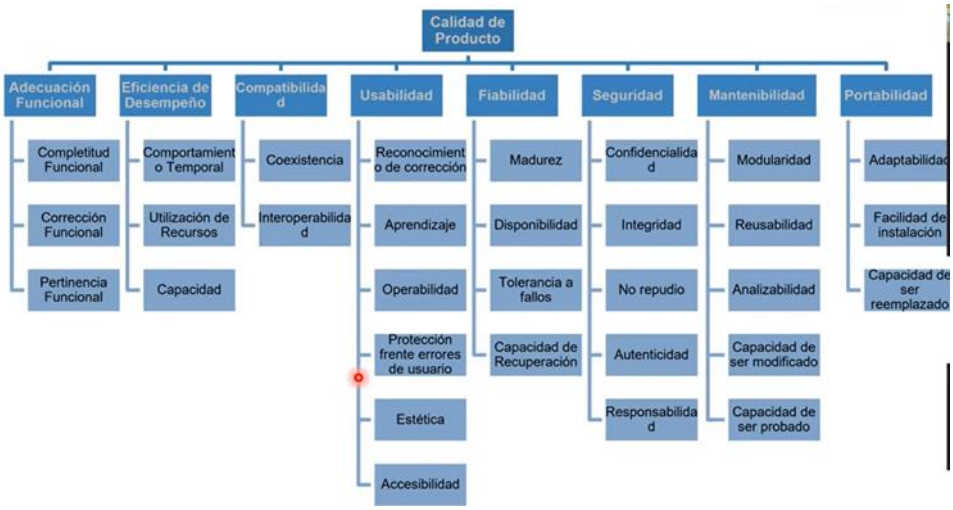
En el equilibrio obtenemos las expectativas de calidad que esperamos del producto y todo lo que esté por fuera de esta es desperdicio o insatisfacción.



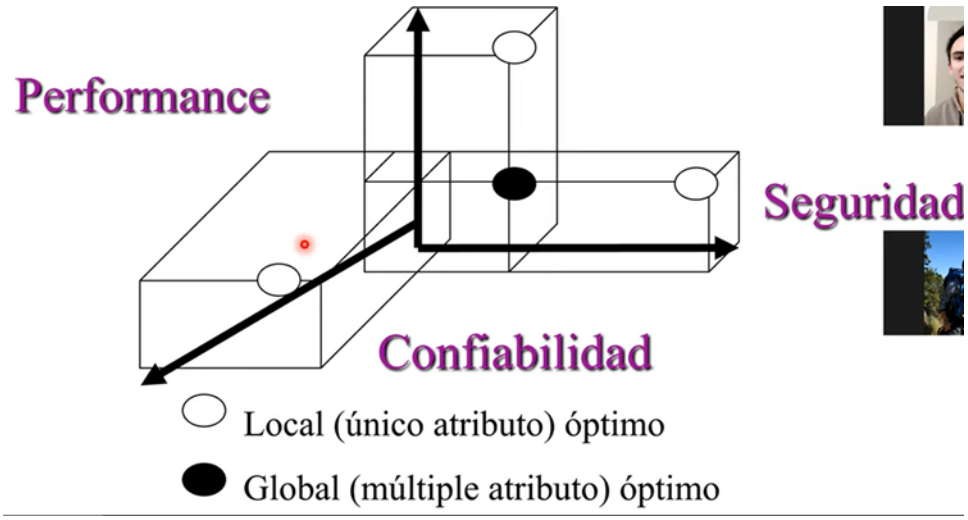
Modelos de calidad de Producto

Al variar los requerimientos para cada producto en particular, no existen modelos de calidad que acrediten para un determinado software qué nivel de calidad posee. Si existen modelos como el Modelo de Barbacci, Calidad de Software (MCCALL) o la ISO 25.010 que permite medir epifenómenos como los RNF del software, para certificar calidad en ciertos aspectos (no los vemos en esta materia).

ISO 25000 -RNF

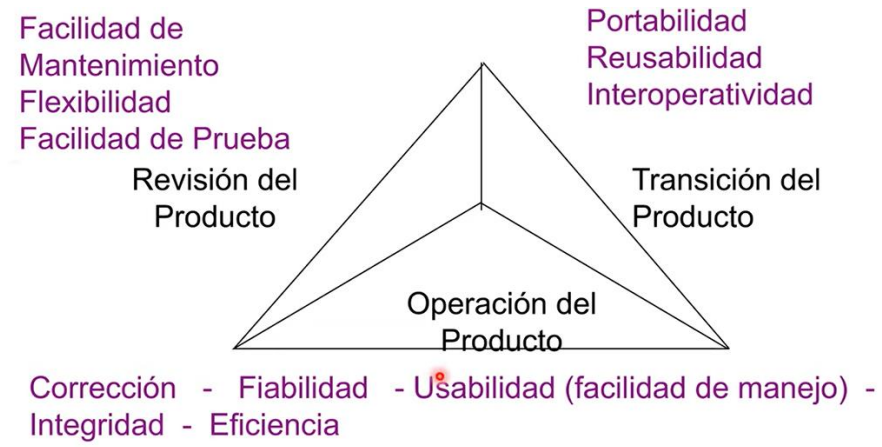


Modelo de Barbacci / SEI



Busca el equilibrio entre la Perfomance, la Confiabilidad y la Seguridad para evaluar la calidad del producto.

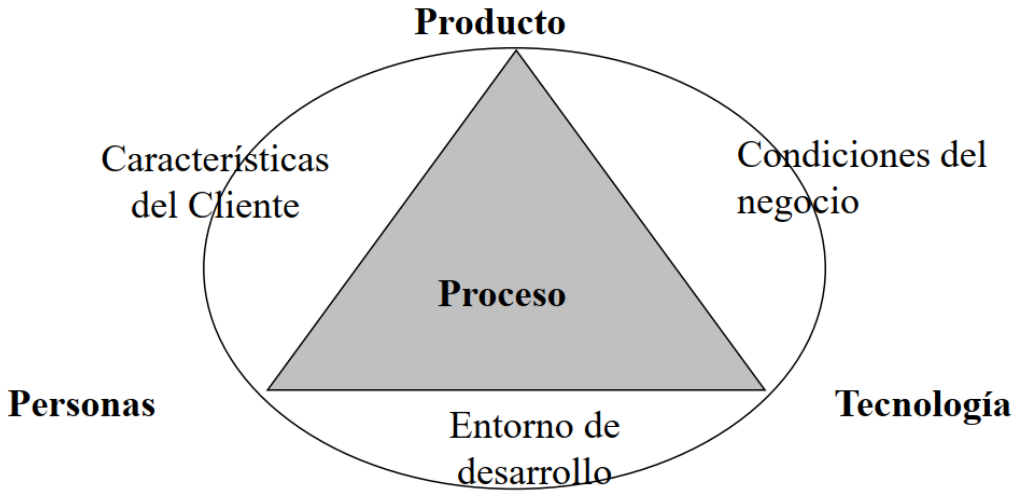
Calidad del SW Mccall



Tiene tres grandes agrupadores unos relacionados a la capacidad de verificación del producto (revisión del producto), otras con el producto en uso (operación del producto) y otras con los factores que influyen que el producto pueda crecer en el tiempo (transición del producto).

Calidad del Proceso

- No existe en la realidad un proceso que sirva para todas las organizaciones y proyectos en general.
- La calidad de proceso nunca es un fin en sí mismo, lo que realmente nos interesa es la calidad de producto.
- Proceso con calidad → Producto con calidad
- Se insiste tanto en la calidad del proceso, debido a que es el único factor controlable para mejorar la calidad del software:



- El avance de las tecnologías que se utilizan para construirlo es ajeno a la organización y no es posible controlarlo.
- Las características y necesidades del cliente tampoco se pueden modificar.
- Las personas involucradas en el proceso de desarrollo son difíciles de controlar también.

Se aplica calidad en el producto y en el proceso. No se habla de aseguramiento de calidad en el proyecto, dado que esta se encuentra implícita por el hecho de que el proyecto implementa un proceso. Para garantizar la calidad se realizan auditorias.

La calidad del producto se realiza mediante actividades de revisiones técnicas y de auditorías. Estas son las herramientas principales para el seguimiento.

Para obtener calidad en el producto se debe definir un proceso que debe ser conocido por todos los involucrados en el equipo, donde además de tener tareas técnicas (requisitos, análisis, diseño, etc.) se incorporan disciplinas transversales como SCM, QA, Planificación de Proyectos y su Seguimiento.

Objetivos de QA:

- Realizar los controles apropiados del software y de su proceso de desarrollo.
- Asegurar el cumplimiento de los estándares y procedimientos para el software y el proceso.
- Asegurar que los defectos en el producto, proceso o estándares son informados a la gerencia para que puedan ser solucionados.

Estándares de Gestión de Calidad del Software:

Estándares de Producto

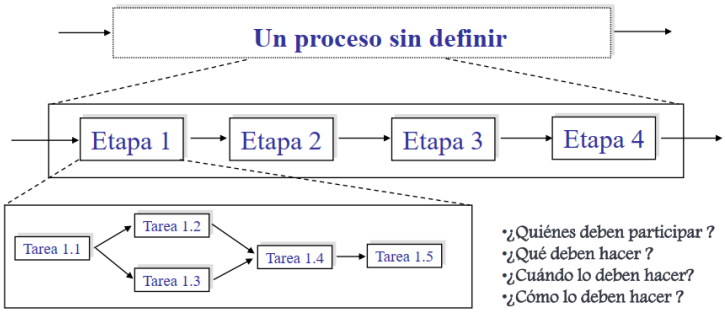
Definen las características que todos los componentes del producto deberían exhibir. Por ejemplos: estilos/buenas prácticas de programación, estándares de documentación.

Estándares de Proceso

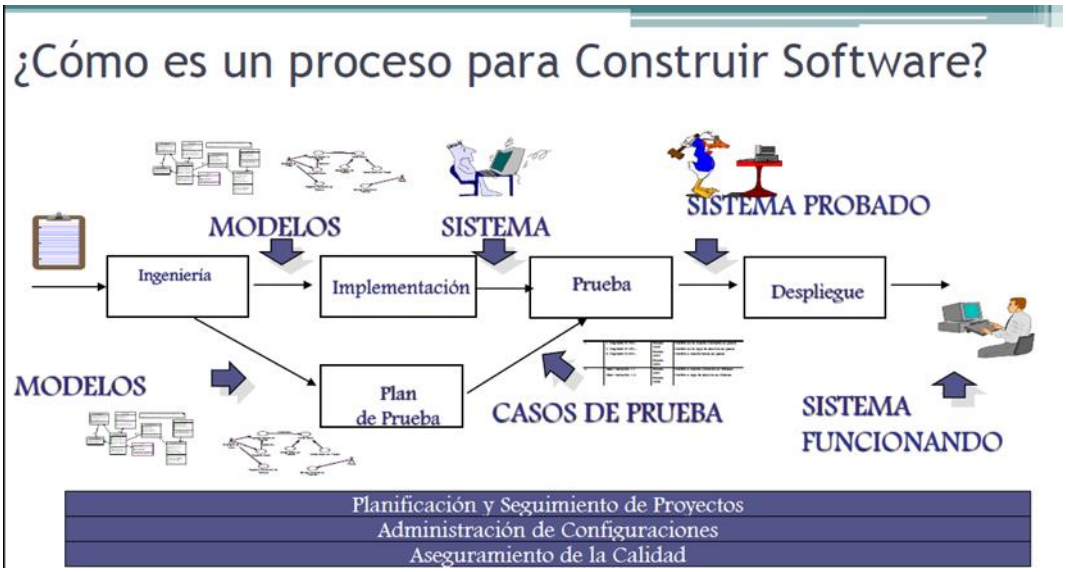
Establecen los procesos que deben seguirse durante el desarrollo, por ejemplo: incluyen definiciones de requerimientos, diseño, validación, etc. Definen como deben ser implementados los procesos de software.

Definición de Procesos

- Lo primero que se debe realizar en una organización para lograr tener un proceso de calidad, es **definirlo** de forma explícita y comunicarlo a todo el equipo, para que así todos tengan una base y el conocimiento de la forma de trabajar. Se deben definir aspectos como etapas, subetapas, roles, qué se debe hacer, en qué momentos se deben hacer, cómo lo deben hacer, etc.



- Dentro de esa definición, la cual plantea las etapas para construir la parte técnica (Ingeniería de Requerimientos, Análisis, Diseño, Implementación, Prueba y Despliegue), también se deben incorporar disciplinas transversales a ellas, como la Planificación y Seguimiento de Proyectos, SCM y QA, independientemente de la metodología adoptada (tradicional o empírica).
- El proceso definido y adoptado, debe aportar valor al producto (es posible utilizar modelos de mejora de proceso como Kanban) y utilizarlos en los distintos proyectos de forma ADAPTADA.
- La adaptación de los procesos se da en aspectos negociables del mismo. Por ejemplo, el realizar Testing no es algo negociable y que se pueda eliminar del mismo.



Procesos definidos

En los procesos definidos se considera que el proceso es el único factor controlable, por lo tanto, se asume que la mejora de la calidad continua se realiza sobre el proceso. Si el proceso cuenta con calidad, entonces el producto será de calidad.

Todos los modelos de calidad están basados en procesos definidos, por lo que asumen que la calidad del producto se obtiene si se tiene un proceso de calidad para construir el producto.

Procesos empíricos

Los procesos empíricos están basados en la experiencia, por lo que no consideran que la calidad en el proceso determine la calidad en el producto. Por otra parte, no están de acuerdo con las auditorias, ya que asumen que los equipos son autoorganizados. Sin embargo, la calidad en estos procesos se lleva a cabo mediante revisiones técnicas y la constante inspección y adaptación del trabajo.

Según los empíricos, siempre que las personas hagan lo que tengan que hacer el producto tendrá calidad.

Actividades relacionadas con el Aseguramiento de la Calidad del Software.

Administración de la calidad de Software

Busca asegurar que se alcancen los niveles requeridos de calidad para el producto de SW, definiendo estándares y proceso de calidad apropiados (que deben ser respetados por todos). El fin en sí de la Administración de Calidad de SW es generar una cultura de calidad y que esta sea vista como una responsabilidad de todos en el equipo.

La idea es insertar en las actividades acciones tendientes a detectar lo más temprano posible oportunidades de mejora sobre el producto y sobre el proceso. Hay que tener ciertas **consideraciones** respecto al Reporte del Grupo de Aseguramiento de Calidad (GAC):

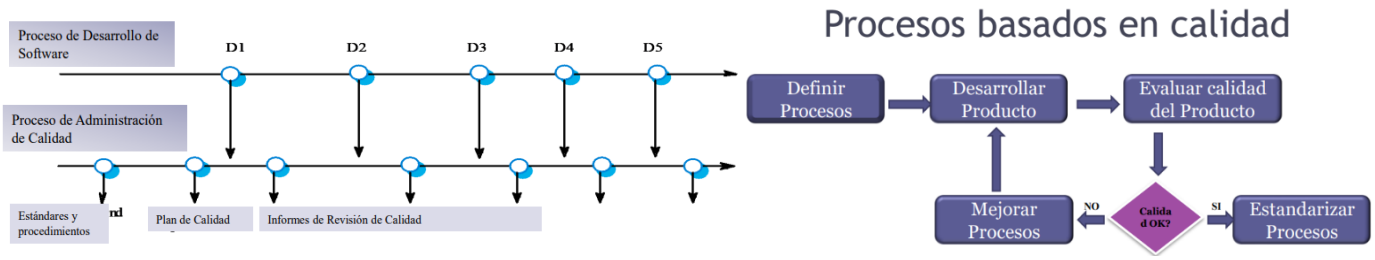
- No debería reportar la gente que hace calidad al mismo gerente que reporta los proyectos (porque le quita independencia y libertad).
- El grupo de aseguramiento de calidad debería depender del gerente general.
- Cuando sea posible, el grupo de aseguramiento de calidad debería reportar a alguien realmente interesado en el aseguramiento de calidad.

Entre las actividades que desempeña la administración de calidad, se encuentran:

- **Aseguramiento de calidad**: define estándares, procesos, procedimientos y modelos de calidad sobre los cuáles se van a realizar las comparaciones.
- **Planificación de Calidad**: se debe planificar la calidad, qué actividades se van a llevar a cabo, cuándo. Se selecciona los estándares aplicables para nuestro proyecto en particular y se los modifica si fuera necesario. Esta actividad abarca determinar los productos de SW que se desea que tengan calidad y determinar sus atributos de calidad más significativos.
- **Control de Calidad**: es la ejecución de lo planificado, y se revisa en qué situación está el proyecto. Asegura que los procedimientos y estándares son respetados por el equipo de desarrollo de SW. Existen dos enfoques para el control de calidad:
    - **Revisiones de calidad**: principal método de validación de la calidad de un proceso o un producto. El grupo examina parte de un proceso/producto + documentación, para encontrar potenciales problemas.
        - Tipos de revisiones de calidad:
            - Inspecciones para remoción de defectos (producto);
            - Revisiones para evaluación de progreso (producto y proceso);
            - Revisiones de Calidad (producto y estándares).
    - Evaluaciones de SW automáticas y mediciones.

Entre las funciones del Aseguramiento de Calidad de SW:

- Prácticas de aseguramiento de calidad.
- Evaluación de la planificación del proyecto de SW.
- Evaluación de requerimientos.
- Evaluación del proceso de diseño.
- Evaluación de las prácticas de programación
- Evaluación del proceso de integración y prueba del software
- Evaluación de los procesos de planificación y control de proyectos.
- Adaptación de los procedimientos de calidad para cada proyecto.



Calidad de procesos en la práctica: ¿cómo garantizamos calidad de proceso en la práctica? Cumpliendo con algunas de las siguientes tareas:

- Definir proceso estándares tales como:
  - Cómo deberían conducirse las revisiones;
  - Cómo debería realizarse la administración de configuración, etc.
- Monitorear el proceso de desarrollo para asegurar que los estándares sean respetados.
- Reportar en el proceso a la Administración De Proyectos y al responsable del SW.
- No use prácticas inapropiadas simplemente porque se han establecido los estándares.

Principales Modelos de Calidad existentes (CMMI – SPICE – ISO) y sus métodos de evaluación.

Lineamientos para la implementación de modelos de calidad en las organizaciones.

Modelos para la mejora de procesos

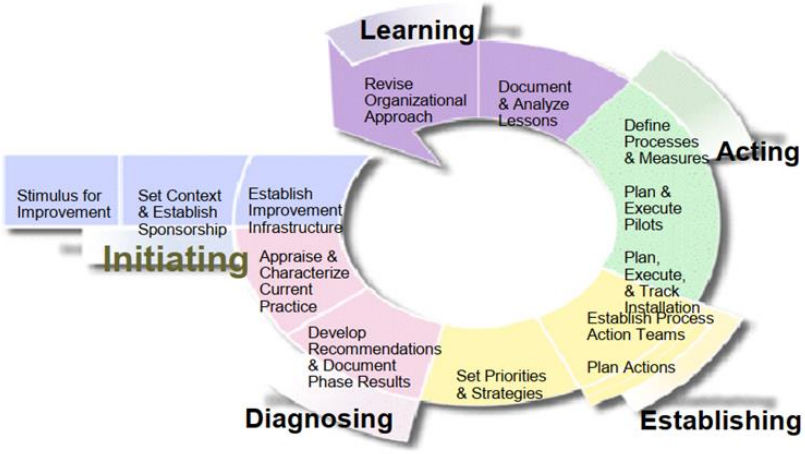
La mejora continua de procesos significa comprender los procesos existentes y cambiarlos para incrementar la calidad del producto o reducir los costos y el tiempo de desarrollo. Los procesos definidos están de acuerdo con esta definición, ya que consideran que la calidad del producto final depende de la calidad del proceso.

Los modelos NO te dicen cómo hacer las cosas. Son modelos descriptivos

El propósito de los modelos de mejora es analizar el proceso que tiene la organización y armar un proyecto cuyo resultado, en lugar de ser un producto, es un proceso mejorado que se vuelca de nuevo a la organización con la idea de que se produzca un producto mejor.

Modelo IDEAL (Initiating, Diagnosis, Establishing, Acting, Learning)

Nos da el contexto para crear un proyecto cuyo resultado va a ser un proceso definido. Es un modelo cíclico que mejora el proceso existente en una organización.



En el inicio empieza buscando un sponsor (apoyo en la organización (económico)). Esto es importante porque una mejora de proceso nunca es crítica, siempre hay algo más importante, entonces si no tengo un aval para ejecutar este tipo de proyectos, suelen no terminar exitosamente. Debemos entonces analizar dónde estamos y dónde queremos ir (se le llama análisis de brecha).

Se trata de un modelo de mejora de procesos que sirve como guía para inicializar, planificar e implementar acciones de mejora. Su nombre se debe a las 5 fases que lo componen:

- **Inicialización**: se reconocen las necesidades de cambio en la organización, razones para iniciar, determinar metas buscadas al proponer un cambio en el proceso. Se requiere que la organización apoye esta decisión de mejora (sponsoreo).
- **Diagnóstico**: Establecer la madurez actual de la organización y los riegos asociados al proceso de mejora (revisar estado actual y futuro de la org.).
- **Establecimiento**: Durante la fase se elabora un plan detallado con acciones específicas, entregables y responsabilidades para el programa de mejora basado en los resultados del diagnóstico y en los objetivos que se quieren alcanzar. Para elaborar el plan se parte de definir las prioridades para el esfuerzo de mejora.
- **Ejecutar/Acción**: Efectuar los cambios y reunir información para aprender de la mejora. Se implementan las acciones planeadas en un proyecto piloto (no en todos los procesos de la organización, ya que es una prueba). Si la solución es satisfactoria para la org, se implanta en la empresa.
- **Aprendizaje**: busca garantizar que el próximo ciclo sea más efectivo. Durante la misma se revisa toda la información recolectada en los pasos anteriores y se evalúan los logros y objetivos alcanzados para lograr implementar el cambio de manera más efectiva y eficiente en el futuro. Extrapolar la mejora al resto de proyectos en caso de que sea exitosa la ejecución o realizar correcciones en caso de que fracase el proyecto piloto.

Modelo SPICE (Software Process Improvement Capability Evaluation)

Es un modelo creado para la mejora de procesos software (ISO 15504). Es una adaptación para la evaluación de procesos de desarrollo software por niveles de madurez, dividido en 6 niveles.

Es un modelo dual, teórico. Tiene 2 partes:

- Modelo de Calidad
- SPICE + IDEAL: son ambos modelos de mejora de proceso. Estos modelos de mejora se usan para crear Proyectos de Mejora para una organización. El resultado de este proyecto va a ser un proceso definido que se usará para hacer Proyectos.

Este modelo establece conjuntos predefinidos de procesos con objeto de definir un camino de mejora para una organización. En concreto, establece 6 niveles de madurez para clasificar a las organizaciones.

Nivel	Estado
Nivel 0 - Organización inmadura	La organización no tiene una implementación efectiva de los procesos
Nivel 1 - Organización básica	La organización implementa y alcanza los objetivos de los procesos
Nivel 2 - Organización gestionada	La organización gestiona los procesos y los productos de trabajo se establecen, controlan y mantienen
Nivel 3 - Organización establecida	La organización utiliza procesos adaptados basados en estándares
Nivel 4 - Organización predecible	La organización gestiona cuantitativamente los procesos
Nivel 5 - Organización optimizando	La organización mejora continuamente los procesos para cumplir los objetivos de negocio

Modelo de Calidad para evaluar procesos

Plantean una definición teórica (lineamientos) del proceso que se utilizará en una organización (con diferentes niveles de detalle según lo que se necesite), para luego compararlo con la ejecución del proyecto donde se implementa (instancia) ese proceso de desarrollo. Esto permite medir qué tanto se está cumpliendo en el proyecto con lo que plantea la definición teórica del proceso de desarrollo (definición de roles, asignación de responsabilidades, actividades a realizar, etc.), a través de las auditorías de proyecto.

¡No es lo mismo que un estándar de proceso! El objetivo de estos modelos de calidad es acreditar que una organización tiene cierto nivel de madurez en su proceso de desarrollo adoptado, o bien que cierta área de esa organización tiene determinado nivel de madurez.

Existen un montón de modelos de calidad. Los más utilizados en Argentina son el CMMI, CMMI 2.0, etc.

Capability Maturity Model Integration – CMMI

Es un modelo de calidad para la mejora y evaluación de procesos para el desarrollo, mantenimiento y operación de sistemas de software, que constituye un conjunto de buenas prácticas que son publicadas en modelos.

Es un modelo de referencia, son descriptivos. Te dice que tienes que alcanzar determinado objetivo, pero vos definís qué práctica vas a implementar para lograrlo.

Este modelo empírico recopiló buenas prácticas a lo largo de la historia de diferentes organizaciones, tomando diferentes prácticas de aquellas que han logrado desarrollos exitosos.

El objetivo es que pueda ser usado para guiar la mejora de procesos en un proyecto, división o una organización completa.

La acreditación se logra evaluando de manera objetiva (comparación con el modelo de evaluación SCAMPI) y subjetiva (Experiencia y pensamiento del evaluador) el proceso, y el uso de este proceso instanciado en proyectos.

Constelaciones

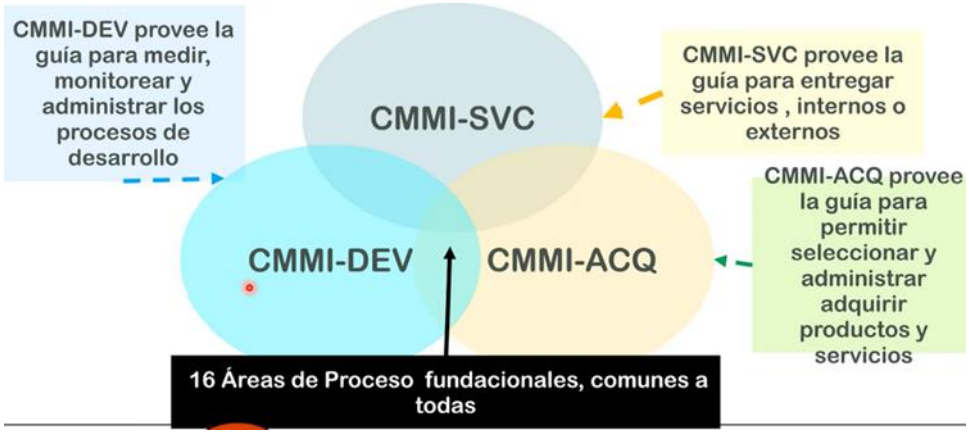
Existen 3 constelaciones o modelos de negocio que cubren los modelos de CMMI:

- **Desarrollo (CMMI-DEV)**: provee la guía para medir, monitorear y administrar los procesos de desarrollos de software. Tiene dos representaciones, por etapas y continua.
- **Adquisición (CMMI-ACQ):** provee la guía para permitir seleccionar, administrar y adquirir productos y servicios a otra empresa que posee su propia forma de trabajo, delegando el control de ese proyecto para obtener el

producto o servicio final. Es decir, cuando la empresa no hace cosas, sino que contrata gente que haga las cosas/trabajos por ellos (no es lo mismo que subcontratación de personal).

- **Servicios (CMMI-SVC):** provee la guía para entregar servicios, externos o internos.

Estas constelaciones poseen los modelos, capacitaciones y toda la información que es necesaria para que las empresas puedan acreditar distintos niveles de madurez.



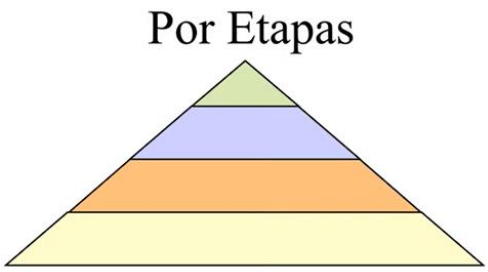
Áreas de proceso

- Un área de proceso es un conjunto de prácticas relacionadas en una zona que, cuando se implementan en conjunto, satisfacen un conjunto de objetivos considerados importantes para hacer mejoras significativas en el mismo proceso.
- Existen en total 22 áreas de procesos en todos los niveles de madurez, las cuales son iguales en ambas representaciones.
- 16 de esas 22 áreas de procesos son comunes a todas las constelaciones CMMI.
- Nivel 1: no existen áreas de proceso, ya que se considera que la organización está en estado de inmadurez.
- Nivel 2: son 7 en total, de las cuales la última es opcional (depende si la organización realiza o no actividades relacionadas)
    - **REQM** → Requirement Management — *Gestión de Reqs*
    - **PP** → Project Planning — *Gestión de proyecto*
    - **PMC** → Project Monitor and Control — *Gestión de proyecto*
    - **MA** → Measure and Analytics — *Métricas de proyecto*
    - **PPQA** → Product and Process Quality Assurance — *Disciplina transversal*
    - **SCM** → Software Configuration Management — *Disciplina transversal*
    - **SAM** → Supplier Agreement Management

Nivel	Categoría			
	Administración de Proyectos	Soporte	Administración de Procesos	Ingeniería
5 Optimizado		• Análisis y Causal y Resolución (CAR)	• Administración de Performance Organizacional (OID)	
4 Cuantitativamente Administrado	• Administración Cuantitativa del Proyecto (QPM)		• Performance del Proceso Organizacional (OPP)	
3 Definido	• Administración de Riesgos (RSKM) •Administración Integrada de Proyectos (IPM)	•Análisis y Resolución de Decisión (DAR)	• Definición del Proceso Organizacional (OPD) • Foco en el Proceso Organizacional (OPF) • Capacitación Organizacional (OT)	• Desarrollo de Requerimientos (RD) • Solución Técnica (TD) • Integración de Producto (PI) • Verificación (VER) • Validación (VAL)
2 Administrado	•Administración de Requerimientos (REQM) •Planificación de Proyectos (PP) • Monitoreo y Control de Proyectos (PMC) • Administración de Acuerdo con el Proveedor (SAM)	• Aseguramiento de calidad de Proceso y de Producto (PPQA) • Administración de Configuración (CM) • Medición y Análisis (MA)		
1 Inicial	**Procesos sin definir o improvisados**			

Representaciones CMMI-DEV

- **Por etapas:** CMM (el antecesor a CMMI), se basaba mucho en por etapas. Identificaba a las organizaciones y las dividía en maduras (eran las del nivel del 2 al 5) e inmaduras (nivel 1), mientras más maduras más capacidad de lograr sus objetivos, bajando sus riesgos. La ventaja de esta es que evaluaba la organización (el área que se quería trabajar). Esto facilita la comparación entre organizaciones. (madurez organizacional).



Es necesario cumplir con los lineamientos definidos para cada área de proceso de los niveles inferiores, y de las áreas de proceso definidas en el nivel que se está acreditando. Es decir, la acreditación es acumulativa para los niveles inferiores.

Niveles que se acreditan en la representación por etapas:

- **Inicial:** Las organizaciones en este nivel no disponen de un ambiente estable para el desarrollo y mantenimiento de software. Aunque se utilicen técnicas correctas de ingeniería, los esfuerzos se ven minados por falta de planificación. El éxito de los proyectos se basa la mayoría de las veces en el esfuerzo personal, aunque a menudo se producen fracasos y casi siempre retrasos y sobrecostes. El resultado de los proyectos es impredecible. Nivel de las organizaciones inmaduras.
- **Administrado:** la organización ha logrado todos los objetivos genéricos y específicos del nivel de madurez 2, es decir, los proyectos de la organización han asegurado que los requisitos son gestionados y de que los procesos se planifican, se realizan, se miden y controlado. También, hay un razonable seguimiento de la calidad.
- **Definido:** la organización ha alcanzado todos los objetivos específicos y de las áreas de proceso asignadas a los niveles de madurez 2 y 3. En este nivel los procesos están bien caracterizados y entendidos, y se describen en las normas, procedimientos, herramientas y métodos. Aquí los procesos se describen con más detalle y más rigurosidad que en el nivel de madurez 2. También se realiza formación del personal, técnicas de ingeniería más detalladas y un nivel más avanzado de métricas en los procesos. Se implementan técnicas de revisión de a pares.

- **Cuantitativamente administrado:** Se caracteriza porque las organizaciones disponen de un conjunto de métricas significativas de calidad y productividad, que se usan de modo sistemático para la toma de decisiones y la gestión de riesgos. El software resultante es de alta calidad. Las medidas detalladas del rendimiento de los procesos son recogidas y analizadas estadísticamente.
- **Optimizado:** este nivel se centra en mejora continua del rendimiento de los procesos a través de los aumentos y mejoras tecnológicas innovadoras.
  Los objetivos cuantitativos de mejora de procesos para la organización se establecen y se revisan de forma continua a fin de reflejar los cambios objetivos de negocio, y se utilizan como criterios para la administración de la mejora de procesos. El alcanzar estas áreas se detecta mediante la satisfacción o insatisfacción de varias metas claras y cuantificables.

Para ser de un nivel, se debe cumplir con los requisitos de ese nivel y con los de los anteriores.

Nosotros hacemos foco en el nivel 2, las áreas del proceso son:

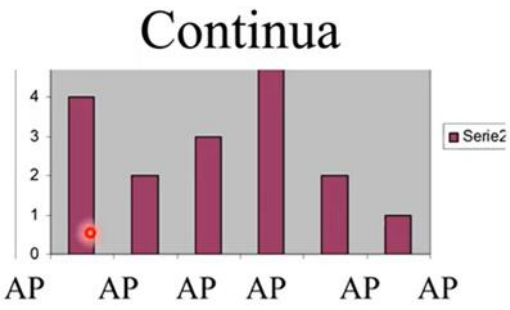


Administración de acuerdo con el proveedor no es obligatoria. Es necesaria si se contrata a una empresa externa como proveedora, ya que se debe comprobar que cumpla con el nivel de calidad que se tiene internamente. Hay 2 tipos de proveedores:

- Proveedor Tipo A: aquellos que cumplen con el mismo nivel de calidad.
- Proveedor Tipo B: aquellos que no cumplen con el mismo nivel de calidad.

Si alcanzo nivel 2 significa que la organización tiene la madurez para gestionar sus proyectos y que el producto que se producirá será algo esperable.

- **Continua:** El objetivo del uso de esta representación, es acreditar la capacidad de la organización ante cada una de las áreas de proceso de forma individual. En lugar de medir la madurez de toda la organización, mido la capacidad de un proceso en particular. Procesos de nivel cero son los que no se ejecutan (Por ejemplo: si le pregunto a una empresa si hace gestión de configuración de software y me dice que no lo hace, entonces, es nivel 0 en esa área). Apunta a mejorar áreas de proceso que uno elige. Se centra en la mejora de un proceso o un conjunto de ellos relacionados a un área de proceso que una organización desea mejorar, una organización puede obtener la certificación para un área de proceso en cierto nivel de capacidad.



Roles – Grupos

Los roles se adaptan a las necesidades de la organización. Cuando hablamos de grupo no nos referimos a conjunto de personas, sino de alguien que asuma ese rol.

Lo importante es que exista alguien responsable de cubrir las actividades de cada uno de los roles o grupos.

Relación de CMMI con Ágil

CMMI cara a cara con Ágil

Referencias:
Verde : Da soporte,
Negro: Neutral,
Rojo: Desigual

- "Nivel 1"
  - Identificar el alcance del trabajo
  - Realizar el trabajo
- "Nivel 2"
  - Política Organizacional para planear y ejecutar
  - Requerimientos, objetivos o planes
  - Recursos adecuados
  - Asignar responsabilidad y autoridad
  - Capacitar a las personas
  - Administración de Configuración para productos de trabajo elegidos
  - Identificar y participar involucrados
  - Monitorear y controlar el plan y tomar acciones correctivas si es necesario
  - Objetivamente monitorear adherencia a lo procesos y QA de productos y/o servicios
  - Revisar y resolver aspectos con el nivel de administración más álto



No coincide con el monitoreo objetivo pues hace referencia a las auditorías realizadas por agentes externos al equipo. Para poder usar CMMI siendo ágil, ambas deben ceder un poco: Ágil plantea de gestión ágil de requerimientos y no pide que haya un control de qué requerimientos tiene el producto en cada momento de tiempo, pero CMMI sí. De alguna forma se debe tener trazabilidad de los requerimientos en US y demás para CMMI. Así como CMMI acepta no tener el registro de las Daily por ejemplo. Ágil debe ceder Auditorías.

Negro:
Rojo:

- "Nivel 3"
  - Mantener un proceso definido
  - Medir la performance del proceso
- "Nivel 4"
  - Establecer y mantener objetivos cuantitativos para el proceso
  - Estabilizar la performance para uno o más subprocesos para determinar su habilidad para alcanzar logros
- "Nivel 5"
  - Asegurar mejora continua para dar soporte a los objetivos
  - Identificar y corregir causa raíz de los defectos



¿Por qué no coincide a medida que subimos de nivel? CMMI dice que tienes que definir un proceso y que después lo tienes que cumplir (lo que se verifica con las auditorías). En cambio, ágil en ningún momento evalúa que hayas seguido el proceso que definiste. CMMI a partir de nivel 3 o 4 plantea métricas/estadísticas de los proyectos y productos, y en base a la comparación de esas medidas se arma una medida del proceso. Ágil plantea que la experiencia no es extrapolable a otros proyectos.

Valores esenciales de cada metodología

CMMI	MÉTODOS ÁGILES
→ Medir y mejorar el proceso	→ Respuestas a clientes
[Mejores Procesos ↓ Mejor Producto]	→ Mínima sobrecarga
	→ Refinamiento de Requerimientos
	- Metáforas - Casos de negocio
→ Características de las personas	→ Características de las personas
- Disciplinados - Siguen reglas - Aversión al riesgo	- Comfortable - Creative - Risk Takers
→ Comunicación	→ Comunicación
- Organizacional - Macro	- Person to Person - Micro
→ Gestión de Conocimiento	→ Gestión de Conocimiento
- Activos de proceso	- Personas

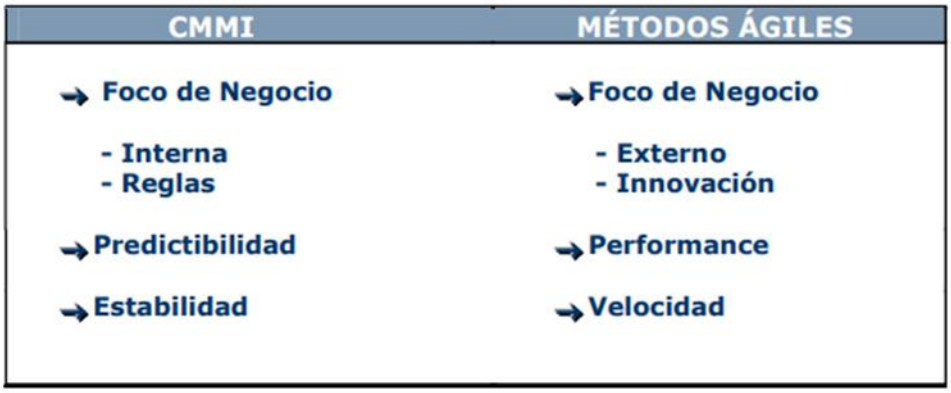
Características CMMI vs. Metodologías ágiles

CMMI	MÉTODOS ÁGILES
➥Mejora Organizacionalmente -Uniformidad -Nivelación	➥Mejora en el Proyecto - Tradición Oral - Innovación
➥Capacidad/Madurez - Éxito por Predictibilidad	➥Capacidad/Madurez - Éxito por darse cuenta de oportunidades
➥Cuerpo de Conocimiento - Cruzando dimensiones - Estandarizado	➥Cuerpo de Conocimiento - Personal - Evolucionando - Temporal
➥Reglas de Atajo - Desalentadas	➥Reglas de Atajo - Alentadas

CMMI	MÉTODOS ÁGILES
➥Comités	➥Individuos
➥Confianza del Cliente - En la Infraestructura del Proceso	➥ Confianza del Cliente - Sw funcionando, Participantes
➥Cargado al frente - Mover a la derecha	➥Conducido por Pruebas - Mover a la izquierda
➥Alcance de la vista [Involucrado, Producto] - Amplio - Inclusivo - Organziacional	➥Alcance de la vista [Involucrado, Producto] - Pequeño - Focalizado
➥Nivel de Discusión - Palabras - Definiciones - Duradero - Exhaustivo	➥Nivel de Discusión - Trabajo en mano

En cuanto al enfoque

CMMI	MÉTODOS ÁGILES
➥Descriptivo	➥Prescriptivo
➥Cuantitativo - Número científicos y duros	➥Cualitativo - Conocimiento tácito
➥Universalidad	➥Situacional
➥Actividades	➥Producto
➥Estratégico	➥Táctico
➥"¿Cómo lo llamaremos?"	➥"Sólo hazlo!"
➥Gestión de Riesgos - Proactiva	➥Gestión de Riesgos - Reactiva



Similitudes entre ambas

- Meta: organizaciones de alto desempeño;
- Ambas planean;
- Ambas son Consultant Money Makers (CMMs);
- Ninguna es completa;
- Ninguno es aplicable a cualquier proyecto;
- No nuevas ideas;
- Ambas tienen reglas (reglas == requerimientos del proceso):
    - Incumplir tiene serias repercusiones;
    - 'SEPG' (Grupo de proceso de ingeniería de software) & 'Política de Proceso'.

Diferentes tipos de Auditorias: Auditorías de Proyecto y Auditorías al Grupo de Calidad.

Auditorías de calidad de Software

Es una actividad incluida dentro de las disciplinas de soporte, la cual implica una evaluación **independiente** (realizada por un grupo de trabajo que no tienen nada que ver con el equipo del proyecto) de productos o procesos de software que permiten asegurar el cumplimiento con estándares, lineamientos, especificaciones y procedimientos basada en un criterio objetivo incluyendo documentación.

Las auditorías implican esfuerzo y costo para los proyectos, sin embargo, sus beneficios son superiores.

Son un instrumento para el Aseguramiento de Calidad en el Software.

Beneficios de las auditorías

- Se obtienen opiniones objetivas e independientes;
- Permiten identificar áreas de insatisfacción potenciales del cliente:
- Permite asegurar que se están cumpliendo con las expectativas;
- Permite dar visibilidad sobre los procesos de trabajo;
    - Permite visualizar los puntos fuertes de una organización;

- Permite identificar oportunidades de mejora;

- Da visibilidad a la gerencia sobre los procesos de trabajo;
- Determinan que se implementen de manera efectiva: el proceso de desarrollo organizacional y del proyecto, y las actividades de soporte.

Tipos de auditorías

Auditoría de proyecto

Responsable de ver que el proyecto se haya ejecutado con el proceso que se definió. Esto bajo la premisa de que en un proyecto se debe realizar lo que define el proceso, obviamente adaptado a lo que sirve en ese contexto del proyecto particular (teniendo en cuenta que, para obtener una acreditación de calidad, se tiene que ajustar hasta cierto punto al modelo teórico de referencia).

Lo que se hace es tomar evidencias de lo que se está haciendo y contrastarlo con lo documentado en las ERS, arquitectura, diseño, etc.

Acá puede haber diferencias respecto a metodologías tradicionales o empíricas. En SCRUM, por ejemplo, el mismo equipo realiza estas auditorías en la ceremonia de Sprint Retrospective. En cambio, en metodologías tradicionales, el auditor debe ser externo al proyecto y a veces a la empresa.

En resumen: se evalúa el proceso adoptado (definición teórica), pero esa adopción del proceso se da en un proyecto, por ende, la auditoría se realiza sobre el proyecto.

Estas auditorías se llevan a cabo de acuerdo a lo establecido en las PACS (Plan de Aseguramiento de Calidad de Software).

El objetivo de esta auditoría es verificar objetivamente la consistencia del producto a medida que evoluciona a lo largo del proceso de desarrollo.

Auditorías de Configuración Funcional

Valida que el producto cumpla con los requerimientos. La auditoría funcional compara el software que se ha construido (incluyendo sus formas ejecutables y su documentación disponible) con los requerimientos de software especificados en la ERS.

El propósito de la auditoría funcional es asegurar que el código implementa sólo y completamente los requerimientos y las capacidades funcionales descriptos en la ERS.

El responsable de QA deberá validar si la matriz de rastreabilidad está actualizada.

Auditoría de configuración física

Valida que el ítem de configuración cumpla con la documentación técnica que lo describe (esto permiten la trazabilidad y la satisfacción de los requerimientos).

La auditoría física compara el código con la documentación de soporte.

Su propósito es asegurar que la documentación que se entregará es consistente y describe correctamente al código desarrollado.

El PACS debería indicar la persona responsable de realizar la auditoría física.

El software podrá entregarse sólo cuando se hayan arreglado las desviaciones encontradas.

Auditorías externas de Aseguramiento de Calidad (No sé si son un tipo más)

Son auditorías realizadas por un personal externo a la organización, a quienes se encargan de realizar las auditorías de calidad. Esto permite evaluar a los miembros de QA, para determinar si sus evaluaciones dentro de la organización se están realizando de forma correcta.

Roles en una auditoría

- **Auditado:** Participa en la auditoría, y es quien propone la fecha de la auditoría, entrega evidencia, contesta las dudas del auditor, propone planes de acción para las desviaciones encontradas, contesta el reporte de auditoría, propone el plan de acción para las deficiencias citadas en el reporte. Generalmente es el Líder de Proyecto, pero puede ser otro.

- **Auditor:** puede ser una persona o dos y deben ser de afuera del proyecto que se está auditando. Puede ser el grupo de aseguramiento de calidad, que es el grupo que le da soporte y auditorías de estos tipos.
    - Acuerda la fecha de la auditoría.
    - Comunica el alcance de esta.
    - Recolecta y analiza la evidencia objetiva que es relevante y suficiente para tomar conclusiones acerca del proyecto auditado.
    - Realiza la auditoría.
    - Prepara el reporte.
    - realiza el seguimiento de los planes de acción acordados con el auditado.

- **Gerente de SQA (Software Quality Assurance):** Es quien prepara el plan de auditoría, calcula su costo, asigna recursos, resuelve las no-conformidades entre auditor y auditado. (orientado a la gestión de la auditoría)

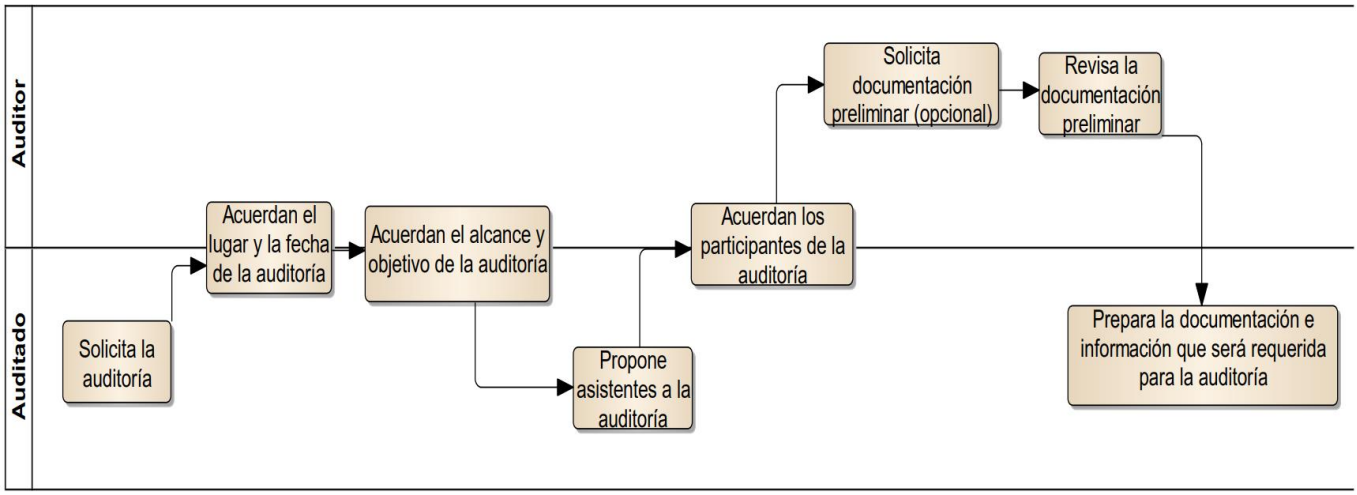
Proceso de Auditorías: Responsabilidades. Preparación y ejecución. Reporte y seguimiento.

Etapas de una auditoría

1. Preparación y planificación

No son sorpresa. El auditado solicita la auditoria y junto con el auditor definen la fecha, el alcance y objetivo, acordando los participantes de esta.

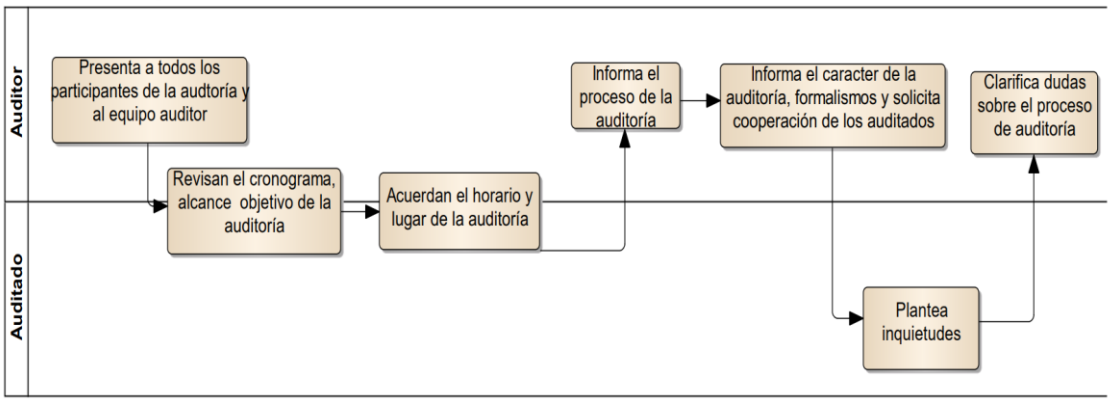
Se deben definir métricas de auditoría, como por ejemplo: esfuerzo por auditoría, cantidad de desviaciones, duración de la auditoría, cantidad de desviaciones clasificadas por auditor de CMMI, etc.



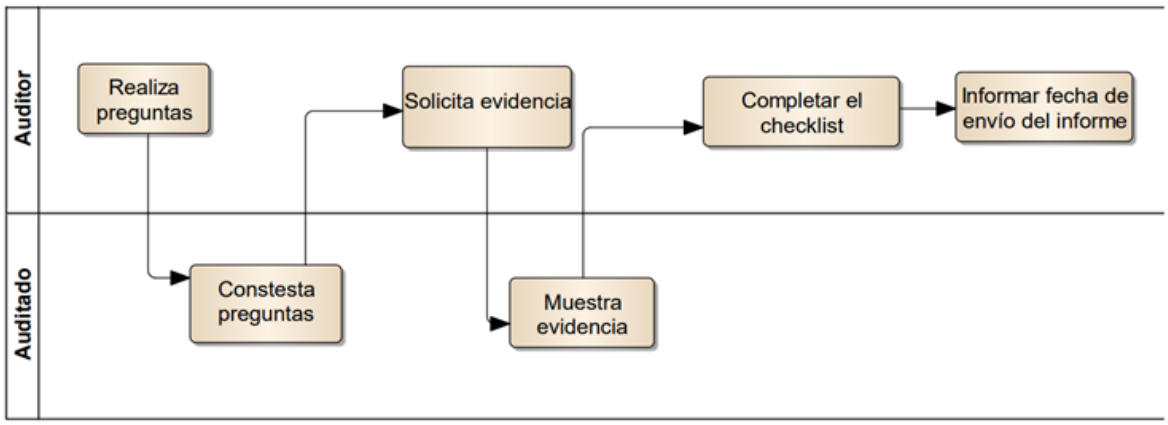
2. Ejecución

El auditor escucha lo que la gente dice que hace y luego ve la documentación, pide evidencia (lo que debería estar haciéndose).

- Reunión de apertura: se informa cómo será el proceso, indicando el lugar y la hora.



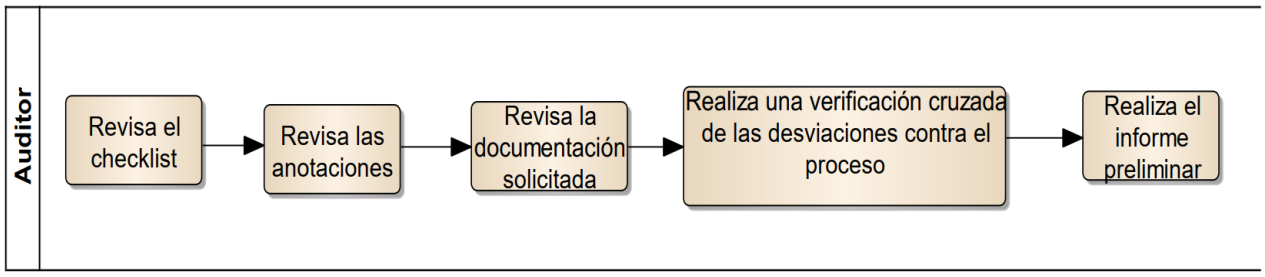
- Ejecución: se responden las preguntas, se muestra la evidencia y se completa el checklist. El auditor escucha lo que la gente dice que hace y luego ve la documentación (lo que debería estar haciéndose).



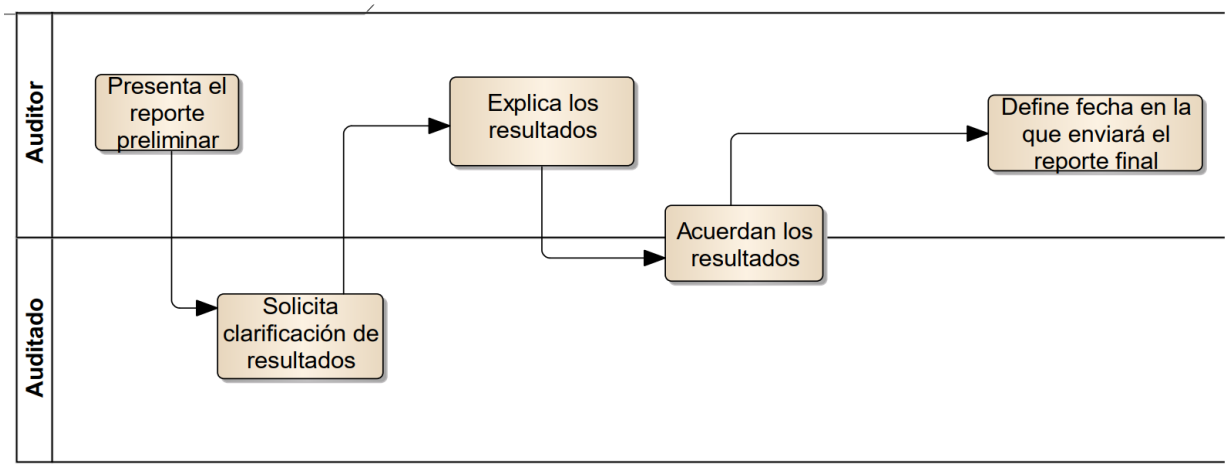
3. Análisis y reporte de resultados

El auditor realiza el reporte de resultados y el auditado analiza la respuesta, puede o no estar de acuerdo con algunas prácticas y se deja asentado el documento final. Se evalúan los resultados, se hace una reunión de cierre y se hace entrega del reporte final.

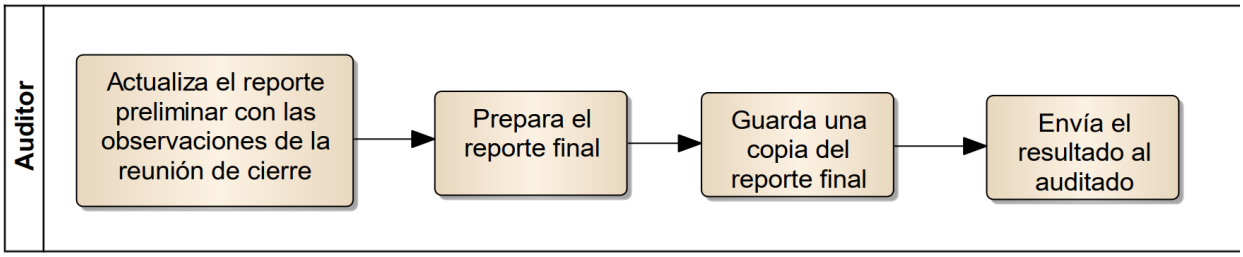
- Evaluación de Resultados



- Reunión de cierre

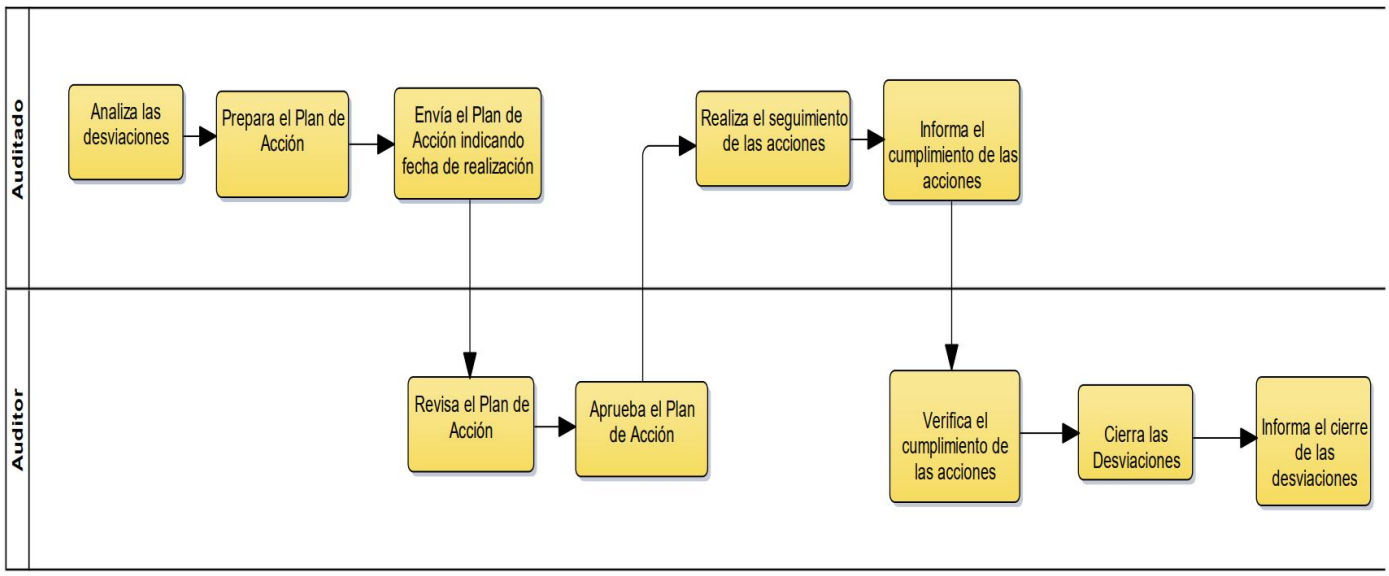


- Entrega de reporte final



4. Seguimiento

Se analizan las desviaciones y prepara un plan de acción que se envía al auditor para que lo revise y apruebe. En función del acuerdo entre auditado y auditor, puede que el auditor haga un seguimiento de las desviaciones que encontró hasta que considere que han sido resueltas.



Herramientas y técnicas utilizadas en auditorías

- Checklists: tienen preguntas tipo para garantizar que independientemente de quién haga la auditoría el foco de las cosas que se controlan sea el mismo. Son de mínima, es decir, se debe garantizar de mínimo contestar estas preguntas, pero el auditor puede solicitar más cosas (hacer más preguntas).
- Muestreo: consiste en seleccionar una muestra representativa de los productos y/o procesos a auditar.
- Revisión de registros
- Herramientas automatizadas

Resultados/hallazgos de una auditoría

- **Buenas prácticas:** cuando es algo superador a lo definido que tenemos que hacer. Mucho mejor de los esperado. Ejemplo: se armó una muy buena herramienta para gestionar el plan de proyecto para que la gente pueda accederlo y mejorarlo.
- **Desviaciones:** cualquier cosa que no se hizo o que no se hizo cómo el proceso lo definió. Hay dos grados:
    - No Adecuado: lo que realmente está mal.
    - Necesita Mejora: si lo estás haciendo, pero necesita mejorarse.

  Requiere un plan de acción por parte del auditado.
- **Observaciones:** son cosas que se advierten por el auditor que no llegan a ser desviaciones pero que pueden llegar a ser riesgosas las prácticas y pueden llegar a generar un problema, por eso se destacan a **consideración** del equipo para que ellos decidan si hacerlo o no. Algo para revisar. Deberían mejorarse, pero no requieren un plan de acción. Ejemplo: técnica de estimación mala.

Reporte de auditoría

En este documento se deben informar las siguientes desviaciones:

- Cualquier desviación que resulta en la disconformidad de un producto respecto de sus requerimientos
- Cualquier desviación al proceso definido o a los requerimientos documentados.

Métricas de auditoría

Cada organización establece según sea apropiada o no:

- Esfuerzo por auditoría.
- Cantidad de desviaciones.
- Duración de auditoría.

Calidad de Producto: Planificación de pruebas para el software- Niveles y tipos de pruebas para el software. Técnicas y herramientas para probar software. Técnicas y Herramientas para la realización de revisiones técnicas del software.

Verificación y validación

- Es un proceso de ciclo de vida completo.
- Inicia con las revisiones de los requerimientos y continúa con las revisiones del diseño, inspecciones del código hasta la prueba.
- Validación: ¿Estamos construyendo el producto correcto?
- Verificación: ¿Estamos construyendo el producto correctamente?

Falla: error en un producto de trabajo.

Producto de trabajo: salida de cualquier actividad correspondiente al ciclo de vida de desarrollo.

¿Por qué existen las fallas?

- Problemas de comunicación.
- Limitaciones de memoria.



Principios

- Prevenir es mejor que curar.
- Evitar es más efectivo que eliminar.
- La retroalimentación enseña efectivamente.
- Priorizar lo rentable.

- Olvidarse de la perfección, no se puede conseguir.
- Enseñar a pescar, en lugar de dar el pescado.

Existen dos aproximaciones complementarias:

- Revisiones técnicas.
- Pruebas de Software.

Revisiones técnicas – Peer Review

Es una actividad realizada por un colega, cuyo propósito es mejorar la calidad de software, mediante la detección temprana de errores en cualquier artefacto que se genere, por ejemplo, en el código, requerimientos, diseño, arquitectura, riesgos, estimaciones, planes, etc.

Es un proceso estático de validación y verificación; y no corrige errores.

Objetivo: introducir el concepto de verificación y validación. Busca evitar el retrabajo. Motiva a realizar un mejor trabajo.

Nunca se pone en juicio el autor del artefacto, sólo el artefacto en sí, ya que es necesario que se revelen todos los errores entre los miembros del equipo. Se debe desarrollar una cultura de trabajo que brinde apoyo y no buscar culpables cuando se descubran errores.

Es una actividad que trascendió cualquier metodología, filosofía y en muchas ocasiones la utiliza ágil, para transformar las auditorías en peer reviews evitando así traer a alguien externo al equipo.

Por ejemplo, siendo un programador Jr, solicitar a un Sr una revisión técnica respecto a un patrón de diseño arquitectónico que se quiere implementar.

Ventajas:

- Pueden descubrirse muchos errores;
- Pueden inspeccionarse versiones incompletas;
- Pueden considerarse otros atributos de calidad.

Desventajas:

- Es difícil introducir las inspecciones formales;
- Sobrecargan al inicio los costos y conducen a un ahorro sólo después de que los equipos adquieran experiencia en su uso;
- Requieren tiempo para organizarse y “parecen” ralentizar el proceso de desarrollo.

Tipos de revisiones

- Formales: tienen un proceso definido con roles.
  - Inspecciones (Inspección de código de Fagan e inspección de Gilb).
- Informales: cuando no existe un proceso de cómo realizarlo.
  - Walkthrough o recorrido.

Walkthrough o recorrido (informal)

Técnica de análisis estático en la que un diseñador o programador dirige miembros del equipo de desarrollo y otras partes interesadas a través de un producto de software, donde los participantes formulan preguntas y realizan comentarios acerca de posibles errores, violación de estándares de desarrollo y otros problemas.

No existe un proceso formal. Consiste en reuniones informales de colegas donde se debaten las correcciones a aplicar al producto de trabajo. No hay control del proceso.

Tiene los siguientes objetivos:

1. Mínima Sobrecarga
2. Capacitación de Desarrolladores
3. Rápido retorno

Esta técnica no obtiene métricas para aprender y dejar registros. Sin embargo, es una de las técnicas más elegidas en los enfoques agiles. Hay una junta de revisión entre colegas después de completar cada iteración del software (una revisión rápida), en la que pueden exponerse los conflictos y problemas de calidad sobre el producto.

Inspecciones (formal)

Tiene un proceso formal y cuenta con un conjunto de roles. Es necesario la utilización de un checklist, que ayuda a la memoria para saber que cosas controlar. Se toman métricas y finalmente se realiza un reporte de la revisión al final de la inspección para analizar los defectos encontrados.

Es una actividad que garantiza la calidad del software, cuyo éxito depende de la planificación. Tiene como objetivos:

1. Descubrir errores.
2. Verificar que el software alcanza sus requerimientos.
3. Garantizar que el software fue construido de acuerdo con ciertos estándares.
4. Conseguir un software desarrollado de manera uniforme.
5. Hacer que los proyectos sean más manejables.

Son procesos time boxing y exigen un alto esfuerzo intelectual.

Roles participantes

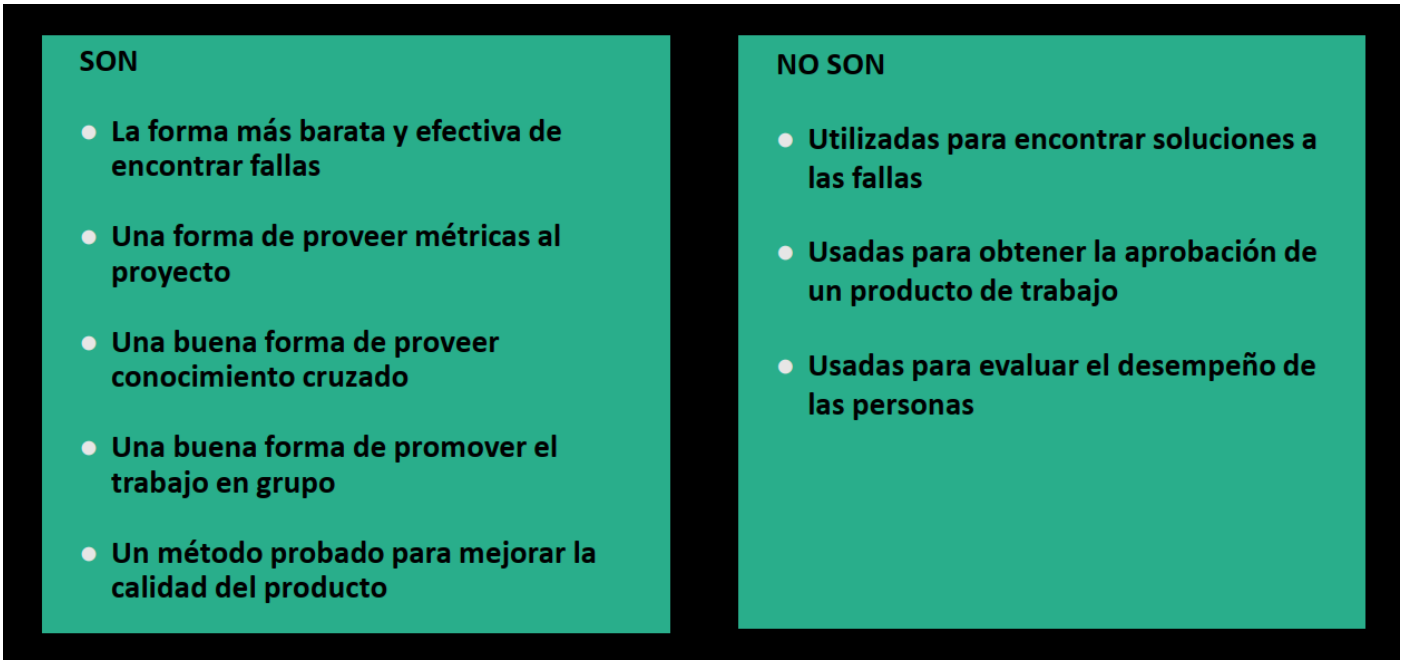
Al ser una técnica formal, si o si debe contar con los siguientes roles:

1. **Autor:** creador o encargado de mantener el producto a inspeccionar. Inicia el proceso seleccionando a un moderador y junto a este eligen al resto de los roles. Entrega el producto a ser inspeccionado al moderador.
2. **Moderador:** planifica y lidera la revisión. Trabaja junto al autor para elegir los demás roles. Entrega el producto a inspeccionar al inspector 2 días antes de la reunión. Coordina la reunión de forma tal que no ocurran conductas inapropiadas y realiza un seguimiento de los defectos encontrados.
3. **Anotador:** registra los hallazgos de la inspección. Usualmente termina confeccionado el reporte de la revisión.
4. **Lector:** lee el producto a ser inspeccionado. Este rol es necesario para que los participantes no se dispersen.

5. **Inspector:** Examina el producto antes de la reunión para encontrar defectos. Registra sus tiempos de preparación. todos pueden ser inspectores. Todos pueden inspeccionar.

Ciertos roles pueden ser asumidos por la misma persona.

Las revisiones técnicas…



Etapas del proceso de inspección

1. **Planificación:** el moderador, a pedido del autor, planifica la inspección definiendo el lugar, el tiempo de duración y los roles. La duración de las reuniones no debe superar las 2 horas, dado que la inspección es un proceso de alto esfuerzo intelectual.
2. **Visión general:** esta etapa es opcional. El autor realiza una descripción general del producto a inspeccionar.
3. **Preparación:** es la preparación de cada rol para la reunión. Cada rol adquiere una copia del producto de trabajo que deberá leer y analizar, con el fin de encontrar potenciales defectos. Esta preparación permite que la reunión de inspección sea más productiva.
4. **Reunión de inspección**: el equipo realiza un análisis para recolectar los potenciales defectos previos y descartar falsos positivos. El lector lee el producto de trabajo y los inspectores comparten los defectos encontrados, los cuales son registrados por el anotador. La reunión finaliza con una conclusión acerca de si se acepta o no el producto de trabajo inspeccionado. Finalmente se realiza un informe detallando que se revisó, por quien, que se descubrió y que se concluyó.
5. **Corrección:** finalizada la reunión, el autor realiza las correcciones de los defectos encontradas.
6. **Seguimiento:** Dependiendo de la gravedad puede existir un proceso de re-inspección. En caso de que los defectos sean graves, se realiza nuevamente una inspección. De lo contrario, si el defecto era muy simple, el autor simplemente lo corrige. Otros defectos que no implican una re-inspección, pueden implicar que el autor se reúna con el moderador únicamente para tratar las correcciones realizadas.

Definición de estándares

Un aspecto importante del aseguramiento de calidad es la definición o selección de estándares que deben aplicarse al proceso de desarrollo de software o al producto de software.

Los estándares son importantes por tres razones:

- Se basan en conocimiento sobre la mejor o más adecuada práctica para la organización. Con frecuencia, este conocimiento se adquiere sólo después de una gran cantidad de pruebas y errores. Configurarla dentro de un estándar, ayuda a la empresa a reutilizar esta experiencia y a evitar errores del pasado.
- Al usar estándares se establece una base para decidir si se logró un nivel de calidad requerido. Desde luego, esto depende del establecimiento de estándares que reflejen las expectativas del usuario para la confiabilidad, la usabilidad y el rendimiento del software.
- Los estándares aseguran que todos los trabajadores dentro de una organización adopten las mismas prácticas. En consecuencia, se reduce el esfuerzo de aprendizaje requerido al iniciar un nuevo proyecto o trabajo.

Para la gestión de calidad de software se utilizan dos estándares:

- Estándares de producto: Se aplican al producto de software a desarrollar. Incluyen estándares de documentos y estándares de codificación.
- Estándares de proceso: establecen los procesos que deben seguirse durante el desarrollo, por ejemplo, incluyen las definiciones de requerimientos, procesos de diseño y validación, etc.

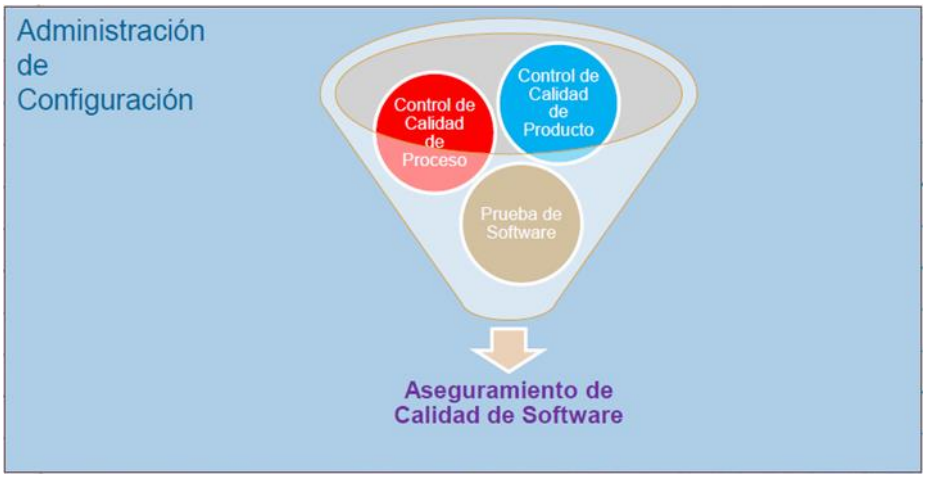
Testing en ambientes Ágiles.

Contexto

El testing de software, es una de las tareas a realizar en el ámbito del aseguramiento de calidad de software, en conjunto con el control de calidad del proceso y del producto. La calidad del software se obtiene y se logra a lo largo de todo el desarrollo de este (detectar errores en etapas tempranas es siempre menos costoso, que detectarlos después), por lo que una buena administración de configuración (SCM) es el puntapié inicial para permitir la realización de tareas de aseguramiento de calidad.

Hay que recordar que SCM permite determinar la configuración de ítems, trazabilidad, control de cambios, auditorías del producto y reportes de estado. QA agrega a esto, el control de calidad del proceso. Existen diversos estándares (ISO, CMMI, etc.) que permiten acreditar la calidad del proceso, debido a que la calidad del producto no es algo estandarizable por la variación de requerimientos de un caso a otro.

Dentro del aseguramiento de la calidad del software, existen actividades destinadas al control de calidad del proceso y del producto. Se realiza un control de calidad sobre el proceso de desarrollo, bajo el argumento de que el proceso de desarrollo posee un gran impacto en la calidad del producto. Es decir, en teoría la calidad del producto depende de la calidad del proceso que se está utilizando.



Luego de desarrollar el software, las actividades de testing revelan la calidad del Software en sí, es decir, si el producto satisface con los requerimientos definidos por el cliente en etapas anteriores. Cabe destacar que la actividad de testing no asegura la calidad por sí solo, sino que debe estar acompañada de las demás actividades.

QA != Testing

Definición de Testing

Proceso mediante el cual se somete a un software o un componente de software a condiciones específicas con el fin de determinar y demostrar si el mismo es válido o no en función de los requerimientos especificados.

El testing es una actividad destructiva cuyo objetivo es encontrar defectos, cuya presencia es asumida de antemano en el código.

Testing de software es un proceso, o una serie de procesos, diseñados para asegurar que el código hace lo que fue diseñado para hacer, o visto de otra manera, que no haga nada que no esté intencionado. Se dice que es el proceso de ejecutar un programa con la intención de encontrar fallas. El software debería ser predecible y consistente, sin presentarle sorpresas a los usuarios.

El Testing es parte del QA (Quality Assurance) y forma parte del control de calidad. Se hace cuando el producto ya está construido (o lo que se desea testear), en cambio el QA se aplica en todo el ciclo de vida (las buenas prácticas en la implementación son parte de QA).

Las actividades de testing se consideran exitosas si se encuentran defectos en la ejecución de los casos de prueba. Por lo que, un software con alta calidad lograda a lo largo de todo el desarrollo (implementación de QA), dificulta encontrar defectos y puede dirigir a un testing no exitoso. Por otro lado, si el software tiene un bajo nivel de calidad, el costo de retrabajo es alto, ya que van a existir muchos idas y vueltas entre las actividades de testing y las de desarrollo, para la corrección de errores.

El testing es una actividad costosa, por lo que es necesario lograr un nivel de cobertura óptima teniendo en cuenta el costo-beneficio. Nunca se podrá cubrir el 100% de las pruebas, por cuestiones lógicas de tiempo y costos. Esta cobertura se comienza a definir desde el momento en los cuales se establecen los criterios de aceptación.

Principios del Testing

- El Testing es una actividad destructiva que encuentra defectos cuya presencia se asume.
- Se testea con una actitud negativa tratando de demostrar que algo es incorrecto.
- El Testing exitoso es aquel que encuentra defectos.
- El costo del Testing está entre un 30% y un 50% del valor del producto.
- El Testing pone en evidencia defectos, pero no agrega calidad ni garantiza que el producto no tiene errores.
- Un desarrollo exitoso puede llevar a un Testing no exitoso.
- El Testing es necesario siempre.
- Se parte de la suposición de que siempre se tendrá defectos por encontrar, ya que es una característica inherente de los productos desarrollados por equipos de personas.
- Puede empezar antes de la codificación porque necesita de los requerimientos para armar los casos de prueba.
- El testing NO asegura que se tenga un producto de calidad (ni la agrega), ni que el proceso por el que se desarrolló sea de calidad.

- Agrupamiento de defectos: los defectos en el software suelen agruparse en un conjunto limitado de módulos o áreas (regla de Pareto, 80% de los defectos se agrupan en el 20% de la funcionalidad). Bajo este principio, las pruebas deben priorizarse y enfocar el esfuerzo en ese conjunto acotado de funcionalidades.

Principio	Explicación
1. Una parte necesaria de un caso de prueba es definir el resultado esperado.	Si el resultado esperado de un caso de prueba no ha sido predefinido, lo más probable es que un resultado erróneo se interpretará como un resultado correcto, debido a el fenómeno de "el ojo viendo lo que quiere ver", a pesar de la definición destructiva adecuada de prueba, hay todavía un deseo subconsciente de ver el resultado correcto.
2. Un programador debe evitar testear su propio programa.	Los propios desarrolladores “no quieren” encontrar sus propios defectos, por lo que el carácter de pruebas destructivas se deja de lado. Además, puede que el programa contenga errores por malentendidos del programador sobre el dominio, y si él mismo testea, no se van a detectar estos defectos.
3. Una empresa de desarrollo no debe testear sus propios programas.	Misma explicación que el principio 2 orientado a empresas, agregando que si las mismas empresas testean sus programas, es posible que destinen menos recursos y eviten encontrar defectos para cumplir con el calendario y los costos establecidos.
4. Cualquier proceso de prueba debe incluir una inspección minuciosa de los resultados de cada prueba.	Se deben realizar inspecciones minuciosas para detectar la totalidad de los defectos encontrados tras la ejecución de una prueba, ya que pueden surgir más de un defecto (y esto es lo que se busca) por cada caso de prueba.
5. Los casos de prueba deben escribirse para condiciones de entrada que no son válidas e inesperadas, así como para las que son válidas y esperadas.	Muchos errores que se descubren repentinamente en el desarrollo de software aparecen cuando se usa de alguna manera nueva o inesperada, y no responde cómo debería (catcheando el error)
6. Examinar un programa para ver si no hace lo que se supone que debe hacer es sólo la mitad de la batalla; la otra mitad es ver si el programa hace lo que no se supone que debe hacer	Los programas deben ser examinados para detectar defectos secundarios no deseados.
7. Evite los casos de prueba desechables, a menos que el programa sea realmente un programa de descarte.	Los casos de pruebas utilizados en el testing deben ser reproducibles, es decir, no se deben realizar casos de prueba sobre la marcha (ad-hoc) ya que es imposible reportar un defecto sin tener las condiciones y los pasos en los cuáles el defecto surgió. Además, realizar testing es muy costoso, por lo cuales los casos de prueba deben ser reutilizados para volver a testear escenarios luego de la corrección de errores.

8. No planee un esfuerzo de prueba bajo la suposición tácita de que no se encontrarán errores.	Es un error pensar de esta manera al momento de realizar software. Se debe tener en claro la definición de testing, en la cual se define que el objetivo de esta actividad es encontrar errores, y se presume de antemano su existencia.
9. La probabilidad de que existan más errores en una parte de un programa es proporcional al número de errores ya encontrados en esa parte.	El concepto es útil porque nos da una idea de en qué sección del programa hacer foco o asignar más recursos, si una sección particular de un programa parece ser mucho más propenso a errores que otras secciones, es recomendable realizar pruebas adicionales y es probable que encontremos más errores.
10. Las pruebas son extremadamente creativas e intelectualmente desafiantes.	Aunque existen métodos y estrategias para abarcar un mejor nivel de cobertura testing en los casos de pruebas, siempre es necesario un poco de creatividad del diseñador de estos.

Mitos del Testing

- "El testing es el proceso para demostrar que los errores no están presentes".
- "El propósito del testing es demostrar que un programa realiza sus funciones previstas de forma correcta"
- "El testing es el proceso que demuestra que un programa hace lo que se supone que debe hacer''

Estas definiciones o afirmaciones sobre el testing son incorrectas, ya que el objetivo de testear un programa es agregar valor al producto revelando su calidad y brindando confianza en el software, de forma más concreta, encontrar y remover defectos en el código de este. Entonces, no se prueba un sistema para mostrar que funciona, sino que se comienza con la suposición de que el software contiene defectos, y se realiza el testing para encontrar la mayor cantidad de ellos posible.

Por otro lado, un programa puede hacer lo que se supone que debe hacer, y aun así, contener defectos. Es decir, un error está claramente presente si un programa no hace lo que se supone que debe hacer, pero los errores también están presentes si un programa hace lo que no se supone que debe hacer.

¿Cuánto Testing es suficiente?

El **testing exhaustivo es imposible** por la cantidad de tiempo que requiere. El momento en que se deja de hacer testing depende del nivel de riesgo o costo asociado al proyecto. Los riesgos permiten definir prioridades de que se debe testear primero y con qué esfuerzo.

El criterio de aceptación se utiliza normalmente para decidir si una determinada fase de testing ha sido completada. Este puede ser definido en términos de:

- Costos.
- % de tests corridos sin fallas.
- Inexistencia de defectos de una determinada severidad.

- Pasa exitosamente el conjunto de pruebas diseñado y la cobertura estructural.
- Good Enough: Cierta cantidad de fallas no críticas es aceptable.
- Defectos detectados es similar a la cantidad de defectos estimados.

El criterio de aceptación sirve para definir y negociar en ágil con el Product Owner y en tradicional con el Líder del Proyecto a cuántos defectos son aceptables para terminar.

Conceptos importantes

Defecto versus Error

La diferencia entre ambos conceptos es el momento en el cual se detectan y solucionan los errores o defectos. Un error es detectado y corregido en una misma etapa, y un defecto es un error que se traslada de una etapa a otra etapa posterior en la cual se introdujo. El testing encuentra defectos, ya que son errores que surgieron en etapas anteriores, pero se detectan en la etapa de prueba.

Hay que aclarar que ambos conceptos pueden o no generar fallas en el sistema, es decir, un mal funcionamiento de este. Por ejemplo, un error en los colores de una interfaz no es una falla, ya que no conllevan a un mal funcionamiento del sistema.

Defecto

Un defecto posee dos características principales, que permiten catalogarlos en la etapa de testing:

- Severidad: define la gravedad del defecto, y es determinada por la persona que realiza el testing, por lo que esta característica es de carácter técnico. El valor de la severidad se asigna dependiendo de la siguiente escala:
    - Bloqueante → el defecto no permite continuar con la ejecución del sistema.
    - Crítico → el sistema funciona, pero la funcionalidad que se está testeando tiene un defecto crítico.
    - Mayor → la funcionalidad que se está testeando funciona, pero no de forma correcta.
    - Menor → la funcionalidad se ejecuta correctamente, pero con advertencias erróneas o errores de baja importancia.
    - Cosmética → formato de fechas, formato de números, distribución de componentes en una GUI, temas de presentación de interfaces, etc.
- Prioridad: la prioridad define el impacto del defecto en la funcionalidad para el negocio, y permite ordenar la atención de los defectos según las necesidades del cliente. Una escala posible para la prioridad puede ser:
    - Urgencia
    - Alta
    - Media
    - Baja

Estas características son independientes entre sí, ya que varían dependiendo del tipo de negocio y de sistema que se está desarrollando. Por ejemplo, un defecto cosmético puede tener baja prioridad para un sistema de inventarios, pero puede tener una alta prioridad si se trata de un sistema de una empresa de marketing, en donde el aspecto y la presentación es algo muy importante.

Casos de prueba

Son una secuencia de pasos que hay que seguir para obtener un resultado esperado, y se tienen que dar ciertas condiciones previas que fijan una situación para que se pueda ejecutar la prueba.

Es el artefacto más importante del Testing. Este se hace una vez, pero sirve para realizar infinitas pruebas, debiendo obtener en todas el mismo resultado esperado.

Se trata de minimizar la cantidad de casos de prueba maximizando la cantidad de defectos encontrados.

El objetivo de los casos de prueba es descubrir defectos.

Los casos de prueba salen de los requerimientos, permitiendo hacer tanto la verificación como la validación.

Está compuesto por: un objetivo (lo que se desea controlar), condiciones de prueba (Datos de entrada y de entorno que deben estar presente para llevarse a cabo la prueba) y un resultado esperado.

Ciclos de prueba

Es la ejecución de un conjunto de casos de prueba en una versión determinada del producto. Generalmente se tienen 2 ciclos, y el primero es conocido como ciclo 0. El ciclo 0 siempre es manual, es donde se configura todo y a partir del ciclo 1 ya se pueden automatizar las pruebas.

En caso de detectarse defectos, el producto vuelve a desarrollo para su corrección.

Si existe una cantidad alta de ciclos de prueba, es probable que QA no sea bueno, por lo que deberían revisarse ciertos aspectos acerca de la calidad del proceso y producto, de manera de evitar tener grandes cantidades de ciclos, que se traducen en mucho retrabajo, lo cual a su vez conduce a incrementar los costos de desarrollo.

Ciclo de pruebas con regresión

Este concepto plantea la necesidad de ejecutar exhaustivamente todos los casos de prueba en cada ciclo de prueba, como si fuesen el ciclo 0. Este planteo es el ideal, pero el menos utilizado, por cuestiones de practicidad y tiempo.

El otro enfoque, consiste en aplicar una prueba exhaustiva de todos los casos de prueba en el ciclo 0, pero a partir del ciclo 1 (si el incremento o producto vuelve nuevamente a testing) ejecutar sólo los casos de prueba asociados a defectos encontrados previamente, y no los casos de prueba que hayan pasado sin encontrar defectos. Esto permite agilizar el proceso de pruebas, pero estadísticamente es muy probable que al arreglar un defecto reportado por testing, en el desarrollo de su corrección se introduzcan nuevos errores en otras partes del producto. Por lo que, si no se prueban todos los casos, a pesar de que previamente no se hayan detectado defectos, pueden encontrarse nuevos, debido al proceso descrito anteriormente.

Una solución a esta encrucijada es no aplicar regresión (ejecutar sólo los casos de prueba con defectos asociados), y cuando finalmente estén todos los defectos “corregidos”, hacer una ejecución de todos los casos de prueba (como si se aplicara regresión), para aumentar la confianza de que no se han introducido nuevos errores en las correcciones.

La automatización del testing permite hacer tantos ciclos de prueba con regresión como se deseen. La desventaja es el proceso de automatizar la ejecución de los casos de prueba, pero se puede ver ese esfuerzo de automatización como el esfuerzo propio del ciclo 0 manual, y luego esa automatización permite ejecutar los casos de prueba N veces. En empresas donde el core de negocio no es hacer software, conviene asumir ese esfuerzo y automatizar las pruebas.

Claramente, la mejor estrategia es aplicar regresión, porque no existe un punto medio al ser un método de caja negra, ya que no se puede saber con certeza lo que va a afectar al corregir un defecto y en dónde impactará esa corrección. Sin embargo, en la práctica se utiliza sin regresión, debido a los costos y tiempos que implica ese proceso.

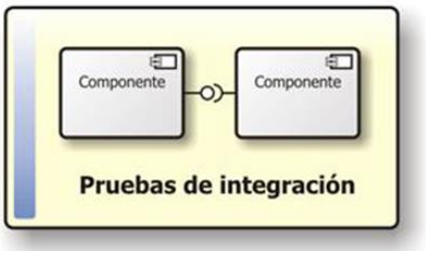
Niveles de prueba

Los niveles de prueba determinan el foco o la granularidad de la prueba que se está por ejecutar. En general, primero se comienzan probando componentes pequeños, y luego se integran en pruebas de mayor granularidad. Esto es al revés de cómo se desarrollan los componentes.

- Pruebas unitarias: las hace el desarrollador y están incluidas en el DoD. Son pruebas que se realizan sobre un componente, de manera independiente a los otros. Se hacen teniendo acceso al código fuente, y se pueden usar herramientas para realizarlas (automatización, depuración). Se suelen reparar los errores apenas se encuentran, sin registrarlos formalmente.



- Pruebas de integración: El propósito de la ejecución de estas pruebas es verificar el funcionamiento de dos o más componentes juntos que se relacionen entre ellos, es decir, clases en las cuales existe una interacción a modo de peticiones, a través de sus interfaces (boundaries). El resultado de estas pruebas es el build (el CI o continuous integration), el cual luego se envía al equipo de testing y es lo que se prueba en siguientes etapas.



  Se suele llevar a cabo una prueba de integración incremental, desde lo más general (que abarca más componentes) a lo más específico (top-down) o desde lo más específico a lo más general (bottom-up).

- Pruebas de sistema o de versión: En estas pruebas se realizan testeos sobre la versión de un incremento de producto o de un producto (dependiendo si se usa Agile o métodos tradicionales), y existen razones psicológicas que demuestran que deben realizarlas personas ajenas al desarrollo de los programas (obligatoriamente). Estas pruebas se llevan adelante siguiendo un proceso sistemático y metodológico, que permite encontrar defectos que puedan ser reproducibles y reportar de qué manera ocurre el defecto detectado, es decir, cual es el camino de pasos que hay que seguir para encontrar el error.
  Estas pruebas están basadas en casos de prueba. Se trata de emular de la mejor manera posible un entorno de trabajo idéntico al entrono real en el que se usara el SW. Se llevan a cabo pruebas del funcionamiento real y cotidiano del producto. Abarca requerimientos funcionales y no funcionales.



- Pruebas de aceptación de usuario: se realizan en el despliegue y debería realizarlas el usuario para verificar que se cumple con lo que el mismo requirió. Busca que el cliente/usuario se familiarice con el producto, generando comodidad y confianza sobre el mismo (su objetivo principal no es encontrar fallas). También busca reproducir un entorno de producción. Comprende tanto la prueba realizada por el usuario en ambiente de laboratorio (pruebas alfa), como la prueba en



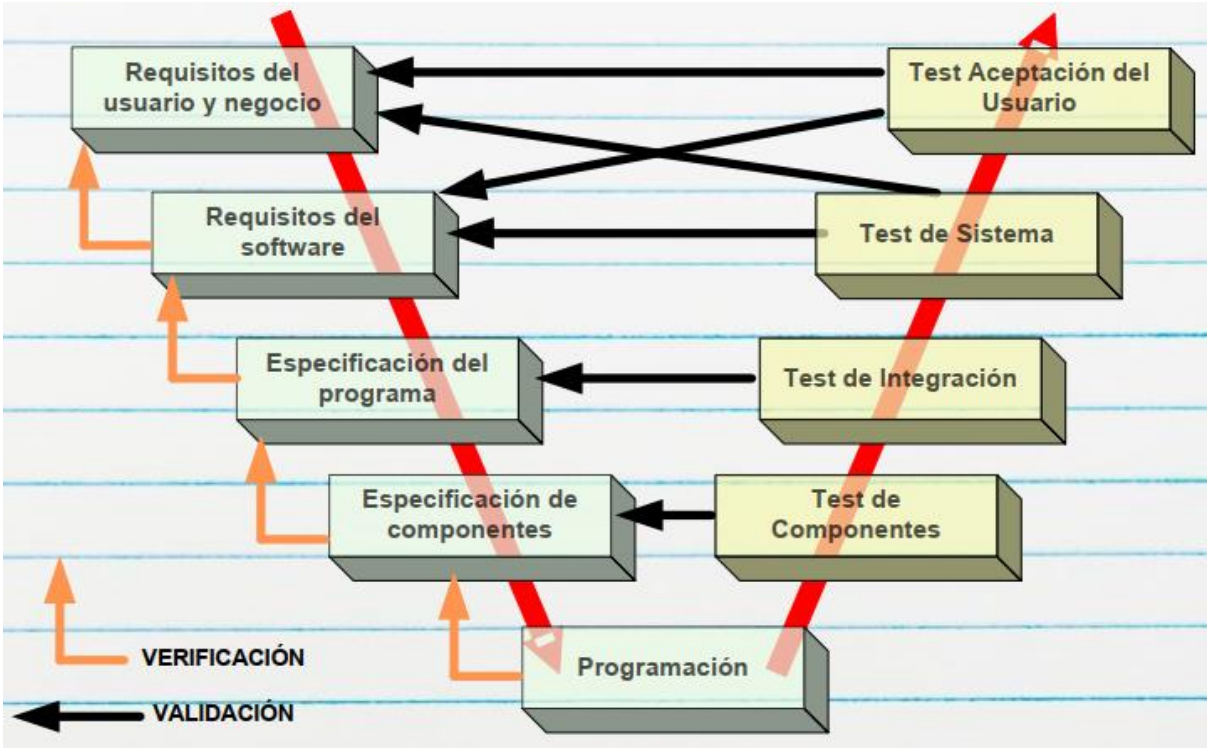
ambientes de trabajo reales (pruebas beta). En la teoría, los usuarios arman sus pruebas de usuario. En la práctica, las arma Testing.
En la teoría, además del usuario, deben estar presentes el gerente de testing, el líder del proyecto y empleados con roles funcionales.
En Agile, además del usuario, deben estar todos los miembros del equipo (Sprint Review).

Modelo en V

Este modelo se utiliza para poder determinar cuándo se está en condiciones de realizar determinadas actividades, en función de la etapa de desarrollo en la que se encuentre el software.

Acá se puede notar cómo las pruebas se realizan en orden de granularidad inverso al proceso de desarrollo del software. Es decir, se desarrolla desde componentes de granularidad gruesa, como son los requerimientos abstraídos de detalles de implementación, hacia componentes de menor granularidad, como son clases o módulos, dependiendo del paradigma. En cambio, para las pruebas, la granularidad se da en sentido inverso.

- Testing de Componentes → Testing unitario.
- Especificación del programa → diseño/arquitectura.



Ambientes de Testing

Son todos los recursos, tanto hardware como software, que se requieren para poder trabajar con el producto. Existen distintos ambientes en el desarrollo de software, los cuales poseen distintas necesidades, según la etapa de desarrollo en la que se encuentre el software.

Desarrollo

Implica hardware y software necesarios (librerías, extensiones, IDEs, compiladores, etc.) para poder desarrollar y desplegar el producto y utilizarlo. En este ambiente se realizan las pruebas unitarias (y también las de integración generalmente).

Pruebas

Es el ambiente que utilizan los testers para llevar a cabo las pruebas, y los desarrolladores no deben tener acceso. En este se realizan las pruebas del sistema. Normalmente acá se realizan las pruebas de sistema.

Preproducción

Debería tener las mismas características que el ambiente productivo para poder comprobar que el producto funcionará una vez desplegado en producción de manera correcta. En este ambiente se realizan las pruebas de aceptación.

El problema de este entorno es que muchas veces resulta difícil (o imposible en algunos casos), debido a que es muy costoso replicar principalmente las configuraciones de hardware. Por ejemplo, sería prácticamente imposible replicar la configuración de servidores del motor de búsqueda de Google, para realizar pruebas de aceptación.

Producción

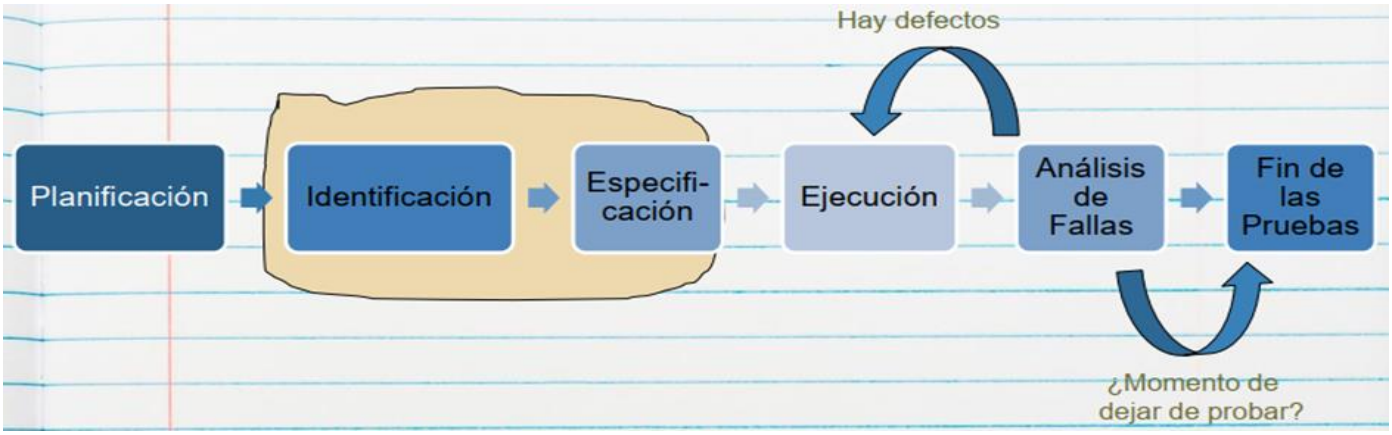
Este entorno, es la configuración de software y hardware que tienen los usuarios finales del software, y que utilizan para el desempeño de sus actividades laborales. Obviamente en este entorno no se realizan pruebas (probar en este ambiente tiene grandes consecuencias), ya que en teoría la versión del producto cumple con los criterios del Definition of Done.

Proceso de pruebas

El Testing es un proceso definido (que se lleva a cabo durante todo el ciclo de vida del producto), por lo que está compuesto de etapas detalladas.

Testing Ad Hoc: se realiza testing sin ningún proceso definido, perdiendo la trazabilidad ya que no se registra el paso a paso de lo que se probó, sino que se prueba libremente sin llevar ningún registro.

Generalmente el proceso de pruebas es estándar (puede tener algunas variaciones, pero sigue una estructura general).



- **Planificación**: Determina como se incluirá el Testing en el plan del proyecto, y como será el Test Plan (que recursos se usará, que riesgos se tendrá, cuál será el criterio de aceptación, que entornos se emularan, quien realizara cada Testing, cuando, etc.). El resultado de la planificación es el Plan de Pruebas, que debe contener:
    - Riesgos y objetivos del Testing.
    - Estrategia de Testing.
    - Recursos.

- Criterio de Aceptación.

- **Diseño (Identificación y especificación de casos de prueba)**: Revisando las bases de la planificación del Testing, se identifican los datos necesarios, diseñan y priorizan los CP que se llevaran a cabo (se define que entorno se usara, como se llevaran a cabo, por quien, cuando, etc.). Analiza si los requerimientos son testeables o no. Se define si se usa regresión o no.

- **Ejecución**: Cuando se ejecutan los CP, se registran y se comparan los resultados generando un reporte de defectos (con defectos encontrados, condiciones, entornos). Se trata de automatizar lo más que se pueda respecto de estas ejecuciones (ahorrando tiempo/costo). Incluye: Creación de los datos necesarios para la prueba, automatización de todo lo que sea necesario, implementar y verificar el ambiente, ejecutar los casos de prueba, registrar los resultados de la ejecución y comparar los resultados reales con los esperados.

- **Análisis de fallas (Evaluación y Reporte)**: Se hace un seguimiento de la corrección de los defectos encontrados hasta que se cierren todos los CP, es decir que se hayan solucionado todos. Se evalúa el criterio de aceptación, se reporta el resultado de las pruebas a los interesados, se verifica los entregables y que los defectos se hayan corregido y se evalúa cómo resultaron las actividades de testing y se analizan las lecciones aprendidas.

  Acá se confecciona el informe de reportes.

- **Fin de las pruebas:** En las empresas más maduras se deja de probar recién cuando no hay defectos bloqueantes, críticos, mayores y los defectos menores y cosméticos son muy pocos, los que se ponen en la nota de Release. Se confecciona el informe final (opcional).

Artefactos de Testing

Plan de pruebas

Este plan, es análogo al plan de proyecto que se elabora en el enfoque tradicional (no forma parte de ese plan). En este plan, se definen:

- Datos necesarios para las pruebas.
- Recursos destinados a las pruebas.
- Herramientas a utilizar.
- Si se aplicará regresión (o no).
- Etc.

Para la confección de este plan no es necesario el código, sino que sólo se necesitan los requerimientos, en el formato que sea que se encuentren (ERS o Casos de Uso en métodos tradicionales, User Stories en Agile, etc.)

Casos de prueba

- Es el artefacto más importante del testing, y consiste en una secuencia de pasos a seguir, las cuales describen un conjunto de acciones a realizar para lograr un resultado concreto y esperado. Para esto se deben explicitar las condiciones de inicio (las cuales fijan una situación particular), y los datos utilizados en ese escenario.
- Se confeccionan con el objetivo de descubrir la mayor cantidad de defectos y minimizar el esfuerzo y tiempo para elaborarlos y ejecutarlos.
- Mientras no cambien los requerimientos, estos casos pueden ser utilizados las veces que sea necesario.
- La característica más importante de los casos de prueba es ser REPRODUCIBLES. Si se encuentra un defecto y no es reproducible a través de un caso de prueba, no es un defecto.

Los casos de prueba se derivan de diferentes fuentes:

- Documentos del cliente;
- Información relevada;
- Requerimientos;
- Especificaciones de programación;
- Código.

Reporte de incidentes o defectos

Luego de la ejecución de los casos de prueba, se elabora un reporte, indicando cuáles fueron los casos de prueba que han pasado, y aquellos en los que se ha encontrado defectos, indicando cuáles fueron esos defectos encontrados y cómo reproducirlos para su detección y corrección.

Este artefacto permite a los desarrolladores corregir esos defectos, para enviarlos nuevamente a testing.

Informe final

suele contener métricas e información final del incremento del producto, se reporta cuántos ciclos y la cantidad de errores por ciclo, métodos, tipos de prueba utilizados, etc. Este es el único artefacto que es negociable en algunas organizaciones informales, se acuerda en la contratación del trabajo.

Tiene utilidades estadísticas.

El Testing y el Ciclo de Vida

Si desarrollamos con ciclo de vida en cascada (Ciclo de Vida Secuencial) las pruebas se hacen al final ya que es el momento en el que nos entregan el producto. En cambio, sí desarrollamos con ágil (Ciclo de Vida Iterativo/Incremental), hacemos testing por iteración. El problema es que se pueden incluir errores por la integración de los incrementos.

Estrategias de prueba

Se utilizan para pasar por la mayor cantidad de funcionalidades con la menor cantidad de casos pruebas confeccionados, debido a que el tiempo y el presupuesto para diseñarlos y ejecutarlos es limitado.

Hay dos grandes enfoques: Caja Negra y Caja Blanca. Cada uno tiene fortalezas y debilidades particulares: un método puede ser bueno para algunas cosas, y no para otras cosas. El mejor método es no usar un único método, sino usar una variedad de técnicas ayudará a un testing efectivo.

Caja Negra

En esta estrategia no se dispone de la estructura interna de la implementación, sino que se analizan las funcionalidades como una caja negra, en términos de entradas y salidas de esa “caja”.

El proceso consiste en ingresar determinados datos a esa funcionalidad vista como caja negra, y luego comparar los resultados obtenidos con los resultados esperados. Para ello, se realiza un testing exhaustivo de entrada, probando

cada posible entrada conducida como caso de prueba, no necesariamente las válidas. En transacciones no sólo se debe considerar todos los datos posibles, sino todas las secuencias posibles de transacciones previas.

Se clasifican en basados en especificaciones y basados en experiencia:

- Métodos basados en especificaciones: Son aquellos que se ejecutan utilizando la documentación de especificaciones realizadas del producto.
    - Partición de equivalencias: proceso sistemático que consiste en identificar clases de equivalencia, que definen subconjuntos de datos que producen un resultado equivalente.
    - Análisis de los valores limites: variante del anterior. Utilizar los limites o valores de borde de las clases de equivalencia para la definición de los casos de prueba.
- Métodos basados en experiencia: la experiencia y los conocimientos del tester son fundamentales para determinar las entradas del sistema y analizar los resultados.
    - Adivinanza de defectos: enfoque basado en la intuición y experiencia para identificar pruebas que probablemente expongan defectos del software, elaborando una lista de defectos posibles o situaciones propensas a error y realizando pruebas a partir de esa lista.
    - Testing exploratorio: el tester mientras va probando el software, va aprendiendo a manejar el sistema y junto con su experiencia y creatividad, genera nuevas pruebas a ejecutar.

Partición de equivalencias

Analiza las condiciones externas (entradas y salidas) involucradas en una funcionalidad determinada. A esas condiciones externas las divide en clases de equivalencia, las cuales son subconjuntos de valores posibles, que arrojan resultados equivalentes en la funcionalidad que se quiere probar.

Ejemplos de entradas: campos de texto, fechas, coordenadas de un mapa, archivos, señales de sensores, etc.

Ejemplos de salidas: mensaje en pantalla, advertencias, listados, tablas, emisión de señal, etc.

Procedimiento

1. Identificar condiciones externas de entrada y salida.
    a. Si hay que ingresar dos datos que son iguales, pero con distintos valores, separarlo en condiciones externas diferentes. Por ejemplo: 2 calles, una condición externa es calle1 y la otra calle2.
2. Identificar subconjuntos de valores posibles (válidos y no válidos) de las condiciones externas identificadas, que produzcan resultados equivalentes en la ejecución de la funcionalidad.
    a. Los subconjuntos no válidos tienen asociados mensajes de error en la condición externa de salida “Mensajes de Error” o “Mensajes de Advertencia” (no es necesario poner todos los mensajes, con algunos genéricos es suficiente).
    b. Para no olvidarse de algún subconjunto → la unión de todos los subconjuntos debe ser igual al universo de la condición externa (teoría de conjuntos).
3. Armar los casos de prueba tomando de cada condición externa, un sólo valor particular (especificar) de cada subconjunto de valores posibles.

Consideraciones

- Prioridad
    - Alta → caminos felices o errores críticos en el dominio de negocio.
    - Baja → casos de prueba relacionados a validaciones (valores no ingresados, con mal formato, etc.).
- Nombre caso de prueba: describir el escenario de la funcionalidad que se está probando, ¡de forma clara! Por ejemplo: “Ingresar al sitio web con edad menor a 18 años”
- Precondiciones
    - Conjunto de características que tienen que cumplirse en el contexto de la funcionalidad, para que pueda ser ejecutada.
    - Deben ser valores concretos, específicos. Por ej: “El usuario Juan está logueado con permiso de administrador.” - “La fecha actual es 25/10/2022” - “Las coordenadas del GPS son: 41°24’12.2” N” - “La tarjeta tiene el número: XXXX XXXX XXXX XXXX”
    - Los datos que se seleccionan de entre un grupo determinado (por ejemplo forma de pago), son precondiciones que deben estar previamente cargadas en el sistema.
- Pasos
    - Siempre arrancan seleccionando la opción que desencadena la funcionalidad: “El *nombreUsuario* ingresa a la opción …”
    - Después: “El *nombreUsuario* ingresa/selecciona …
    - No tiene que haber ambigüedad, pensar que otra persona los tiene que leer y ejecutar.
    - Normalmente finalizan con un botón de confirmación o algo así.
- Resultado esperado
    - Puede mostrar varias cosas el sistema.
    - Deben ser resultados concretos, específicos.
    - Suelen ser mensajes de advertencia/error, listados, posición en el mapa, etc. pero siempre indicando con un dato particular. Por ejemplo: El sistema muestra el mensaje de advertencia “Debe seleccionar una fecha”.
    - Se debe relacionar la salida con el paso en la cual se produce.

Caja blanca

En estos se dispone del código (puede ser también pseudocódigo o un diagrama de flujo). Nos permiten diseñar casos de prueba, maximizando la cantidad de defectos con la menor cantidad de casos de prueba posibles.

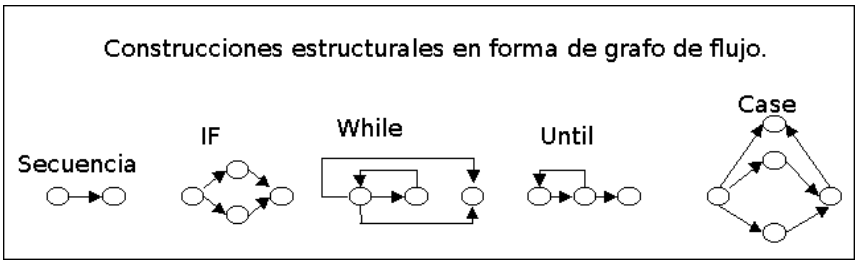
Tiene dos fallas: La primera es que el número de caminos lógicos únicos puede ser sumamente grande, tomando muchísimo tiempo de probar absolutamente todas de ellas. La segunda falla es que las pruebas de camino exhaustivas no aseguran que el programa sea el correcto para las especificaciones, tampoco detecta caminos faltantes ni determina errores de datos sensibles.

Hay distintas coberturas con nivel diferente (la forma de recorrer los distintos caminos que provee el código para desarrollar una funcionalidad):

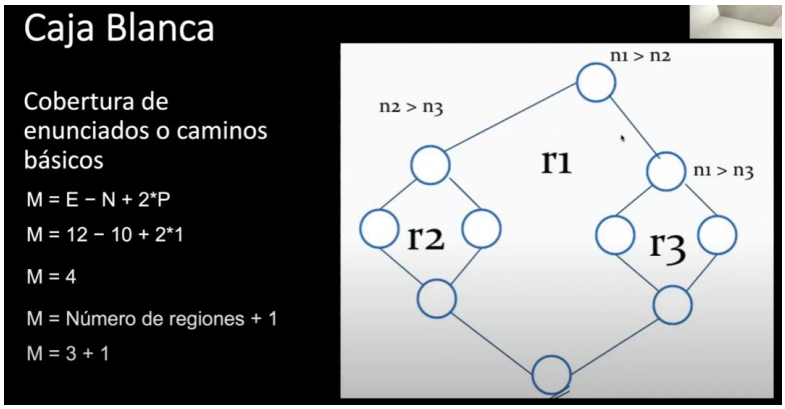
Cobertura de enunciados o caminos básicos

- Objetivo → encontrar todos los caminos independientes de una funcionalidad que deben recorrerse (testear) al menos una vez, para probar cada uno de ellos con casos de prueba.
- Permite obtener una métrica denominada *complejidad ciclomática* (M), que representa la cantidad de caminos independientes que posee una funcionalidad, y nos da un límite inferior para el número de casos de prueba que hay que construir, para ejecutar todas las instrucciones de la funcionalidad al menos una vez.
- El procedimiento es:

1. En primer lugar, se requiere representar la funcionalidad a través de un grafo de flujo (los algoritmos que implementen métodos recursivos quedan fuera de esta prueba de cobertura)



2. Luego se calcula la complejidad ciclomática (M). Para esto hay dos formas:
   a. M = E - N + 2*P, siendo:
      - M = complejidad ciclomática.
      - E = número de aristas del grafo.
      - N = número de nodos del grafo.
      - P = número de componentes conexos o nodos de salida.
   b. También se puede obtener la complejidad ciclomática como M = número de regiones cerradas + 1.



En el ejemplo, la cantidad mínima de casos de prueba para garantizar la cobertura de enunciados es de 4. Esto no quita que se puedan construir más de 4, pero esa es la menor cantidad de casos de prueba, que me permite maximizar la detección de defectos.

3. Se define el conjunto mínimo de caminos independientes (asignando valores que provocan ir tomando cada uno de los caminos de la funcionalidad). No son casos de prueba. Siguiendo el ejemplo:

TC 1	TC 2	TC 3	TC 4
N1 = 8	N1 = 8	N1 = 4	N1 = 4
N2 = 4	N2 = 4	N2 = 8	N2 = 8
N3 = 4	N3 = 8	N3 = 4	N3 = 8

4. Luego se diseñan y ejecutan los casos de prueba que permitan la ejecución de cada uno de los caminos identificados, donde los datos de la tabla de arriba se mapean a las precondiciones o a valores que ingresa el usuario en los casos de prueba (dependiendo de cómo es la funcionalidad).

5. Se ejecutan los casos de prueba y se comprueba que los resultados sean los esperados.

- Dependiendo del valor de la complejidad ciclomática (M), el programa puede entrar en una de las siguientes clasificaciones (heurística):

Complejidad Ciclomática	Evaluación del Riesgo
1-10	Programa Simple, sin mucho riesgo
11-20	Más complejo, riesgo moderado
21-50	Complejo, Programa de alto riesgo
50	Programa no testeable, Muy alto riesgo

Cobertura de sentencias

- Sentencia: cualquier instrucción como asignación de variable, invocación de métodos, etc. Las estructuras de control NO son sentencias, son decisiones.
- Objetivo → buscar la cantidad mínima de casos de pruebas que me permitan pasar, ejecutar o evaluar todas las sentencias.
- Ejemplo: Partiendo de la siguiente porción de código, podemos determinar que existen 2 sentencias (de asignación de variables). Entonces, para ejecutar o evaluar las sentencias, es necesario solo un caso de prueba, asignando valores a "A", "C", "B" y "D" es posible ejecutar ambas sentencias ya que las condiciones de IF son independientes entre sí.

```
IF (A>0 && C==1)
        X = X + 1
IF (B==3 || D<0)
        Y=0
END
```

TC1
A=5
C=1
B=3
D=-3

Cobertura de decisión

- Decisión: estructura de control concreta, y dentro de cada decisión existen combinaciones que están unidas por operaciones booleanas de condiciones.
- Objetivo → buscar la cantidad mínima de casos de prueba que me permitan evaluar todas las ramas de las decisiones. Esto apunta a que el código tome cada rama de forma correcta, es decir, evaluar que la decisión derive en la rama del true y en la rama del false cuando corresponda en cada caso.

- En una decisión IF, si o si necesitamos dos casos de prueba: rama true y rama false.
- Las decisiones son las que se encuentran dentro de los paréntesis de los IF, y cada uno de los términos son condiciones, es decir A>0 y C==1, y el conjunto de las condiciones unidas por el booleano, representan una combinación (A>0 && C==1).
- En esta cobertura, no interesa qué condición (o combinación de condiciones) hay dentro de la decisión, siempre y cuando se garantice que la decisión es evaluada en su rama verdadera y en su rama falsa.
- En el ejemplo, con tan solo dos casos de prueba es suficiente, ya que las decisiones son independientes entre sí (no están anidados), por lo que independientemente de la rama que se evalúe en la primera decisión, voy a poder valuar cualquiera en la segunda. Entonces en el TC1 se evalúa la rama de "true" en ambas decisiones, y en TC2 se evalúa la rama de "false" en la dos.

```
IF (A>0 && C==1)
        X = X + 1
IF (B==3 || D<0)
        Y=0
END
```

TC1	TC2
A=5 C=1 B=3 D=-3	A=5 C=5 B=5 D=5

Cobertura de condición

- Condiciones: son cada una de las evaluaciones lógicas dentro de una decisión, que están unidas por operadores lógicos.
- Objetivo → buscar la cantidad mínima de casos de pruebas que me permitan evaluar cada una de las condiciones, en su valor verdadero y en su valor falso, independientemente de que rama se tome en la decisión en la que se encuentre la condición (recordar que la decisión está formada por 1 o más condiciones combinadas de forma booleana).

TC1	TC2
A=0 C=1 B=3 D=-3	A=5 C=5 B=5 D=5

- Siguiendo el ejemplo anterior, son necesarios 2 casos de prueba, ya que las condiciones son independientes y se pueden evaluar todas las condiciones en true en una prueba, y todas las condiciones en false en otra prueba, debido a que no importa la rama que tome cada decisión.

Cobertura de decisión condición

- Objetivo → evaluar las ramas verdaderas y falsas de las decisiones, y a su vez evaluar valores verdaderos y falsos de las condiciones.
- Siguiendo el ejemplo anterior, se necesitan 2 test cases, los cuales permiten evaluar las condiciones y decisiones al mismo tiempo en un mismo caso de prueba, con los valores de la imagen.

```
IF (A>0 && C==1)
        X = X + 1
IF (B==3 || D<0)
        Y=0
END
```

TC1	TC2
A=5 C=1 B=3 D=-3	A=0 C=5 B=5 D=5

Cobertura múltiple

- Objetivo → evaluar el combinatorio de todas las condiciones, en todos sus valores de verdad posibles.
- Por ejemplo: Si tuviésemos un caso como el representado en el grafo de flujo, los casos de prueba van a ser 7, ya que solo el valor de A y B verdaderos al mismo tiempo, permiten

1 DECISIÓN	A	F	F	V	V	V	V	V
	B	F	V	F	V	V	V	V
2 DECISIÓN	C	-	-	-	V	V	F	F
	D	-	-	-	V	F	V	F

ejecutar escenarios en donde se incluye la decisión de C y D. Este es un escenario con decisiones dependientes entre sí, si fuesen independientes, con 4 pruebas alcanza.

Tipos de prueba

Smoke Test

Es un tipo de prueba que se hace para validar que no haya fallas groseras, de gran magnitud, en el producto de software. Se realiza antes de comenzar el ciclo 0. Nos ahorra empezar a testear formalmente si encuentra un error, porque todavía hay algo que corregir.

Consiste en una corrida rápida para ver si el producto está en condiciones de pasar al ciclo de prueba.

Testing Funcional

Controla que el software se comporte de la misma manera que lo especificado en la documentación, cumpliendo con las funcionalidades y características definidas. Se basa en los requerimientos funcionales y el proceso de negocio. Hay testing funcional basado en dos aspectos:

- Basado en requerimientos: cuando se prueban requerimientos específicos, apunta a probar una funcionalidad sola (utilizan a los requisitos definidos en una ERS o los acuerdos que contienen las pruebas de usuario y los criterios de aceptación de una US para realizar las pruebas.)
- Basado en proceso de negocio: cuando se prueba un proceso de negocio completo, es decir, se prueba todo el proceso. Por ejemplo, en una venta se prueba la búsqueda del artículo, la selección y facturación del mismo.

Testing No Funcional

Se basa en cómo trabaja el sistema haciendo foco en los requerimientos no funcionales (foco en el "como", no en el "que"). Son las pruebas más complejas, por su gran dependencia al entorno del sistema, por lo que debe ser lo más parecido posible al del cliente. Sin embargo, se tienen características que no pueden probarse, como el ancho de banda del internet, la seguridad o performance en una determinada situación. Y se pueden solucionar con las pruebas de aceptación. Incluye varios tipos de prueba como:

- Performance: Se ve el tiempo de respuesta (escenario esperado respecto a los tiempos de respuesta) y la concurrencia. Deben pasar esta prueba sí o sí.
- Carga: no solo mira performance, mira el comportamiento de los dispositivos de hardware (procesadores, discos, etc.) y de las comunicaciones.
- Stress: queremos forzar al sistema para que falle, se lo somete a condiciones más allá de las normales. Se ve el tiempo que hay que esperar para probar nuevamente el sistema y la robustez del sistema (tiempo de recuperación).
- Mantenimiento: para ver si el producto está en condiciones de evolucionar, se controla que haya documentación, manual de configuración, etc. Se observa la facilidad que existe para corregir un defecto.
- Usabilidad: que sea cómodo para el usuario.
- Portabilidad: se prueba en los distintos entornos acordados con el cliente.

- Fiabilidad: probamos que podemos depender del sistema. Resultados que se obtienen, seguridad física del software.
- De interfaz de usuario: suelen ser más complejas las GUI's que las interfaces de comandos.
- De configuración.

Test Driven Development – TDD

Este es un tipo de proceso en el que nos enfocamos en construir primero la prueba unitaria cuando tengo los requerimientos y luego codear el componente. Si pienso en las pruebas después tengo menos errores, el fundamento filosófico de esto es que si no tengo claro que tengo que hacer no puedo crear pruebas para eso.

Es una técnica de desarrollo del software que involucra dos prácticas:

1. **Test first development:** en esta técnica primero se escriben las pruebas unitarias referentes a la característica de producto a implementar. Definidas las pruebas unitarias, se piensa en la codificación necesaria para que las pruebas se ejecuten con éxito. Dado que las pruebas unitarias prueban unidades concretas de código, esta técnica obliga al desarrollador a modularizar su codificación haciendo que los métodos o clases a probar tengan una única responsabilidad. Además de pruebas unitarias, pueden incluirse en primera instancia pruebas de integración.
2. **Refactoring:** esta técnica consiste en reestructurar el código existente sin modificar el comportamiento que el mismo provee. Implica la mejora de aspectos no funcionales del software, cuyo objetivo es mejorar la claridad del código y reducir su complejidad, con el fin de hacerlo más mantenible.

Mejora continua de procesos con Kanban

Kanban

No es ni:

- Un proceso de desarrollo de software.
- Una metodología de administración de proyectos.

Definición

Kanban es un enfoque para la gestión de formas de trabajo (procesos) para obtener mejora continua.

Esto lo logra a través del principio de "empieza por donde estés", es decir, no plantea introducir cambios revolucionarios, sino que introduce mejoras graduales a un proceso de desarrollo de software o a una metodología de administración de proyectos **ya existente** en una organización. Esto permite reducir la resistencia al cambio por parte de las personas, fomentando la evolución gradual de los procesos existentes.

Este enfoque surge con estudios de Toyota, para mejorar las técnicas de almacenamiento y tiempo de stockeo en supermercados.

Para su aplicación en el desarrollo de software, al igual que Lean fue necesaria una adaptación.

Kanban aprovecha los principios de Lean:

- Definiendo el **valor** desde la perspectiva del cliente.

- Limitando el trabajo en proceso WiP.
- Identificando y eliminando desperdicios.
- Identificando y eliminando las barreras en el flujo, es decir, todo lo que atrasaría el proceso: Relacionado con el principio de lograr entregar lo antes posible.
- Cultura de mejora continua.

Prácticas de Kanban

Visualización del trabajo

El método Kanban utiliza un mecanismo de señalización para hacer visible el trabajo que es requerido por el cliente. Para ello, divide el trabajo en piezas de trabajo (que pueden ser U.S., Features, bugs, temas, épicas, cambios, etc.) y las escribe en tarjetas señalizadoras (kan-bans) que serán ubicadas en tableros kanban.

Estas tarjetas permiten indicar cuando el trabajo se puede "pullear" para realizar un trabajo determinado, además de señalar en qué parte del proceso se encuentra.

El tablero kanban representa un sistema de flujo en el que las piezas de trabajo fluyen a través de las diversas etapas de un proceso, de izquierda a derecha. Cada etapa es representada por una columna del tablero, sobre las cuales se aplica la teoría de colas.

Se logra transparencia al hacer visible para todo el equipo el trabajo mediante un tablero kanban siempre disponible y a la vista.

Existen dos tipos de columnas:

- Producción: piezas sobre las que se está trabajando.
- Acumulación: piezas que están listas para pasar a la siguiente etapa (sistema de arrastre).

Limitar el WiP (Work in Progress)

Se deben asignar límites concretos a cuántas piezas pueden estar en progreso en cada etapa del flujo de trabajo (en cada columna), para evitar atascamientos o cuellos de botella.

Las políticas para limitar el WiP crean un sistema de arrastre (pull): el trabajo es "arrastrado" al sistema cuando otro de los trabajos es completado y queda capacidad disponible, en lugar de "empujar" estos trabajos al paso siguiente cuando hay nuevo trabajo demandado.

Tener demasiado trabajo no finalizado es un desperdicio de tiempo, de dinero y alarga los tiempos de entrega. Observar, limitar y optimizar la cantidad de trabajo en progreso es esencial para tener éxito con Kanban, consiguiendo mejorar la calidad y el tiempo de entrega de servicio y aumentar la tasa de entrega.

Gestionar el flujo de trabajo

El trabajo fluye sobre un tablero permanente, no existe el concepto de iteración, proceso o proyecto. El tablero puede ser compartido con otros proyectos y otros equipos, ya que se trabaja cada pieza de trabajo de forma individual. Lograr que el flujo sea continuo e ininterrumpido.

Al flujo que se observa en ese tablero, se lo debe analizar, identificando diferentes características del proceso de trabajo: cuellos de botella, recursos ociosos, tiempos de entrega, tiempos de espera, etc.

Hacer explícitas las políticas

Las políticas de proceso deben ser escasas, simples, estar bien definidas, visibles, deben aplicarse siempre, y tienen que ser fácilmente modificables. Es una buena práctica poder cambiar fácilmente las políticas, ya que si producen efectos contraproducentes para nuestro proceso o también se considera que no pueden aplicarse las mismas, deben poder cambiarse.

También es importante visualizar las políticas; por ejemplo, colocando resúmenes entre las columnas donde se describe lo que debe estar hecho antes de que una tarjeta se mueva de una columna a la siguiente. También se suele colocar el WIP de cada columna.

Las políticas de calidad o DoD deben estar definidas, publicadas y promovidas para lograr no sólo que se cumplan, si no buscar mejorar continuamente.

Mejorar y evolucionar

Kanban propone una cultura de mejora continua, donde no introduce cambios significativos en los procesos de desarrollo, sino que identifica ese proceso, lo visualiza (hacer explícito para todos los miembros) y propone mejoras sobre él, las cuales se van aplicando de forma gradual a lo largo del tiempo.

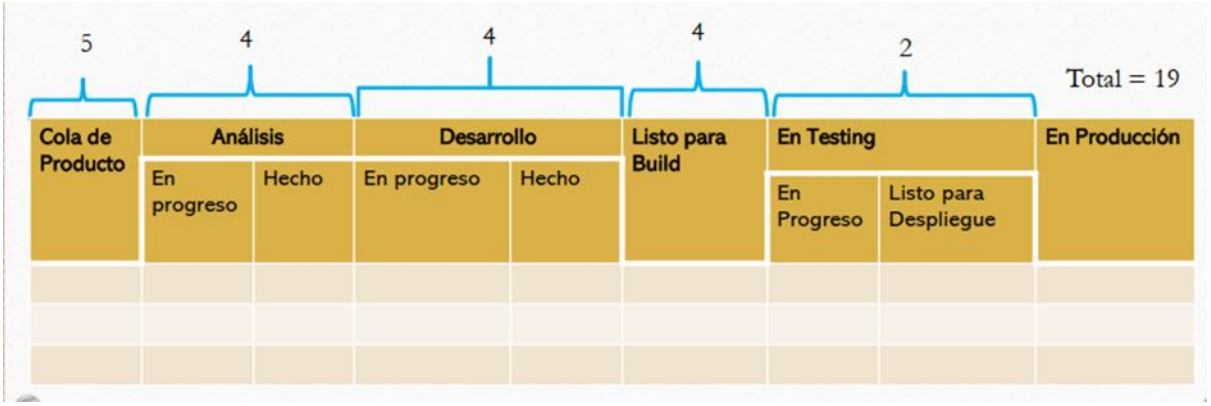
Circuito de retroalimentación

Son una parte esencial para cualquier proceso controlado que nos ayuda a realizar cambios evolutivos. Se debe definir con qué frecuencia se realizan las reuniones, donde depende en qué contexto se presentan ya que es importante para el resultado.

Si se realizan revisiones demasiadas frecuentes pueden obligar a cambiar cosas que no se vieron con los cambios anteriores, pero si no son demasiadas frecuentes, existe un bajo rendimiento durante mucho tiempo.

¿Cómo aplicar Kanban?

1. Empezar con lo que se tiene: entender el proceso de desarrollo actual que se está utilizando.
2. Identificar las unidades de trabajo a utilizar (US, CU, defectos).
3. Identificar clases de servicio: diferentes trabajaos tienen distintas políticas para tratarlos (DoD). Por ejemplo: requerimientos, defectos, desarrollo y solicitudes.
4. Visualizar el flujo de trabajo diseñando un tablero representando el flujo de trabajo a través de columnas.
5. Definir políticas para cada clase de servicio identificada. Esto implica acordar los WiP para cada columna, asignar color a las tarjetas kan-ban, capacidad de trabajo destinada, fechas de entrega, etc.
6. Identificar cuellos de botella y resolverlos (asignando más recursos o ajustando el WiP donde corresponda).



Métricas de Kanban

Las métricas más representativas de Kanban son:

- Cycle Time.
- Lead Time.
- Touch Time.
- Eficiencia del Ciclo de Proceso.

Estas métricas están desarrolladas en el tema de métricas de la unidad 2 de este apunte.

Valores de Kanban

Los valores de Kanban se podrían resumir en una sola palabra, "respeto". Sin embargo, es importante desgranar esto en una serie de nueve valores.

- **Transparencia:** La creencia de que compartir información abiertamente mejora el flujo de valor de negocio. Utilizar un lenguaje claro y directo es parte del valor.
- **Equilibrio:** El entendimiento de que los diferentes aspectos, puntos de vista y capacidades deben ser equilibrados para conseguir efectividad. Algunos aspectos (como demanda y capacidad) causarán colapso si no se encuentran equilibrados por un periodo prolongado.
- **Colaboración:** Trabajar juntos. El Método Kanban fue formulado para mejorar la manera en que las personas trabajan juntas, por ello, la colaboración está en su corazón.
- **Foco en el cliente:** Conociendo el objetivo para el sistema. Cada sistema kanban fluye a un punto de valor realizable — cuando los clientes reciben un elemento solicitado o servicio. Los clientes en este contexto son externos al servicio, pero pueden ser internos o externos a la organización como un todo. Los clientes y el valor que estos reciben es el foco natural en Kanban.
- **Flujo:** La realización de ese trabajo es el flujo de valor, tanto si es continuo como puntual. Ver el flujo es un punto de partida esencial en el uso de Kanban.
- **Liderazgo:** La habilidad de inspirar a otros a la acción a través del ejemplo, de las palabras y la reflexión. Muchas organizaciones tienen diferentes grados de jerarquía estructural, pero en Kanban, el liderazgo es necesario a todos los niveles para alcanzar la entrega de valor y la mejora.
- **Entendimiento:** Principalmente conocimiento de sí mismo (tanto individual como de la organización) para ir hacia adelante. Kanban es un método de mejora, por lo que conocer el punto de inicio es la base de todo.
- **Acuerdo:** El compromiso de avanzar juntos hacia los objetivos, respetando — y donde sea posible, acomodando — las diferencias de opinión o aproximaciones. Esto no es gestión por consenso sino un compromiso dinámico para mejorar.
- **Respeto:** Valorando, entendiendo y mostrando consideración por las personas. De manera apropiada al pie de esta lista se encuentra la base sobre la cual reposan el resto de los valores.

Principios directores

1. **Sostenibilidad:** relativo a encontrar un foco sostenible y un foco en la mejora.
2. **Orientación al servicio:** enfocado a conseguir rendimiento y satisfacción del cliente.
3. **Supervivencia:** relativa al mantenimiento de la competitividad y adaptabilidad.

Principios fundacionales

Hay seis principios fundacionales de Kanban, los cuales pueden ser divididos en dos grupos: los principios de gestión de cambio y los principios de entrega o despliegue de servicios (Fig 3)

Principios de gestión de cambio

Cada organización es una red de individuos, conectados psicológicamente y sociológicamente para resistir al cambio. Kanban reconoce estos aspectos humanos con *tres principios de gestión de cambios.*

1. Empezar con lo que estés haciendo ahora:
   - Entender los procesos actuales tal y como están siendo realizados en la actualidad, y
   - Respetar los roles actuales, las responsabilidades de cada persona y los puestos de trabajo.



Figure 3 Principios de Kanban

2. Acordar en buscar la mejora a través del cambio evolutivo.
3. Fomentar el liderazgo en cada nivel de la organización — desde las contribuciones individuales de cada persona hasta las posiciones más senior de la organización.

Hay dos razones clave para que "empezar desde donde estés" sea una buena idea. La primera es que se minimiza la resistencia al cambio ya que respetamos las prácticas actuales y a las personas que las llevan a cabo. Esto es crucial para involucrar a todos en los retos y desafíos del futuro. La segunda es que los procesos actuales, junto con sus deficiencias obvias, contienen la sabiduría y la potencialidad de mejora que incluso las personas que trabajan con ellos no aprecian en su totalidad. Dado que el cambio es esencial, no hay que imponer soluciones desde diferentes contextos, sino buscar la mejora evolutiva en todos los niveles de la organización. A partir de las prácticas actuales hay que establecer los objetivos de mejora desde la situación actual a la situación deseada y evaluar las mejoras y como éstas nos van acercando hacia el objetivo.

Principios de despliegue de servicios

Las organizaciones, independientemente de su tamaño, son ecosistemas de servicios interdependientes. Kanban reconoce esto con tres *principios de despliegue de servicios*, aplicables no solo a un servicio, sino a todos ellos

1. Entender las necesidades y expectativas de tus clientes y focalizarse en ellas.
2. Gestionar el trabajo: dejar que la gente se auto-organice alrededor de las tareas.
3. Evolucionar las políticas para mejorar los resultados hacia el cliente y del negocio

Estos principios están muy alineados con el principio director de orientación a servicios y entrega de valor al cliente. Cuando el trabajo en sí mismo y el flujo de valor al cliente no es claramente visible, las organizaciones se enfocan en vez de eso en lo que *es* visible: la gente trabajando en el servicio. ¿Están siempre ocupados? ¿Tienen las suficientes habilidades? ¿Podrían trabajar más duro? El cliente y el valor al cliente reciben menos atención. Estos principios hacen hincapié en que el foco debe estar en los consumidores del servicio y en el valor que reciben del mismo.

Anexo

Diferencias entre Scrum y Kanban

Diferencia	**SCRUM**	**KANBAN**
Tiempo	Las iteraciones son de tiempo fijo.	El tiempo fijo es opcional. La cadencia puede variar. Pueden estar marcadas por la previsión de los eventos en lugar de tener un tiempo prefijado.
Compromiso	El equipo asume un compromiso de trabajo por iteración.	El compromiso es opcional.
Métrica por defecto	Para la planificación y mejora es la **velocidad**.	El por defecto es **Lead Time** (tiempo de entrega promedio)
Equipos	Multifuncionales	Multifuncionales o especializados.
Diagramas de seguimiento	Deben emplearse gráficos Burndown	No se prescriben diagramas de seguimiento.
Limitación WIP	Es indirecta y está marcada por el sprint.	Es directa para cada etapa de trabajo o el estado del trabajo.
Agregar trabajo.	No se puede agregar alcance una vez comenzado el sprint.	Siempre que haya capacidad disponible, se puede agregar trabajo.
Estimaciones.	Se deben realizar estimaciones.	La estimación es opcional.
Compartido entre equipos	Cada equipo tiene su sprint backlog.	Todos los equipos comparten la misma pizarra o tabla kanban.
Permanencia tablero	En cada sprint se limpia.	El tablero es persistente.
Roles	Se prescriben tres roles. PO, SM y Equipo.	No se prescriben roles.
Priorización	El product backlog debe estar priorizado.	La priorización es opcional.
Funcionalidades	Espera que las funcionalidades se dividan tal que se completen en un sprint.	No se prescribe el tamaño de la funcionalidad.

Kanban no tiene el concepto de iteración ni proyecto, porque el trabajo fluye a lo largo de las distintas etapas del proceso de manera continua. No se corta o para en ningún momento este proceso.

El tablero en Kanban es permanente, mientras que, en Scrum, el mismo se limpia al finalizar cada sprint, ya que contiene las columnas de “to-do”, “doing” y “done” propio del sprint que se está ejecutando.

Ser ágil y hacer ágil

Hacer ágil

Cuando implementamos métodos y prácticas ágiles estamos “haciendo” algo para lograr la agilidad. Esto nos ayuda a tener los eventos artefactos y procesos adecuados para entregar valor de forma continua con un buen nivel de calidad, gracias a esto podemos tener algunos beneficios como: adaptación a los cambios, mejorar la visibilidad, incrementar la productividad, mejorar la calidad y reducir los riesgos. Un buen programa de entrenamiento nos puede ayudar a alcanzar este estado.

Doing Agile (hacer agil) es practicar rituales y ceremonias ágiles. Las reuniones de pie, las demostraciones y revisiones quincenales, la planificación del sprint, los tableros Kanban y los puntos de la historia. Todos estos elementos son indicaciones claras de “Doing Agile”.

Pero simplemente realizar algunas prácticas que se reconocen como Agile no se traducen en un entorno de trabajo ágil, en equipos de alto rendimiento, en mejor calidad o en un tiempo de comercialización más rápido.

Ser ágil

Cuando además de implementar prácticas adoptamos una forma diferente de pensar estamos “siendo” alguien distinto, alguien que vive la agilidad y cuyo comportamiento está alineado a los valores y principios ágiles, esto nos da beneficios adicionales como: deleite del cliente, placer por el trabajo, compromiso, innovación, creatividad y aprendizaje continuo. Cuando hablamos de ser ágil a nivel organizacional nos referimos a empresas que adoptan una cultura que permite obtener estos beneficios mediante un cambio de mentalidad en todos los niveles de liderazgo, no solo de los equipos de trabajo.

Ser ágil (Being Agile) realmente funciona de manera diferente. Incluye una estructura de toma de decisiones drásticamente nueva, participación empresarial totalmente integrada, equipos persistentes autoorganizados y multidisciplinares.

Además, el enfoque es en el producto (no proyecto), usando DevOps y una cultura totalmente pensada en el aprendizaje continuo.

En definitiva, significa convertir esos rituales en hábitos diarios que impulsan la acción de mejora continua. Esto es más que hacer agilidad, ser ágil es lo que es.

A medida que la transformación se expande y madura, deja de hacer agilidad, ser ágil es en lo que se transforma. Hay varias prácticas que pueden emplearse para ayudar a las organizaciones a mejorar:

- Confía en tus equipos. Direccionalmente, el liderazgo sénior establece los objetivos estratégicos, pero los equipos autoorganizados y auto empoderados (liderados por Products Owners autónomos y autorizados) determinarán los mejores enfoques, arquitecturas técnicas y todo el “cómo” a desarrollar, probar y lanzar. Esto significa que no hay más comités directivos. Los problemas se resuelven en el nivel más bajo posible (nivel de equipo).

- Practica la transparencia. Esto se aplica a todo en lo que trabaja el equipo. La transparencia debe ser un mantra para todos los equipos ágiles, así como para su liderazgo. Todos los entregables son trabajos en progreso, por lo tanto, deben estar abiertos para inspección en todo momento. La clave es que los Product Owners establezcan expectativas con el liderazgo superior sobre la naturaleza de esos entregables y cómo se entregan. La mejor medida de progreso es el incremento de trabajo disponible para los usuarios.

- Gestionar los indicadores de progreso de manera diferente. Dejamos de administrar por informes de estado y eliminamos los mandatos basados en fechas. Podemos seleccionar objetivos de lanzamiento de productos e hitos, pero no serán de alcance fijo; serán de naturaleza más temática sin características específicas completamente definidas. Los radiadores de información son increíblemente importantes. La transparencia es un principio básico ágil y, como tal, debe integrarse en cada transformación.

Para aquellos que son habitualmente ágiles, ver errores rápidamente es una prueba de que estás aprendiendo, mejorando constantemente y acercándote a tus clientes.

Aquellos que piensan en una mentalidad ágil ponen la confianza y la transparencia por encima de todo en su cultura. Entonces, mírate en el espejo y pregúntate si eres realmente ágil, o si simplemente estás haciendo postureo. Porqué puede ser que no te valga solo hacer agilidad, ser ágil es lo que tienes que ser.

Conclusión

En mi experiencia las mejores transiciones hacia la agilidad se logran cuando además de la adopción de prácticas ágiles también se adopta el mindset Ágil, esto definitivamente es mucho más complejo, ya que un entrenamiento no será suficiente, se necesita de un acompañamiento adecuado, de preferencia de un coach experimentado que pueda ayudar a lograr una transición cultural.

Hacer ágil es definitivamente distinto a ser ágil, y aunque ninguno de los dos es fácil no debemos perder de vista los objetivos que queremos alcanzar, si queremos el mayor de los beneficios debemos desafiar nuestras actuales formas de pensar y promover ese cambio alrededor de nosotros. Mi recomendación es solicitar apoyo de un experto, ya sea interno o externo, no solo en metodologías Ágiles sino también en el proceso de transformación hacia nuevos paradigmas.

Cuadro comparativo Gestión Tradicional vs. Gestión ágil

Aspecto a comparar	**Gestión Tradicional**	**Gestión Ágil**
Respuesta a cambios	Poco flexible	Altamente flexible
Incertidumbre en el desarrollo del proyecto	Baja → se elimina en etapa de requerimientos	Alta → se elimina cuando sea necesario
Tiempos de entrega de software útil	Tardío	Temprano
Retroalimentación del cliente para incluir nuevas necesidades o correcciones	Baja	Alta
Participaciones en estimaciones	Las hace el líder de proyecto	Las hace el equipo de desarrolladores
Conformación de equipo	Jerárquico, con roles definidos.	Autogestionado según fortalezas de cada miembro
Proceso de desarrollo	Definido	Empíricos

Ciclos de vida admitidos	Cualquiera	Iterativos e incrementales
Se estima	Recursos y tiempos necesarios para el desarrollo	Alcances a implementar en iteraciones (Tamaño)
Son constantes	Los requisitos identificados en etapa de requerimientos	El equipo, recursos y tiempo de trabajo.
Planificación de tareas	Estimadas con detalle al planificar el proyecto	Estimadas con detalle al planificar la iteración

Preguntas frecuentes de la ingeniería de software (Sommerville)

Pregunta	Respuesta
¿Qué es software?	Programas de cómputo y documentación asociada. Los productos de software se desarrollan para un cliente en particular o para un mercado en general.
¿Cuáles son los atributos del buen software?	El buen software debe entregar al usuario la funcionalidad y el desempeño requeridos, y debe ser sustentable, confiable y utilizable.
¿Qué es ingeniería de software?	La ingeniería de software es una disciplina de la ingeniería que se interesa por todos los aspectos de la producción de software.
¿Cuáles son las actividades fundamentales de la ingeniería de software?	Especificación, desarrollo, validación y evolución del software.
¿Cuáles son los principales retos que enfrenta la ingeniería de software?	Se enfrentan con una diversidad creciente, demandas por tiempos de distribución limitados y desarrollo de software confiable.

¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?	Aun cuando todos los proyectos de software deben gestionarse y desarrollarse de manera profesional, existen diferentes técnicas que son adecuadas para distintos tipos de sistema. Por ejemplo, los juegos siempre deben diseñarse usando una serie de prototipos, mientras que los sistemas críticos de controles de seguridad requieren de una especificación completa y analizable para su desarrollo. Por lo tanto, no puede decirse que un método sea mejor que otro.

Cuadro comparativo Auditorías vs. Revisiones técnicas

Aspecto	**Auditoría**	**Revisión**
Para qué sirven	Para revelar qué se está haciendo en la realidad, contrastado con lo que se comprometió a hacer.	Encontrar errores en un artefacto lo antes posible, para evitar su propagación.
Roles	Auditor, auditado y gerente de calidad	Autor, moderador, anotador, lector, inspector
Beneficios	Opiniones independientes, identificar áreas de insatisfacción potencial para el cliente, asegurar que se cumplen expectativas, dar visibilidad a procesos de trabajo, etc.	Detección de errores, evitar propagación de defectos, reducción de retrabajo (costos), aumentar calidad de un artefacto, etc.
Etapas	1. Planificación 2. Ejecución 3. Análisis y Reporte de resultados 4. Seguimiento	En revisiones informales no hay pasos a seguir. En revisiones formales: 1. Planificación 2. Visión general 3. Preparación 4. Reunión de Inspección 5. Corrección 6. Seguimiento
Tipos	De proyecto, de calidad, de producto (física y funcional)	Formales e informales

Hallazgos	Buenas prácticas, observaciones y desviaciones	Errores

Cuadro comparativo CMMI por Etapas y Continuo

Aspecto	**Por etapas**	**Continuo**
Características	Acreditan determinado nivel de madurez de una organización como un todo, dependiendo de qué tantas prácticas realicen, de acuerdo a lo especificado en el modelo CMMI.	Acreditan determinado nivel de madurez de un área de proceso de una organización, dependiendo de qué tantas prácticas realicen concernientes a esa área, de acuerdo a lo que la organización quiera especializar.
Ventajas	Permiten integrar las formas de trabajo de toda la organización. Garantiza cierto nivel de calidad en toda la organización, según el nivel que se acredite. Permite sentar bases en la organización y aumentar los niveles de calidad en toda la empresa.	Permite a una organización especializarse en un área de proceso, independientemente del resto de áreas. La organización elige qué prácticas realizar, de acuerdo al área en la que se especialice.
Desventajas	Se deben realizar ciertas actividades en áreas de proceso que quizás no sean relevantes para la empresa.	Pueden quedar áreas de proceso sin demasiada calidad, debido a que se centran los esfuerzos en unas pocas. La organización como un todo se sigue considerando inmadura, si es que no implementa las actividades que plantea CMMI nivel 1 en las áreas de proceso.
Diferencias	● Niveles: 5 → 1 a 5 ● Niveles: indican madurez organizacional ● Definidos por un conjunto de áreas de proceso ● Provee una secuencia para mejorar determinadas áreas de proceso progresivamente	● Niveles: 6 → 0 a 5 ● Niveles: indican capacidad de un área de proceso ● Definidos por cada área de proceso ● Permite mejorar en un área de proceso que se desee

Respuesta a pregunta de parcial

“Realice un análisis de los principios Lean y explique las prácticas que propone Kanban para aplicar en forma concreta los principios.”

- Eliminar desperdicios
    - Visualizar el trabajo (permite visualizar el flujo de trabajo y exponer aquellas actividades que no aportan valor y aquellas que generan tiempos de espera que “traban” el flujo de trabajo)
    - Limitar el WIP (el trabajo a medias genera retrabajo, lo cual es un desperdicio. Al limitar el WIP, se evita ese trabajo a medias para concentrar esfuerzos)
- Amplificar el aprendizaje
    - Hacer explícitas las políticas (que todos los miembros tengan acceso a las políticas usadas para cada unidad de trabajo, actividades a realizar, información necesaria del dominio, etc.)
    - Evolución y mejora continua (no introducir cambios grandes, sino que visualizar el proceso de trabajo actual y aplicar mejoras en él)
- Embeber la integridad conceptual
- Postergar decisiones hasta el momento responsable
    - Limitar el WIP (particularmente el WIP de la columna backlog)
- Ver el todo
    - Visualizar el trabajo (a través del tablero permite que todos los miembros observen todo el trabajo que se hace, el sentido de cada una de las tareas y como estas conforman el flujo de trabajo como un todo que desencadena en la satisfacción de los clientes)
- Dar poder al equipo
    - Haciendo explícitas las políticas y conformando un sistema de trabajo pull, donde cada miembro toma el trabajo cuando esté en condiciones de realizarlo, y no se lo impone nadie.
- Mejorar colaborativamente
- Entregar lo antes posible
    - Gestionar el flujo de trabajo (permite ver de principio a fin los pasos que se deben seguir para lograr cumplir con las necesidades del cliente y hacer aquellos ajustes que sean necesarios para entregar el mayor valor posible, en el menor tiempo posible)